FROM THEORY TO PRACTICE: ADVANCING MULTI-ROBOT PATH PLANNING ALGORITHMS AND APPLICATIONS

By

TENG GUO

A dissertation submitted to the

School of Graduate Studies

Rutgers, The State University of New Jersey

In partial fulfillment of the requirements

For the degree of

Doctor of Philosophy

Graduate Program in Department of Computer Science

Written under the direction of

Jingjin Yu

And approved by

_____

_____

_____

_____

New Brunswick, New Jersey

October 2025

ABSTRACT OF THE DISSERTATION

From Theory to Practice: Advancing Multi-Robot Path Planning Algorithms and Applications

by TENG GUO

Dissertation Director: Jingjin Yu

The labeled MRPP problem, despite its various setups and variations, generally involves routing a set of mobile robots from a start configuration to a goal configuration as efficiently as possible while avoiding collisions. In recent years, MRPP has garnered significant attention, with notable advancements in both solution quality and efficiency. However, due to its inherent complexity and critical role in industrial applications, there remains a continuous drive for improved performance.

This dissertation introduces methods with provable guarantees and effective heuristics aimed at advancing MRPP algorithms, particularly in terms of scalability and optimality. The first part of this dissertation presents a theoretical study of MRPP on grid graphs with high robot density. Specifically, we investigate MRPP on dense two-dimensional grids, with practical applications in areas such as warehouse automation and parcel sorting. While previous methods provide polynomial-time solutions with $O(1)$ optimality guarantees, their high constant factors limit practical applicability. In contrast, we introduce the Rubik Table method, which achieves $(1 + \delta)$-makespan optimality, where $\delta \in (0, 0.5]$, for large grids. Our approach supports up to $\frac{m_1 m_2}{2}$ robots and integrates effective heuristics, enabling near-optimal makespan solutions in minutes for instances involving tens of thousands of robots. This establishes a new theoretical benchmark for scalable, provably optimal multi-robot

routing algorithms.

The second part of this dissertation shifts focus to real-world applications of MRPP. First, we consider the design of optimal layouts for well-structured infrastructures, which are essential for traditional autonomous warehouses and parking systems. The effectiveness of these systems heavily depends on efficient space utilization, minimizing robot congestion, and optimizing task execution time. To further enhance operational efficiency, we propose a puzzle-based multi-robot parking system that enables high-density storage and retrieval of autonomous vehicles. By leveraging combinatorial optimization techniques, we design algorithms that ensure smooth transitions between vehicle placements while avoiding unnecessary movements and deadlocks. We analyze different retrieval strategies, demonstrating how our approach can significantly improve throughput and reduce waiting times compared to conventional parking methodologies. Additionally, we investigate MRPP when robots are modeled as autonomous Reeds-Shepp cars, which introduce additional constraints due to nonholonomic motion. These vehicles require smooth, continuous paths that respect turning radius limitations, making traditional grid-based planning techniques insufficient. By extending state-of-the-art MRPP algorithms, we develop novel heuristics and trajectory optimization techniques to address these constraints. Our approach incorporates motion primitives and local path smoothing, ensuring both feasibility and efficiency in real-world autonomous navigation scenarios. Through extensive simulations and real-world experiments, we validate our methods, demonstrating their applicability in urban autonomous driving, robotic convoy coordination, and automated transport systems.

ACKNOWLEDGMENTS

First and foremost, I would like to express my heartfelt gratitude to my advisor, Prof. Jingjin Yu, director of the Algorithmic Robotics and Control Lab (ARCL). Throughout my Ph.D. journey, he has been an exceptional advisor, providing invaluable guidance in research, supporting my studies through funding, and helping me navigate academic challenges. His professional advice, especially during the difficult times of my final semester, was a source of immense support. It has been both a privilege and a joy to work on algorithmic robotics and engage in numerous exciting projects under his mentorship.

I am deeply grateful to my committee members, Prof. Kostas Bekris, Prof. Abdeslam Boularias and Prof. Bo Yuan, for their guidance and support. Prof. Bekris held numerous insightful talks, from which I learned a great deal about robotics. Prof. Boularias's course, Robotics Learning, provided me with a strong foundation in robot learning. Their valuable feedback and encouragement, especially during my qualification exam, were instrumental in shaping my research journey.

My labmates at ARCL have been an integral part of this journey. I am deeply grateful to Shuai D. Han, Si Wei Feng, Kai Gao, Baichuan Huang, Wei Tang, Zihe Ye, Duo Zhang, Kowndinya Boyalakuntla, Tzvika Geft, and Justin Yu for their camaraderie and support.

Pursuing a Ph.D. is a demanding endeavor, and I am immensely thankful to my friends who made my graduate school life more enjoyable and meaningful. I would like to acknowledge Shiyang Lu, Haonan Chang, Junchi Liang, Rui Wang, Aravind Sivaramakrishnan, Yinglong Miao, Yuhan Liu, , and Xinyu Zhang for their unwavering friendship and encouragement.

Special thanks go to my collaborators in the League of Robot Runners, including Zhe Chen, Shaohung Chan, Han Zhang, Yue Zhang and Prof. Daniel Harabor. It has been an honor and a pleasure to work and learn alongside such talented individuals.

The pandemic, which spanned the first three years of my Ph.D., brought unprecedented

challenges to our lives and work. Despite these difficulties, we have been fortunate to return to normalcy. I would like to extend my gratitude to the selfless medical workers and volunteers who dedicated themselves to combating the pandemic and supporting communities worldwide.

Finally, I wish to express my deepest appreciation to my parents for their unconditional love, care, and support throughout my life. Their belief in me has been a constant source of strength and motivation.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

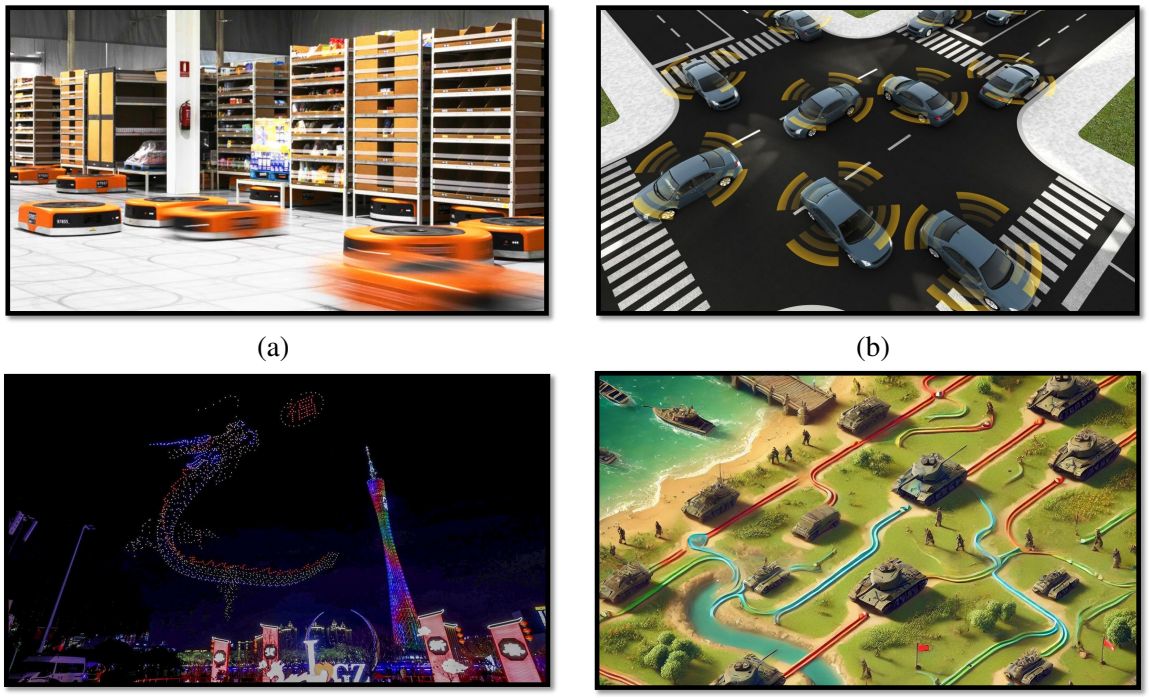
CHAPTER 1
INTRODUCTION

Cooperative path and motion planning for multiple mobile robots is a fundamental problem in robotics. Its complexity has been well studied [1, 2, 3], and it has numerous applications, including rigid body assembly [4], evacuation planning [5], formation control [6, 7], collaborative localization [8], micro-droplet manipulation [9], object transportation [10], human-robot interaction [11], search and rescue [12], coverage and surveillance [13], and notably, warehouse automation [14] (see Figure 1.1). Efficient algorithms for different settings can significantly enhance process efficiency.

In Figure 1.1, we highlight four real-world Multi-Robot Path Planning (MRPP) applications: (a) Autonomous mobile robots transporting goods in an Amazon fulfillment center; (b) Self-driving cars navigating an intersection with intelligent traffic control; (c) Drone swarms creating dynamic aerial formations in a night show; (d) Multi-agent pathfinding in real-time strategy (RTS) games. In this dissertation, we study the labeled MRPP on its multiple different variations. Typically, in an MRPP problem instance, there is a set of mobile robots in a workspace. Given the start and goal configurations of the robots and their permissible motion primitives, the objective is to route the robots from their start configuration to the goal configuration, collision-free.

Here, collision-free indicates that the robots must avoid colliding with each other and obstacles in the environment. The term labeled indicates that the robots are not interchangeable, i.e., each robot has a distinctive goal that it must reach. After a feasible motion plan is generated, the solution quality is often measured in terms of plan execution time and robot travel distance, which we usually want to minimize. MRPP and its variations have been actively studied for decades [15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26].

On one hand, many algorithms were proposed, and state-of-the-art methods perform



(a) (b)

(c) (d)

Figure 1.1: (a) Autonomous mobile robots navigate in Amazon autonomous fufillment warehouse, transporting goods between shelves; (b) Self-driving cars at an intersection, with intelligent transportation control; (c) Drones form a dragon shape in the night sky; (d) Pathfinding for multiple units in RTS games

well under certain metrics and situations. On the other hand, MRPP, in its many forms, has been shown to be computationally hard [1, 2, 3], which indicates that it is highly unlikely for any algorithm to consistently find optimal solutions for large and challenging problem instances. Observing the discrepancy between the current MRPP algorithm performance and the demand for performance in various MRPP applications, the statement of this thesis is presented as:

Novel algorithms and heuristics can significantly improve the time efficiency and solution quality for solving different Multi-Robot Motion Planning variations, delivering more concrete and promising theoretical guarantees as well as higher throughput in Multi-Robot Motion Planning applications.

To address existing research gaps in MRPP, this dissertation advances both the theoretical foundations and practical applications of MRPP. We begin by examining one of its

most fundamental variants: MRPP on graphs, also known as Multi-Agent Path Finding (MAPF). This extensively studied problem models the shared workspace as a grid graph. Prior research has established that optimally solving MRPP on general graphs with respect to makespan and sum-of-costs objectives is NP-hard [3, 27]. More recent studies further demonstrate that MRPP remains NP-hard even on empty grids without obstacles [28].

Although polynomial-time algorithms exist that achieve constant-factor time optimality for MRPP on grids [28, 29], their high constant factors often lead to significantly suboptimal solutions. Motivated by this limitation, we propose a novel polynomial-time algorithm inspired by prior work on the Grid Rearrangement Algorithm [30]. We show that our algorithm runs in polynomial time while achieving a lower constant-factor optimality ratio, resulting in more efficient solutions.

For MRPP on $m_1 \times m_2$ grids with full (100%) robot density, we integrate the Grid Rearrangement Algorithm at the high level with a novel parallel odd-even sorting algorithm for low-level shuffling. This approach yields a polynomial-time algorithm that guarantees a makespan of $4(m_1 + 2m_2)$. For MRPP on grids with $1/2$ and $1/3$ robot densities, we introduce the highway and line-merge motion primitives, enabling us to achieve an asymptotic makespan optimality ratio of $1 + \frac{m_2}{m_1+m_2}$. Furthermore, we extend our algorithm to high-dimensional grids.

MRPP on grids has numerous real-world applications, including warehouse automation, drone light shows, and automated parking systems. In many modern warehouses, infrastructure is designed to be *well-formed*, meaning that mobile robots can rest at designated endpoints without obstructing others. Such structured layouts are prevalent in automated warehouses and parking lots, as they significantly simplify multi-robot path planning. Prior work [31] has established that prioritized planning is complete for well-formed MRPP on graphs.

In this dissertation, we systematically investigate well-formed layouts by modeling the problem as an optimization task: identifying the *largest well-connected vertex set*. Efficient

space utilization is crucial in dense, high-traffic environments, and finding the largest well-connected vertex set plays a vital role in optimizing throughput. We demonstrate that this problem is NP-hard and propose efficient algorithms to compute maximal and largest well-connected sets.

Beyond structured layouts, space efficiency has become an increasingly pressing concern, particularly in densely populated urban areas where land is scarce and expensive. While well-formed infrastructures simplify pathfinding, they inherently limit space utilization. To address this tradeoff, we introduce a *puzzle-based automated parking and retrieval system* capable of supporting near 100% robot density. To ensure efficient vehicle storage and retrieval, we develop novel MRPP-based algorithms tailored to this system.

Furthermore, we extend our study beyond traditional grid-based MRPP, which assumes point-like robots and neglects real-world motion constraints. We explore a continuous variant of MRPP where robots have non-trivial kinematics and occupy space. Specifically, we consider environments with *Ackermann-steering car-like robots*, a crucial step toward enabling autonomous vehicle coordination in future smart cities. For this problem, we propose both an *enhanced search-based centralized algorithm* and a *decentralized prioritized algorithm*. Our centralized approach exhibits superior scalability, while the decentralized method achieves higher success rates and is more resilient to deadlocks.

A comprehensive literature review is provided in chapter 2. Our five key contributions are systematically presented in chapter 3 to chapter 6, followed by concluding remarks in chapter 7.

CHAPTER 2
BACKGROUND

The field of MRPP has been extensively studied, particularly in graph-theoretic settings where the state space is discrete [1, 32, 33, 34]. The discrete graph-based MRPP version is also known as MAPF in AI communities. MRPP has immense practical value, especially in applications such as e-commerce [14, 35, 36, 37], which are poised for continued growth [38, 39, 40, 41]

2.1 Computational Intractability

While the feasibility of MRPP has been positively established—being polynomial-time decidable on undirected graphs [42] and on strongly biconnected directed graphs [43]—finding optimal solutions remains a significant challenge. Computing time- or distance-optimal solutions is NP-hard across various settings, including general graphs [44, 27, 3], planar graphs [45, 46], directed graphs [47], and regular grids [28, 48, 49]. Moreover, it has been proven that finding a bounded-suboptimal solution whose optimality is within a factor of $\frac{4}{3}$ is intractable [50].

Despite its computational hardness, a variety of algorithmic solutions have been proposed to address MRPP. These solutions can be broadly classified as optimal or suboptimal.

2.2 Optimal Solvers

Optimal solvers leverage either combinatorial search or problem reduction techniques. Search-based methods explore the joint solution space and include algorithms like EPEA*[51], ICTS[52], CBS [53], M*[23], and others. Reduction-based methods reformulate MRPP as another problem, such as SAT[54], answer set programming [55], or integer linear pro-

gramming (ILP) [33]. Although effective, optimal solvers often exhibit limited scalability due to the inherent computational complexity of MRPP.

2.3 Suboptimal Solvers

To balance efficiency and solution quality, various suboptimal solvers have been developed. Unbounded solvers, such as Push-and-Swap [56], Push-and-Rotate [57], Windowed Hierarchical Cooperative A*[58], and BIBOX[59], efficiently compute feasible solutions but often at the cost of optimality.

Bounded-suboptimal search-based algorithms, including ECBS [60], EECBS [61], and their variants, guarantee solutions within a specified suboptimality bound, significantly improving scalability. Other approaches, such as DDM [62], PIBT [63], LNS [64], LA-CAM [65], and PBS [66], achieve impressive runtime performance while maintaining relatively good solution quality.

Additionally, learning-based methods [67, 68, 69, 70, 71] demonstrate strong scalability, particularly in sparse environments. However, these methods lack completeness and optimality guarantees, and they tend to underperform in dense instances. Recent advances include $O(1)$-approximation algorithms [29, 28, 72], which offer theoretical guarantees but face practical limitations due to large constant factors. Parallel techniques aimed at accelerating existing MRPP algorithms have also been investigated [73, 74].

2.4 Beyond Graph-Based MRPP—Applications

2.4.1 Well-Formed Infrastructures

The concept of well-connected vertex sets draws inspiration from well-formed infrastructures [75]. In such infrastructures, including parking lots and fulfillment warehouses [14], endpoints are designed to allow multiple robots (or vehicles) to navigate between them without causing complete obstructions. These endpoints may represent parking slots, pickup

stations, or delivery stations. Many real-world infrastructures are intentionally constructed in this manner to facilitate efficient pathfinding and collision avoidance.

Planning collision-free paths for robots to move from their start positions to target destinations, commonly referred to as multi-robot path planning, is an NP-hard problem to solve optimally [27, 3]. In practical applications, prioritized planning [15, 58] is among the most widely used approaches for finding collision-free paths. This method sequences robots by assigning them priorities and plans paths sequentially, ensuring that each robot avoids collisions with higher-priority robots. While effective in less cluttered settings, prioritized planning is generally incomplete and prone to deadlocks in dense environments.

Previous studies [75, 66] have shown that prioritized planning with arbitrary robot ordering can guarantee deadlock-free solutions in well-formed environments. In contrast, for problems that do not meet well-formed criteria, solutions may still be found using prioritized planning with a carefully selected priority order, as proposed in [66]. However, identifying such an order can be computationally expensive or even infeasible.

To the best of our knowledge, no prior work has investigated how to efficiently design well-formed layouts that maximize workspace utilization.

2.4.2 Puzzle-based Autonomous Parking System

Efficient high-density parking has been widely studied. Centralized systems for self-driving vehicles [76, 77, 78] increase capacity by up to 50% but face complex retrieval challenges due to blockages and maneuverability requirements. For non-self-driving vehicles, robotic valet-based systems like the Puzzle-Based Storage system [79] are promising. PBS uses movable storage units (e.g., AGVs) on a grid with at least one empty cell (escort) for maneuvering, akin to solving the NP-hard 15-puzzle [80]. While optimal algorithms exist for single-vehicle retrieval [79, 81], average retrieval times remain high compared to aisle-based solutions. To improve efficiency, incorporating more escorts and I/O ports can enable simultaneous retrievals, leveraging advanced MRPP algorithms [82].

MRPP addresses finding collision-free paths for multiple robots with unique start and goal positions. Solving MRPP optimally [59, 3] is NP-hard. Optimal solvers [33, 53] are unsuitable for large problems due to scalability issues. Bounded suboptimal solvers [60] improve scalability but struggle in high-density settings. Polynomial-time methods [56] offer scalability at the cost of solution quality, while $O(1)$ time-optimal algorithms [83, 72] focus on minimizing makespan, making them less suitable for dynamic environments.

2.4.3 Multi-Robot Path Planning with Shape and Kinematic Constraints

With the growing interest in autonomous driving, path planning for car-like robots has attracted increasing attention [84, 82]. Many existing MRPP algorithms assume a holonomic point-agent model, which ignores robot shape and kinematic constraints, limiting their applicability to real-world scenarios. Several approaches have been proposed to bridge this gap. Graph-based solvers, such as CL-CBS for car-like robots [85], prioritized trajectory optimization methods [86], and sampling-based techniques [87] effectively handle these additional constraints. However, these methods are centralized, making them difficult to scale for large robot fleets and unsuitable for decentralized applications like self-driving cars. Decentralized approaches, such as B-ORCA [88] and ϵCCA [89], offer faster computation but often lack guarantees of successful navigation, particularly in dense environments.

CHAPTER 3

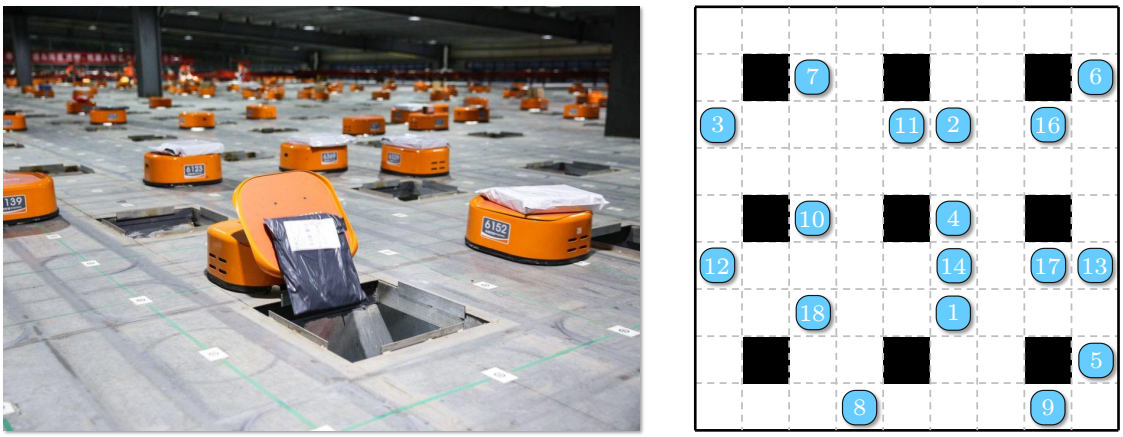
EXPECTED 1.X OPTIMAL MULTI-ROBOT PATH PLANNING

3.1 Introduction

Multi-Robot Path Planning on graphs is NP-hard to solve optimally, even on grids, suggesting no polynomial-time algorithms can compute exact optimal solutions for them. This raises a natural question: How optimal can polynomial-time algorithms reach? Whereas algorithms for computing constant-factor optimal solutions have been developed, the constant factor is generally very large, limiting their application potential. In this chapter, among other breakthroughs, we propose the first low-polynomial-time MRPP algorithms delivering 1-1.5 (resp., 1-1.67) asymptotic makespan optimality guarantees for 2D (resp., 3D) grids for random instances at a very high $1/3$ robot density, with high probability. Moreover, when regularly distributed obstacles are introduced, our methods experience no performance degradation. These methods generalize to support 100% robot density.

Regardless of the dimensionality and density, our high-quality methods are enabled by a unique hierarchical integration of two key building blocks. At the higher level, we apply the labeled Grid Rearrangement Algorithm (GRA), capable of performing efficient reconfiguration on grids through row/column shuffles. At the lower level, we devise novel methods that efficiently simulate row/column shuffles returned by GRA. Our implementations of GRA-based algorithms are highly effective in extensive numerical evaluations, demonstrating excellent scalability compared to other SOTA methods. For example, in 3D settings, GRA-based algorithms readily scale to grids with over $370,000$ vertices and over $120,000$ robots and consistently achieve conservative makespan optimality approaching 1.5, as predicted by our theoretical analysis. We examine MRPP [34] on two- and

three-dimensional grids with potentially regularly distributed obstacles (see Figure 3.1). The main objective of MRPP is to find collision-free paths for routing many robots from a start configuration to a goal configuration. In practice, solution optimality is of key importance, yet optimally solving MRPP is NP-hard [27, 3], even in planar settings [45] and on obstacle-less grids [28]. MRPP algorithms apply to a diverse set of practical scenarios, including formation reconfiguration [90], object transportation [91], swarm robotics [7, 92], to list a few. Even when constrained to grid-like settings, MRPP algorithms still find many large-scale applications, including warehouse automation for general order fulfillment [14], grocery order fulfillment [35], and parcel sorting [36].



(a) (b)

Figure 3.1: (a) Real-world parcel sorting system (at JD.com) using many robots on a large grid-like environment with holes for dropping parcels; (b) A snapshot of a similar MRPP instance our methods can solve in polynomial-time with provable optimality guarantees. In practice, our algorithms scale to 2D maps of size over 450×300, supporting over 45K robots and achieving better than 1.5-optimality (see, e.g., Figure 3.14). They scale further in 3D settings.

Motivated by applications including order fulfillment and parcel sorting, we focus on grid-like settings with extremely high robot density, i.e., with $\frac{1}{3}$ or more graph vertices occupied by robots. Whereas recent studies [29, 28] show such problems can be solved in polynomial time yielding constant-factor optimal solutions, the constant factor is prohibitively high ($\gg 1$) to be practical. In this research, we tear down this MRPP planning barrier, achieving $(1 + \delta)$-makespan optimality asymptotically with high probability in

polynomial time where $\delta \in (0, 0.5]$ for 2D grids and $\delta \in (0, \frac{2}{3}]$ for 3D grids.

This study's primary theoretical and algorithmic contributions are outlined below and summarized in Table 3.1. Through judicious applications of a novel row/column-shuffle-based global Grid Rearrangement (GR) method called the Rubik Table Algorithm [93][1], employing many classical algorithmic techniques, and combined with careful analysis, we establish that in *low polynomial* time:

- For $m_1 m_2$ robots on a 2D $m_1 \times m_2$ grid, $m_1 \geq m_2$, i.e., at maximum robot density, Grid Rearrangement for MRPP (GRM) (Grid Rearrangement for MRPP) computes a solution for an arbitrary MRPP instance under a makespan of $4m_1 + 8m_2$; For $m_1 m_2 m_3$ robots on a 3D $m_1 \times m_2 \times m_3$ grids, $m_1 \geq m_2 \geq m_3$, GRM computes a solution under a makespan of $4m_1 + 8m_2 + 8m_3$;

- For $\frac{1}{3}$ robot density with uniformly randomly distributed start/goal configurations, Grid Rearrangement with Highways (GRH) computes a solution with a makespan of $m_1 + 2m_2 + o(m_1)$ on 2D grids and $m_1 + 2m_2 + 2m_3 + o(m_1)$ on 3D grids, with high probability. In contrast, such an instance has a minimum makespan of $m_1 + m_2 - o(m_1)$ for 2D grids and $m_1 + m_2 + m_3 - o(m_1)$ for 3D grids with high probability. This implies that, as $m_1 \to \infty$, an asymptotic optimality guarantee of $\frac{m_1+2m_2}{m_1+m_2} \in (1, 1.5]$ is achieved for 2D grids and $\frac{m_1+2m_2+2m_3}{m_1+m_2+m_3} \in (1, \frac{5}{3}]$ for 3D grids, with high probability;

- For $\frac{1}{2}$ robot density, the same optimality guarantees as in the $\frac{1}{3}$ density setting can be achieved using Grid Rearrangement with Linear Merge (GRLM) (Grid Rearrangements with Line Merge), using a merge-sort inspired robot routing heuristic;

- Without modification, GRH achieves the same $\frac{m_1+2m_2}{m_1+m_2}$ optimality guarantee on 2D grids with up to $\frac{m_1 m_2}{9}$ regularly distributed obstacles together with $\frac{2m_1 m_2}{9}$ robots (e.g., Figure 3.1 (b)). Similar guarantees hold for 3D settings;

[1] It was noted in [93] that their Rubik Table Algorithm can be applied to solve MRPP; we do not claim this as a contribution of our work. Rather, our contribution is to dramatically bring down the constant factor through a combination of meticulous algorithm design and careful analysis.

- For robot density up to $\frac{1}{2}$, for an arbitrary (i.e., not necessarily random) instance, a solution can be computed with a makespan of $3m_1 + 4m_2 + o(m_1)$ on 2D grids and $3m_1 + 4m_2 + 4m_3 + o(m_1)$ on 3D grids.
- With minor numerical updates to the relevant guarantees, the above-mentioned results also generalize to grids of arbitrary dimension $k \geq 4$.

Moreover, we have developed many effective and principled heuristics that work together with the GRA-based algorithms to further reduce the computed makespan by a significant margin. These heuristics include (1) An optimized matching scheme, required in the application of the grid rearrangement algorithm, based on Linear Bottleneck Assignment (LBA), (2) A polynomial time path refinement method for compacting the solution paths to improve their quality. As demonstrated through extensive evaluations, our methods are highly scalable and capable of tackling instances with tens of thousands of densely packed robots. Simultaneously, the achieved optimality matches/exceeds our theoretical prediction. With the much-enhanced scalability, our approach unveils a promising direction toward the development of practical, provably optimal multi-robot routing algorithms that run in low polynomial time.

Algorithms	GRM 2×3	GRM 4×2	GRLM	GRH
Applicable Density	≤ 1	≤ 1	$\leq \frac{1}{2}$	$\leq \frac{1}{3}$
Makespan Upperbound	$7(2m_2 + m_1)$	$4(2m_2 + m_1)$	$4m_2 + 3m_1$	$4m_2 + 3m_1$
Asymptotic Makespan	$7(2m_2 + m_1)$	$4(2m_2 + m_1)$	$2m_2 + m_1 + o(m_1)$	$2m_2 + m_1 + o(m_1)$
Asymptotic Optimality	$7(1 + \frac{m_2}{m_1+m_2})$	$4(1 + \frac{m_2}{m_1+m_2})$	$1 + \frac{m_2}{m_1+m_2}$	$1 + \frac{m_2}{m_1+m_2}$

Table 3.1: Summary results of the algorithms proposed in this chapter. All algorithms operate on grids of dimensions $m_1 \times m_2$. GRH, GRM, and GRLM are all further derived from Grid Rearrangement Algorithm in 2D (GRA2D), each a variant characterized by distinct low-level shuffle movements. Specifically for GRM, the low-level shuffle movement is tailored for facilitating full-density robot movement

In this chapter, we provide a unified treatment of the problem that streamlined the description of GRA-based algorithms under 2D/3D/kD settings for journal archiving, and we further introduce many new results, including (1) The baseline GRM method (for the full-density case) is significantly improved with a much stronger makespan optimality guaran-

tee, by a factor of $\frac{7}{4}$; (2) A new and general polynomial-time path refinement technique is developed that significantly boosts the optimality of the plans generated by all GRA-based algorithms; (3) Complete and substantially refined proofs are provided for all theoretical developments in the paper; and (4) The evaluation section is fully revamped to reflect the updated theoretical and algorithmic development.

Related work. Literature on multi-robot path and motion planning [1, 32] is expansive; here, we mainly focus on graph-theoretic (i.e., the state space is discrete) studies [33, 34]. As such, in this paper, MRPP refers explicitly to graph-based multi-robot path planning. Whereas the feasibility question has long been positively answered [42], the same cannot be said for finding optimal solutions, as computing time- or distance-optimal solutions are NP-hard in many settings, including on general graphs [44, 27, 3], planar graphs [45, 46], and regular grids [28] that is similar to the setting studied in this chapter.

Nevertheless, given its high utility, especially in e-commerce applications [14, 35, 36] that are expected to grow significantly [38, 39], many algorithmic solutions have been proposed for optimally solving MRPP. Among these, combinatorial-search-based solvers [94] have been demonstrated to be fairly effective. MRPP solvers may be classified as being optimal or suboptimal. Reduction-based optimal solvers solve the problem by reducing the MRPP problem to another problem, e.g., SAT [54], answer set programming [55], integer linear programming (ILP) [33]. Search-based optimal MRPP solvers include EPEA* [51], ICTS [52], CBS [53], M* [23], and many others.

Due to the inherent intractability of optimal MRPP, optimal solvers usually exhibit limited scalability, leading to considerable interest in suboptimal solvers. Unbounded solvers like push-and-swap [56], push-and-rotate [57], windowed hierarchical cooperative A [58], BIBOX [59], all return feasible solutions very quickly, but at the cost of solution quality. Balancing the running time and optimality is one of the most attractive topics in the study of MRPP. Some algorithms emphasize the scalability without sacrificing as much optimality, e.g., ECBS [60], DDM [62], PIBT [63], PBS [66]. There are also learning-based

solvers [67, 68, 69] that scale well in sparse environments. Effective orthogonal heuristics have also been proposed [73]. Recently, $O(1)$-approximate or constant factor time-optimal algorithms have been proposed, e.g. [29, 28, 72], that tackle highly dense instances. However, these algorithms only achieve a low-polynomial time guarantee at the expense of very large constant factors, rendering them theoretically interesting but impractical.

In contrast, with high probability, our methods run in low polynomial time with provable $1.x$ asymptotic optimality. To our knowledge, this paper presents the first MRPP algorithms to simultaneously guarantee polynomial running time and $1.x$ solution optimality, which works for any dimension ≥ 2.

Organization. The rest of the paper is organized as follows. In section 3.2, starting with 2D grids, we provide a formal definition of graph-based MRPP, and introduce the Grid Rearrangement problem and the associated baseline algorithm (GRA) for solving it. GRM, a basic adaptation of GRA for MRPP at maximum robot density which ensures a makespan upper bound of $4m_1 + 8m_2$, is described in section 3.3. An accompanying lower bound of $m_1 + m_2 - o(m_1)$ for random MRPP instances is also established. In section 3.4 we introduce GRH for $\frac{1}{3}$ robot density achieving a makespan upper bound of $m_1 + 2m_2 + o(m_1)$. Obstacle support is then discussed. We continue to show how $\frac{1}{2}$ robot density may be supported with similar optimality guarantees. In section 3.6, we generalize the algorithms to work on 3+D grids. In section 3.5, we introduce multiple optimality-boosting heuristics to significantly improve the solution quality for all variants of Grid Rearrangement-based solvers. We thoroughly evaluate the performance of our methods in section 3.7 and conclude with section 3.8.

3.2 Preliminaries

3.2.1 Multi-Robot Path Planning on Graphs

Consider an undirected graph $\mathcal{G}(V, E)$ and n robots with start configuration $S = \{s_1, \ldots, s_n\} \subseteq V$ and goal configuration $G = \{g_1, \ldots, g_n\} \subseteq V$. A *path* for robot i is a map $P_i : \mathbb{N} \to V$

where $\mathbb{N}$ is the set of non-negative integers. A feasible P_i must be a sequence of vertices that connects s_i and g_i: (1) $P_i(0) = s_i$; (2) $\exists T_i \in \mathbb{N}$, s.t. $\forall t \geq T_i, P_i(t) = g_i$; (3) $\forall t > 0$, $P_i(t) = P_i(t-1)$ or $(P_i(t), P_i(t-1)) \in E$. A path set $\{P_1, \ldots, P_n\}$ is feasible iff each P_i is feasible and for all $t \geq 0$ and $1 \leq i < j \leq n$, $P_i(t) = P$, it does not happen that: (1) $P_i(t) = P_j(t)$; (2) $P_i(t) = P_j(t+1) \wedge P_j(t) = P_i(t+1)$.

We work with $\mathcal{G}$ being 4-connected grids in 2D and 6-connected grids in 3D, aiming to mainly minimize the *makespan*, i.e., $\max_i\{|P_i|\}$ (later, a sum-of-cost objective is also briefly examined). Unless stated otherwise, $\mathcal{G}$ is assumed to be an $m_1 \times m_2$ grid with $m_1 \geq m_2$ in 2D and $m_1 \times m_2 \times m_3$ grid with $m_1 \geq m_2 \geq m_3$ in 3D. As a note, "randomness" in this paper always refers to uniform randomness. The version of MRPP we study is sometimes referred to as *one-shot* MRPP [34]. We mention our results also translate to guarantees on the life-long setting [34], briefly discussed in section 3.8.

3.2.2 Grid Rearrangement Problem (GRP)

The Grid Rearrangement Problem (GRP) (first proposed in [93] as the Rubik Table problem) formalizes the task of carrying out globally coordinated object reconfiguration operations on lattices, with many interesting applications. The problem has many variations; we start with the 2D form, to be generalized to higher dimensions later.

Problem 1 (Grid Rearrangement Problem in 2D (GRP2D) [93]**).** *Let M be an $m_1(row) \times m_2(column)$ table, $m_1 \geq m_2$, containing $m_1 m_2$ items, one in each table cell. The items have m_2 colors with each color having a multiplicity of m_1. In a* shuffle *operation, the items in a single column or a single row of M may be permuted in an arbitrary manner. Given an arbitrary configuration X_I of the items, find a sequence of shuffles that take M from X_I to the configuration where row i, $1 \leq i \leq m_1$, contains only items of color i. The problem may also be* labeled*, i.e., each item has a unique label in $1, \ldots, m_1 m_2$.*

A key result [93], which we denote here as the (labeled) Grid Rearrangement Algorithm in 2D (GRA2D), shows that a colored GRP2D can be solved using m_2 column shuffles fol-

lowed by m_1 row shuffles, implying a low-polynomial time computation time. Additional m_1 row shuffles then solve the labeled GRP2D. We illustrate how GRA2D works on an $m_1 \times m_2$ table with $m_1 = 4$ and $m_2 = 3$ (Figure 3.2); we refer the readers to [93] for details. The main supporting theoretical result is given in Theorem 3.2.2, which depends on Theorem 3.2.1.

Theorem 3.2.1 (Hall's Matching theorem with parallel edges [95, 93])**.** *Let B be a d-regular ($d > 0$) bipartite graph on $n + n$ nodes, possibly with parallel edges. Then B has a perfect matching.*

Theorem 3.2.2 (**Grid Rearrangement Theorem [93]).** *An arbitrary Grid Rearrangement problem on an $m_1 \times m_2$ table can be solved using $m_1 + m_2$ shuffles. The labeled Grid Rearrangement problem can be solved using $2m_1 + m_2$ shuffles.*

GRA2D operates in two phases. In the first, a bipartite graph $B(T, R)$ is constructed based on the initial table where the bipartite set T are the colors/types of items, and the set R the rows of the table (see Figure 3.2 (b)). An edge is added to B between $t \in T$ and $r \in R$ for every item of color t in row r. From $B(T, R)$, which is a *regular bipartite graph*, m_2 *perfect matchings* can be computed as guaranteed by Theorem 3.2.1. Each matching, containing m_1 edges, dictates how a table column should look like after the first phase. For example, the first set of matching (solid lines in Figure 3.2 (b)) says the first column should be ordered as yellow, cyan, red, and green, shown in Figure 3.2 (c). After all matchings are processed, we get an intermediate table, Figure 3.2 (c). Notice each row of Figure 3.2 (a) can be shuffled to yield the corresponding row of Figure 3.2 (c); a key novelty of GRA2D. After the first phase of m_1 row shuffles, the intermediate table (Figure 3.2 (c)) can be rearranged with m_2 column shuffles to solve the colored GRP2D (Figure 3.2 (d)). Another m_1 row shuffles then solve the labeled GRP2D (Figure 3.2 (e)). It is also possible to perform the rearrangement using m_2 column shuffles, followed by m_1 row shuffles, followed by another m_2 column shuffles.

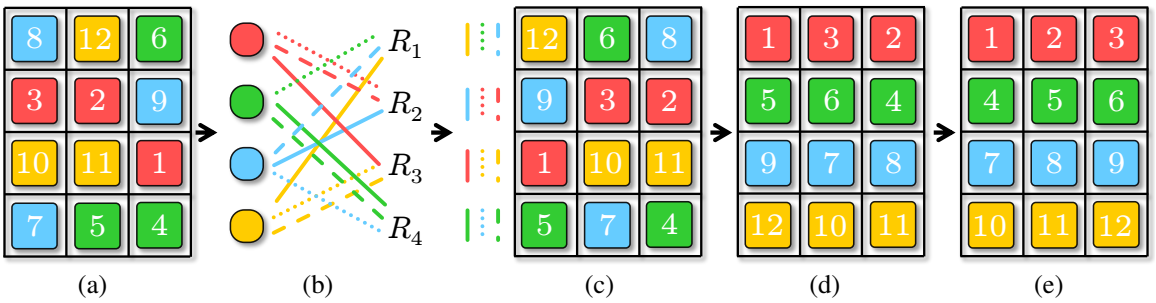


Figure 3.2: Illustration of applying the 11 *shuffles*: (a) The initial 4×3 table with a random arrangement of 12 items that are colored and labeled. The labels are consistent with the colors; (b) The constructed bipartite graph. It contains 3 perfect matchings, determining the 3 columns in (c); only color matters in this phase; (c) Applying 4 row shuffles to (a), according to the matching results, leads to an intermediate table where each column has one color appearing exactly once; (d) Applying 3 column shuffles to (c) solves a colored GRP2D; (e) 4 additional row shuffles fully sort the labeled items.

GRA2D runs in $O(m_1 m_2 \log m_1)$ (notice that this is nearly linear with respect to $n = m_1 m_2$, the total number of items) expected time or $O(m_1^2 m_2)$ deterministic time.

3.3 Solving MRPP at Maximum Density Leveraging RTA2D

The built-in global coordination capability of GRA2D naturally applies to solving makespan-optimal MRPP. Since GRA2D only requires *three rounds* of shuffles and each round involves either parallel row shuffles or parallel column shuffles, if each round of shuffles can be realized with makespan proportional to the size of the row/column, then a makespan upper bound of $O(m_1 + m_2)$ can be guaranteed. This is in fact achievable even when all of $\mathcal{G}$'s vertices are occupied by robots, by recursively applying a *labeled line shuffle algorithm* [29], which can arbitrarily rearrange a line of m robots embedded in a grid using $O(m)$ makespan.

Lemma 3.3.1 (Basic Simulated Labeled Line Shuffle [29])**.** *For m labeled robots on a straight path of length m, embedded in a 2D grid, they may be arbitrarily ordered in $O(m)$ steps. Moreover, multiple such reconfigurations can be performed on parallel paths within the grid.*

The key operation is based on a localized, 3-step pair swapping routine, shown in Figure 3.3. For more details on the line shuffle routine, see [29].

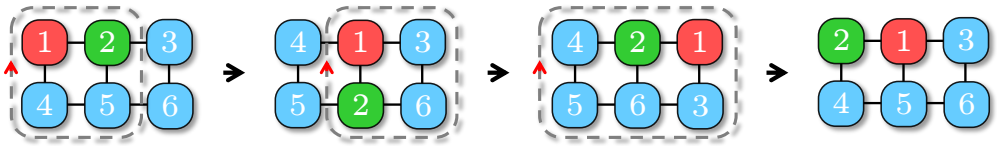


Figure 3.3: On a 2×3 grid, swapping two robots may be performed in three steps with three cyclic rotations.

However, the basic simulated labeled line-shuffle algorithm has a large constant factor. Each shuffle takes 3 steps; doing arbitrary shuffling of a 2×3 takes $20+$ steps in general. The constant factor further compounds as we coordinate the shuffles across multiple lines. Borrowing ideas from *parallel odd-even sort* [96], we can greatly reduce the constant factor in Lemma 3.3.1. We will do this in several steps. First, we need the following lemma. By an *arbitrary horizontal swap* on a grid, we mean an arbitrary reconfiguration of a grid row.

Lemma 3.3.2. *It takes at most* 7*,* 6*,* 6*,* 7*,* 6*, and* 8 *steps to perform arbitrary combinations of arbitrary horizontal swaps on* 3×2*,* 4×2*,* 2×3*,* 3×3*,* 2×4*, and* 3×4 *grids, respectively.*

Proof. Using integer linear programming [33], we exhaustively compute makespan-optimal solutions for arbitrary horizontal reconfigurations on 3×2 ($8 = 2^3$ possible cases), 4×2 grids (2^4 possible cases), 2×3 grids (6^2 possible cases), 3×3 grids (6^3 possible cases), 2×4 grids (24^2 possible cases), and 3×4 grids (24^3 possible cases), which confirms the claim. □

As an example, it takes seven steps to horizontally "swap" all three pairs of robots on a 3×2 grid, as shown in Figure 3.4.

Lemma 3.3.3 (Fast Line Shuffle)**.** *Embedded in a 2D grid,* m *robots on a straight path of length* m *may be arbitrarily ordered in* $7m$ *steps. Moreover, multiple such reconfigurations can be executed in parallel within the grid.*

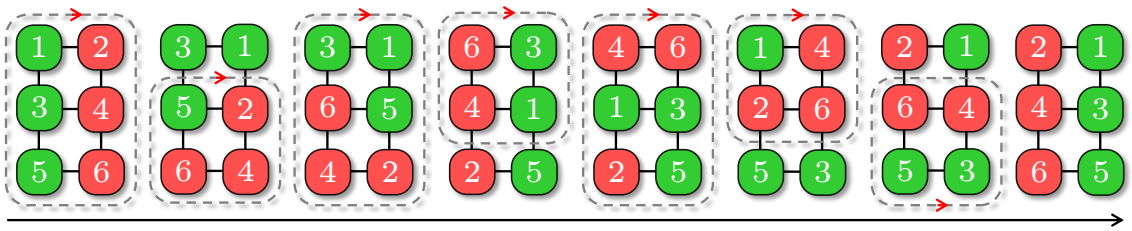


Figure 3.4: An example of a full horizontal "swap" on a 3×2 grid that takes seven steps, in which all three pairs are swapped.

Proof. Arranging m robots on a straight path of length m may be realized using parallel odd-even sort [96] in $m - 1$ rounds, which only requires the ability to simulate potential pairwise "swaps" interleaving odd phases (swapping robots located at positions $2k + 1$ and $2k+2$ on the path for some k) and even phases (swapping robots located at positions $2k+2$ and $2k + 3$ on the path for some k). Here, it does not matter whether m is odd or even. To simulate these swaps, we can partition the grid embedding the path into 3×2 grids in two ways for the two phases, as illustrated in Figure 3.5.

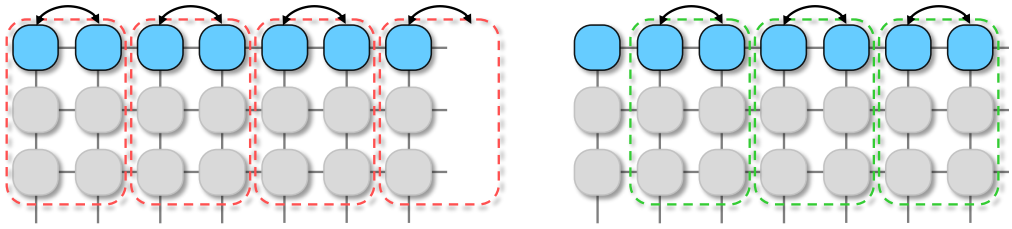


Figure 3.5: Partitioning a grid into disjoint 3×2 grids in two ways for simulating odd-even sort. Highlighted robot pairs may be independently "swapped" within each 3×2 grid as needed.

A perfect partition requires that the second dimension of the grid, perpendicular to the straight path, be a multiple of 3. If not, some partitions at the bottom can use 4×2 grids. By Lemma 3.3.2, each odd-even sorting phase can be simulated using at most $7m$ steps. Shuffling on parallel paths is directly supported. □

After introducing a $7m$ step line shuffle, we further show how it can be dropped to $4m$, using similar ideas. The difference is that an updated parallel odd-even sort will be used with different sub-grids of sizes different from 2×3 and 2×4.

The updated parallel odd-even sort operates on blocks of *four* (Figure 3.6) instead of *two* (Figure 3.5), which cuts down the number of parallel sorting operations from $m-1$ to about $m/2$. Here, if m is not even, a partition will leave either 1 or 3 at the end. For example, if $m=11$, it can be partitioned as $4,4,3$ and $2,4,4,1$ in the two parallel sorting phases.

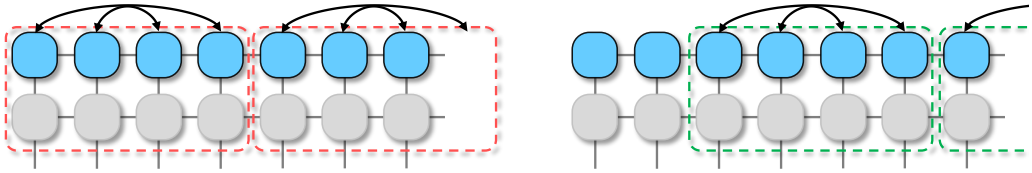


Figure 3.6: We can make the parallel odd-even sort work twice faster by increasing the swap block size from two to four.

With the updated parallel odd-even sort, we must be able to make swaps on blocks of four. We do this by partition an $m_1 \times m_2$ grid into 2×4, which may have leftover sub-grids of sizes 2×3, 3×4, and 3×3. Using the same reasoning in proving Lemma 3.3.3 and with Lemma 3.3.2, we have

Lemma 3.3.4 (Faster Line Shuffle)**.** *Embedded in a 2D grid, m robots on a straight path of length m may be arbitrarily ordered in approximately $4m$ steps. Moreover, multiple such reconfigurations can be executed in parallel within the grid.*

Proof sketch. The updated parallel odd-even sort requires a total of $m/2$ steps. Since each step operated on a 2×4, 2×3, 3×4, or 3×3 grid, which takes at most 8 steps, the total makespan is approximately $4m$. □

Combining GRA2D and fast line shuffle (Lemma 3.3.4) yields a polynomial time MRPP algorithm for fully occupied grids with a makespan of $4m_1 + 8m_2$.

Theorem 3.3.1 (MRPP on Grids under Maximum Robot Density, Upper Bound)**.** *MRPP on an $m_1 \times m_2$ grid, $m_1 \geq m_2 \geq 3$, with each grid vertex occupied by a robot, can be solved in polynomial time in a makespan of $4m_1 + 8m_2$.*

Proof Sketch. By combining the GRA2D and the line shuffle algorithms, the problem can be efficiently solved through two row-shuffle phases and one column-shuffle phase. During the row-shuffle phases, all rows can be shuffled in $4m_2$ steps, and similarly, during column-shuffle phases, all columns can be shuffled in $4m_1$ steps. Summing up these steps, the entire problem can be addressed in $4m_1 + 8m_2$ steps. The primary computational load lies in computing the perfect matchings, a task achievable in $O(m_1 m_2 \log m_1)$ expected time or $O(m_1^2 m_2)$ deterministic time. □

It is clear that, by exploiting the idea further, smaller makespans can potentially be obtained for the full-density setting, but the computation required for extending Lemma 3.3.2 will become more challenging. It took about two days to compute all 24^3 cases for the 3×4 grid, for example. We call the resulting algorithm from the process GRM (for 2D), with "M" denoting *maximum density*, regardless of the used sub-grids. The straightforward pseudo-code is given in algorithm 1. The comments in the main GRM routine indicate the corresponding GRA2D phases. For MRPP on an $m_1 \times m_2$ grid with row-column coordinates (x, y), we say robot i belongs to color $1 \leq j \leq m_1$ if $g_i.y = j$. Function `Prepare()` in the first phase finds intermediate states $\{\tau_i\}$ for each robot through perfect matchings and routes them towards the intermediate states by (simulated) column shuffles. If the robot density is smaller than required, we may fill the table with "virtual" robots [72, 29]. For each robot i, we have $\tau_i.y = s_i.y$. Function `ColumnFitting()` in the second phase routes the robots to their second intermediate states $\{\mu_i\}$ through row shuffles where $\mu_i.x = \tau_i.x$ and $\mu_i.y = g_i.y$. In the last phase, function `RowFitting()` routes the robots to their final goal positions using additional column shuffles.

We now establish the optimality guarantee of GRM on 2D grids, assuming MRPP instances are randomly generated. For this, a precise lower bound is needed.

Proposition 3.3.1 (Precise Makespan Lower Bound of MRPP on 2D Grids)**.** *The minimum makespan of random MRPP instances on an $m_1 \times m_2$ grid with $\Theta(m_1 m_2)$ robots is $m_1 + m_2 - o(m_1)$ with arbitrarily high probability as $m_1 \to \infty$.*

Algorithm 1: Labeled Grid Rearrangement Based MRPP Algorithm for 2D (GRA2D)

Input: Start and goal vertices $S = \{s_i\}$ and $G = \{g_i\}$

Function RTA2D(S, G)**:**
- `Prepare`(S, G) ▷ Computing Figure 3.2 (b)
- `ColumnFitting`(S, G) ▷ Figure 3.2 (a) $\rightarrow$ Figure 3.2 (c)
- `RowFitting`(S, G) ▷ Figure 3.2 (c) $\rightarrow$ Figure 3.2 (d)

Function `Prepare`(S, G)**:**
- $A \leftarrow [1, \ldots, m_1 m_2]$
- **for** $(t, r) \in [1, \ldots, m_1] \times [1, \ldots, m_1]$ **do**
  - **if** $\exists i \in A$ *where* $s_i.x = r \wedge g_i.y = t$ **then**
    - add edge (t, r) to $B(T, R)$
    - remove i from A
- compute matchings $\mathcal{M}_1, \ldots, \mathcal{M}_{m_2}$ of $B(T, R)$
- $A \leftarrow [1, \ldots, m_1 m_2]$
- **foreach** $\mathcal{M}_r$ *and* $(t, r) \in \mathcal{M}_r$ **do**
  - **if** $\exists i \in A$ *where* $s_i.x = r \wedge g_i.y = t$ **then**
    - $\tau_i \leftarrow (r, s_i.y)$ and remove i from A
    - mark robot i to go to τ_i
- perform simulated column shuffles in parallel

Function `ColumnFitting`(S, G)**:**
- **foreach** $i \in [1, \ldots, m_1 m_2]$ **do**
  - $\mu_i \leftarrow (\tau_i.x, g_i.y)$ and mark robot i to go to μ_i
- perform simulated row shuffles in parallel

Function `RowFitting`(S, G)**:**
- **foreach** $i \in [1, \ldots, m_1 m_2]$ **do**
  - mark robot i to go to g_i
- perform simulated column shuffles in parallel

Proof. Without loss of generality, let the constant in $\Theta(m_1 m_2)$ be some $c \in (0, 1]$, i.e., there are cm_1m_2 robots. We examine the top left and bottom right corners of the $m_1 \times m_2$ grid $\mathcal{G}$. In particular, let $\mathcal{G}_{tl}$ (resp., $\mathcal{G}_{br}$) be the top left (resp., bottom right) $\alpha m_1 \times \alpha m_2$ subgrid of $\mathcal{G}$, for some positive constant $\alpha \ll 1$. For $u \in V(\mathcal{G}_{tl})$ and $v \in V(\mathcal{G}_{br})$, assuming each grid edge has unit distance, then the Manhattan distance between u and v is at least $(1-2\alpha)(m_1+m_2)$. Now, the probability that some $u \in V(\mathcal{G}_{tl})$ and $v \in V(\mathcal{G}_{br})$ are the start and goal, respectively, for a single robot, is α^4. For cm_1m_2 robots, the probability that at least one robot's start and goal fall into $\mathcal{G}_{tl}$ and $\mathcal{G}_{br}$, respectively, is $p = 1 - (1 - \alpha^4)^{cm_1m_2}$.

Because $(1 - x)^y < e^{-xy}$ for $0 < x < 1$ and $y > 0$ [2]; multiplying both sides by a positive y and exponentiate with base e then yield the inequality., $p > 1 - e^{-\alpha^4 cm_1m_2}$. Therefore, for arbitrarily small α, we may choose m_1 such that p is arbitrarily close to 1. For example, we may let $\alpha = m_1^{-\frac{1}{8}}$, which decays to zero as $m_1 \to \infty$, then it holds that the makespan is $(1 - 2\alpha)(m_1 + m_2) = m_1 + m_2 - 2m_1^{-\frac{1}{8}}(m_1 + m_2) = m_1 + m_2 - o(m_1)$ with probability $p > 1 - e^{-c\sqrt{m_1}m_2}$. □

Comparing the upper bound established in Theorem 3.3.1 and the lower bound from Proposition 3.3.1 immediately yields

Theorem 3.3.2 (Optimality Guarantee of GRM)**.** *For random MRPP instances on an $m_1 \times m_2$ grid with $\Omega(m_1m_2)$ robots, as $m_1 \to \infty$, GRM computes in polynomial time solutions that are $4(1 + \frac{m_2}{m_1+m_2})$-makespan optimal, with high probability.*

Proof Sketch. By Proposition 3.6.3 and Theorem 3.3.1, the asymptotic optimality ratio is $4\left(1 + \frac{m_2}{m_1+m_2}\right)$. □

GRM always runs in polynomial time and has the same running time as GRA2D; the high probability guarantee only concerns solution optimality. The same is true for other high-probability algorithms proposed in this paper. We also note that high-probability guarantees imply guarantees in expectation.

[2] This is because $\log(1 - x) < -x$ for $0 < x < 1$

3.4 Near-Optimally Solving MRPP with up to One-Third and One-Half Robot Densities

Though GRM runs in polynomial time and provides constant factor makespan optimality in expectation, the constant factor is $4+$ due to the extreme density. In practice, a robot density of around $\frac{1}{3}$ (i.e., $n = \frac{m_1 m_2}{3}$) is already very high. As it turns out, with $n = cm_1m_2$ for some constant $c > 0$ and $n \leq \frac{m_1 m_2}{2}$, the constant factor can be dropped to close to 1.

3.4.1 Up to One-Third Density: Shuffling with Highway Heuristics

For $\frac{1}{3}$ density, we work with random MRPP instances in this subsection; arbitrary instances for up to $\frac{1}{2}$ density are addressed in later subsections. Let us assume for the moment that m_1 and m_2 are multiples of three; we partition $\mathcal{G}$ into 3×3 cells (see, e.g., Figure 3.1 (b) and Figure 3.7). We use Figure 3.7, where Figure 3.7 (a) is a random start configuration and Figure 3.7 (f) is a random goal configuration, as an example to illustrate GRH — Grid Rearrangement with Highways, targeting robot density up to $\frac{1}{3}$. GRH has two phases: *unlabeled reconfiguration* and *MRPP resolution with Grid Rearrangement and highway heuristics*.

In unlabeled reconfiguration, robots are treated as being indistinguishable. Arbitrary start and goal configurations (under $\frac{1}{3}$ robot density) are converted to intermediate configurations where each 3×3 cell contains no more than 3 robots. We call such configurations *balanced*. With high probability, random MRPP instances are not far from being balanced. To establish this result (Proposition 3.4.1), we need the following.

Theorem 3.4.1 (Minimax Grid Matching [97])**.** *Consider an $m \times m$ square containing m^2 points following the uniform distribution. Let ℓ be the minimum length such that there exists a perfect matching of the m^2 points to the grid points in the square for which the distance between every pair of matched points is at most ℓ. Then $\ell = O(\log^{\frac{3}{4}} m)$ with high probability.*

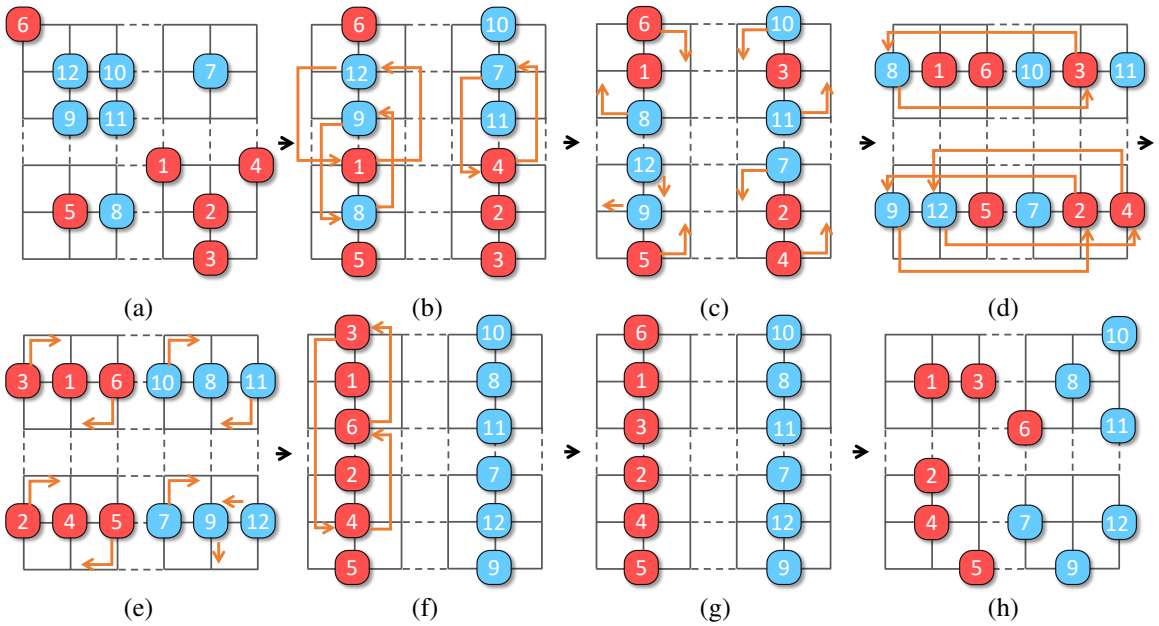


Figure 3.7: An example of applying GRH to solve an MRPP instance. (a) The start configuration; (b) The start balanced configuration obtained from (a); (c) The intermediate configuration obtained from the Grid Rearrangement preparation phase; (d). The intermediate configuration obtained from (c); (e) The intermediate configuration obtained from the column fitting phase; (f) Apply additional column shuffles for labeled items; (g) The goal balanced configuration obtained from the goal configuration; (h) The goal configuration.

Theorem 3.4.1 applies to rectangles with the longer side being m as well (Theorem 3 in [97]).

Proposition 3.4.1. *On an $m_1 \times m_2$ grid, with high probability, a random configuration of $n = \frac{m_1 m_2}{3}$ robots is of distance $o(m_1)$ to a balanced configuration.*

Proof. We prove for the case of $m_1 = m_2 = 3m$ using the minimax grid matching theorem (Theorem 3.4.1); generalization to $m_1 \geq m_2$ can be then seen to hold using the generalized version of Theorem 3.4.1 that applies to rectangles (Theorem 3 of [97], which applies to arbitrarily simply-connected region within a square region).

Now let $m_1 = m_2 = 3m$. We may view a random configuration of m^2 robots on a $3m \times 3m$ grid as randomly placing m^2 continuous points in an $m \times m$ square with scaling (by three in each dimension) and rounding. By Theorem 3.4.1, a random configuration of m^2 continuous points in an $m \times m$ square can be moved to the m^2 grid points at the

center of the m^2 disjoint unit squares within the $m \times m$ square, where each point is moved by a distance no more than $O(\log^{\frac{3}{4}} m)$, with high probability. Translating this back to a $3m \times 3m$ gird, m^2 randomly distributed robots on the grid can be moved so that each 3×3 cell contains exactly one robot, and the maximum distance moved for any robot is no more than $O(\log^{\frac{3}{4}} m)$, with high probability. Applying this argument three times yields that a random configuration of $\frac{m_1^2}{3}$ robots on an $m_1 \times m_1$ gird can be moved so that each 3×3 cell contains exactly three robots, and no robot needs to move more than a $O(\log^{\frac{3}{4}} m_1)$ steps, with high probability. Because the robots are indistinguishable, overlaying three sets of reconfiguration paths will not cause an increase in the distance traveled by any robot. □

In the example, unlabeled reconfiguration corresponds to Figure 3.7(a)→Figure 3.7(b) and Figure 3.7(f)→Figure 3.7(e) (MRPP solutions are time-reversible). We simulated the process of unlabeled reconfiguration for $m_1 = m_2 = 300$, i.e., on a 300×300 grids. For $\frac{1}{3}$ robot density, the actual number of steps averaged over 100 random instances is less than 5. We call configurations like Figure 3.7(b)-(e), which have all robots concentrated vertically or horizontally in the middle of the 3×3 cells, *centered balanced* or simply *centered*. Completing the first phase requires solving two unlabeled problems [98, 99], doable in polynomial time.

In the second phase, GRA is applied with a highway heuristic to get us from Figure 3.7(b) to Figure 3.7(e), transforming between vertical centered configurations and horizontal centered configurations. To do so, GRA is applied (e.g., to Figure 3.7(b) and (e)) to obtain two intermediate configurations (e.g., Figure 3.7(c) and (d)). To go between these configurations, e.g., Figure 3.7(b)→Figure 3.7(c), we apply a heuristic by moving robots that need to be moved out of a 3×3 cell to the two sides of the middle columns of Figure 3.7(b), depending on their target direction. If we do this consistently, after moving robots out of the middle columns, we can move all robots to their desired goal 3×3 cell without stopping nor collision. Once all robots are in the correct 3×3 cells, we can convert the balanced configuration to a centered configuration in at most 3 steps, which is necessary

for carrying out the next simulated row/column shuffle. Adding things up, we can simulate a shuffle operation using no more than $m + 5$ steps where $m = m_1$ or m_2. The efficiently simulated shuffle leads to low makespan MRPP algorithms. It is clear that all operations take polynomial time; a precise running time is given at the end of this subsection.

Theorem 3.4.2 (Makespan Upper Bound for Random MRPP, $\leq \frac{1}{3}$ Density)**.** *For random MRPP instances on an $m_1 \times m_2$ grid, where $m_1 \geq m_2$ are multiples of three, for $n \leq \frac{m_1 m_2}{3}$ robots, an $m_1 + 2m_2 + o(m_1)$ makespan solution can be computed in polynomial time, with high probability.*

First, we introduce two important lemmas about obstacle-free 2D grids which have been proven in [29].

Lemma 3.4.1. *On an $m_1 \times m_2$ grid, any unlabeled MRPP can be solved using $O(m_1 + m_2)$ makespan.*

Lemma 3.4.2. *On an $m_1 \times m_2$ grid, if the underestimated makespan of an MRPP instance is d_g (usually computed by ignoring the inter-robot collisions), then this instance can be solved using $O(d_g)$ makespan.*

Proof. By Proposition 3.4.1, unlabeled reconfiguration requires distance $o(m_1)$ with high probability. This implies that a plan can be obtained for unlabeled reconfiguration that requires $o(m_1)$ makespan (for detailed proof, refer to [29]). For the second phase, by Theorem 3.2.2, we need to perform m_1 parallel row shuffles with a row width of m_2, followed by m_2 parallel column shuffles with a column width of m_1, followed by another m_1 parallel row shuffles with a row width of m_2. Simulating these shuffles require $m_1 + 2m_2 + O(1)$ steps. Altogether, a makespan of $m_1 + 2m_2 + o(m_1)$ is required, with a very high probability. □

Contrasting Theorem 3.4.2 and Proposition 3.3.1 yields

Theorem 3.4.3 (Makespan Optimality for Random MRPP, $\leq \frac{1}{3}$ Density)**.** *For random MRPP instances on an $m_1 \times m_2$ grid, where $m_1 \geq m_2$ are multiples of three, for $n = cm_1m_2$ robots with $c \leq \frac{1}{3}$, as $m_1 \to \infty$, a $(1 + \frac{m_2}{m_1+m_2})$ makespan optimal solution can be computed in polynomial time, with high probability.*

Since $m_1 \geq m_2$, $1 + \frac{m_2}{m_1+m_2} \in (1, 1.5]$. In other words, in polynomial running time, GRH achieves $(1 + \delta)$ asymptotic makespan optimality for $\delta \in (0, 0.5]$, with high probability.

From the analysis so far, if m_1 and/or m_2 are not multiples of 3, it is clear that all results in this subsection continue to hold for robot density $\frac{1}{3} - \frac{(m_1 \bmod 3)(m_2 \bmod 3)}{m_1m_2}$, which is arbitrarily close to $\frac{1}{3}$ for large m_1 and m_2. It is also clear that the same can be said for grids with certain patterns of regularly distributed obstacles (Figure 3.1(b)), i.e.,

Corollary 3.4.1 (Random MRPP, $\frac{1}{9}$ Obstacle and $\frac{2}{9}$ Robot Density)**.** *For random MRPP instances on an $m_1 \times m_2$ grid, where $m_1 \geq m_2$ are multiples of three and there is an obstacle at coordinates $(3k_1 + 2, 3k_2 + 2)$ for all applicable k_1 and k_2, for $n = cm_1m_2$ robots with $c \leq \frac{2}{9}$, a solution can be computed in polynomial time that has makespan $m_1 + 2m_2 + o(m_1)$ with high probability. As $m_1 \to \infty$, the solution approaches $1 + \frac{m_2}{m_1+m_2}$ optimal, with high probability.*

Proof. Because the total density of robots and obstacles is no more than $1/3$, if Theorem 3.4.1 extends to support regularly distributed obstacles, then Theorem 3.4.3 applies because the highway heuristics do not pass through the obstacles. This is true because each obstacle can only add a constant length of path detour to a robot's path. In other words, the length ℓ in Theorem 3.4.3 will only increase by a constant factor and will remain as $\ell = O(\log^{\frac{3}{4}} m)$. Similar arguments hold for Proposition 3.4.1. □

We now give the running time of GRM and GRH.

Proposition 3.4.2 (Running Time, GRH)**.** *For $n \leq \frac{m_1m_2}{3}$ robots on an $m_1 \times m_2$ grid, GRH runs in $O(nm_1^2m_2)$ time.*

Proof. The running time of GRH is dominated by the matching computation and solving unlabeled MRPP. The matching part takes $O(m_1^2 m_2)$ in deterministic time or $O(m_1 m_2 \log m_1)$ in expected time [100]. Unlabeled MRPP may be tackled using the max-flow algorithm [101] in $O(nm_1m_2T) = O(nm_1^2m_2)$ time, where $T = O(m_1+m_2)$ is the expansion time horizon of a time-expanded graph that allows a routing plan to complete. □

3.4.2 One-Half Robot Density: Shuffling with Linear Merging

Using a more sophisticated shuffle routine, $\frac{1}{2}$ robot density can be supported while retaining most of the guarantees for the $\frac{1}{3}$ density setting; obstacles are no longer supported.

To best handle $\frac{1}{2}$ robot density, we employ a new shuffle routine called *linear merge*, based on merge sort, and denote the resulting algorithm as Grid Rearrangement with Linear Merge or GRLM. The basic idea is straightforward: for m robots on a $2 \times m$ grid, we iteratively sort the robots first on 2×2 grids, then 2×4 grids, and so on, much like how merge sort works. An illustration of the process on a 2×8 grid is shown in Figure 3.8.

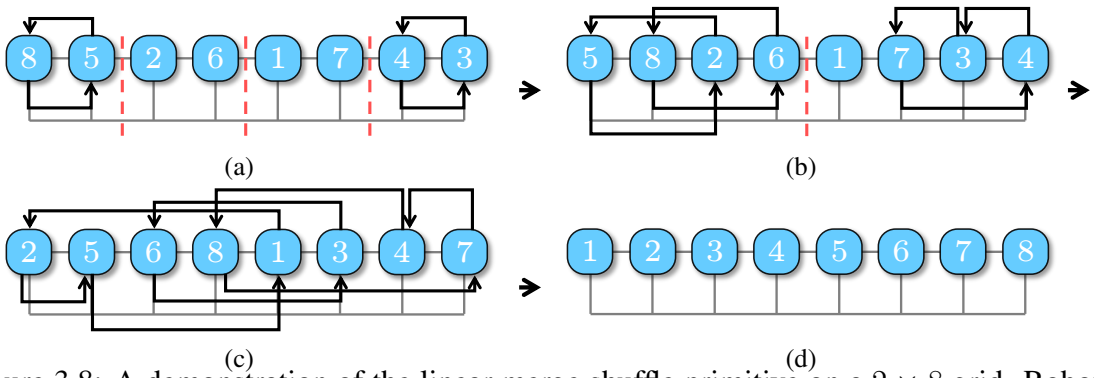


Figure 3.8: A demonstration of the linear merge shuffle primitive on a 2×8 grid. Robots going to the left always use the upper channel while robots going to the right always use the lower channel.

Lemma 3.4.3 (Properties of Linear Merge)**.** *On a $2 \times m$ grid, m robots, starting on the first row, can be arbitrarily ordered using $m + o(m)$ steps, using the algorithm 2, inspired from merge sort. The motion plan can be computed in polynomial time.*

Proof. We first show *feasibility*. The procedure takes $\lceil \log m \rceil$ phases; in a phase, let us

Algorithm 2: Line Merge Algorithm

Input: An array arr representing the vertices labeled from 1 to n with current and intermediate locations of n robots.

Function `LineMerge(`arr`)`**:**
- **if** *length of* $arr > 1$ **then**
  - $mid \leftarrow$ length of $arr \div 2$
  - $left \leftarrow arr[0 \ldots mid - 1]$
  - $right \leftarrow arr[mid \ldots$ length of $arr - 1]$
  - `LineMerge(`$left$`)`
  - `LineMerge(`$right$`)`
  - `Merge(`arr`)`

Function `Merge(`arr`)`**:**
- Sort robots located at the vertices in arr to obtain intermediate states
- **for** $i \in arr$ **do**
  - $a_i \leftarrow$ robot at vertex i
  - **if** a_i.*intermediate* $> i$ **then**
    - Route a_i moving rightward using the bottom line while avoiding blocking those robots that are moving leftward
  - **else**
    - Route a_i moving leftward using the upper line without stopping
- Synchronize the paths

denote a section of the $2 \times m$ grid where robots are treated together as a *block*. For example, the left 2×4 grid in Figure 3.8(b) is a block. It is clear that the first phase, involving up to two robots per block, is feasible (i.e., no collision). Assuming that phase k is feasible, we look at phase $k+1$. We only need to show that the procedure is feasible on one block of length up to 2^{k+1}. For such a block, the left half-block of length up to 2^k is already fully sorted as desired, e.g., in increasing order from left to right. For the $k+1$ phase, all robots in the left half-block may only stay in place or move to the right. These robots that stay must be all at the leftmost positions of the half-block and will not block the motions of any other robot. For the robots that need to move to the right, their relative orders do not need to change and, therefore, will not cause collisions among themselves. Because these robots that move in the left half-block will move down on the grid by one edge, they will not interfere with any robot from the right half-block. Because the same arguments hold for the right half-block (except the direction change), the overall process of merging a block occurs without collision.

Next, we examine the *makespan*. For any single robot r, at phase k, suppose it belongs to block b and block b is to be merged with block b'. It is clear that the robot cannot move more than $len(b') + 2$ steps, where $len(b')$ is the number of columns of b' and the 2 extra steps may be incurred because the robot needs to move down and then up the grid by one edge. This is because any move that r needs to do is to allow robots from b' to move toward b. Because there are no collisions in any phase, adding up all the phases, no robot moves more than $m + 2(\log m + 1) = m + o(m)$ steps.

Finally, the merge sort-like linear merge shuffle primitive runs in $O(m \log m)$ time since it is a standard divide-and-conquer routine with $\log m$ phases. □

We distinguish between $\frac{1}{3}$ and $\frac{1}{2}$ density settings because the overhead in GRLM is larger. Nevertheless, with the linear merge, the asymptotic properties of GRH for $\frac{1}{3}$ robot density mostly carry over to GRLM.

Theorem 3.4.4 (Random MRPP, $\frac{1}{2}$ Robot Density)**.** *For random MRPP instances on an*

$m_1 \times m_2$ grid, where $m_1 \geq m_2$ are multiples of two, for $\frac{m_1 m_2}{3} \leq n \leq \frac{m_1 m_2}{2}$ robots, a solution can be computed in polynomial time that has makespan $m_1 + 2m_2 + o(m_1)$ with high probability. As $m_1 \to \infty$, the solution approaches an optimality of $1 + \frac{m_2}{m_1+m_2} \in (1, 1.5]$, with high probability.

Proof Sketch. The proof follows by combining Proposition 3.6.3 and Lemma 3.4.3. □

3.4.3 Supporting Arbitrary MRPP Instances on Grids

We now examine applying GRH to arbitrary MRPP instances up to $\frac{1}{2}$ robot density. If an MRPP instance is arbitrary, all that changes to GRH is the makespan it takes to complete the unlabeled reconfiguration phase. On an $m_1 \times m_2$ grid, by computing a matching, it is straightforward to show that it takes no more than $m_1 + m_2$ steps to complete the unlabeled reconfiguration phase, starting from an arbitrary start configuration. Since two executions of unlabeled reconfiguration are needed, this adds $2(m_1+m_2)$ additional makespan. Therefore, the following results hold.

Theorem 3.4.5 (Arbitrary MRPP, $\leq \frac{1}{2}$ Density)**.** *For arbitrary MRPP instances on an $m_1 \times m_2$ grid, $m_1 \geq m_2$, for $n \leq \frac{m_1 m_2}{2}$ robots, a $3m_1 + 4m_2 + o(m_1)$ makespan solution can be computed in polynomial time.*

3.5 Optimality-Boosting Heuristics

3.5.1 Reducing Makespan via Optimizing Matching

Based on GRA, GRH has three simulated shuffle phases. The makespan is dominated by the robot, which needs the longest time to move, as a sum of moves in all three phases. As a result, the optimality of Grid Rearrangement methods is determined by the first preparation/matching phase. Finding arbitrary perfect matchings is fast but the process can be improved to reduce the overall makespan.

For improving matching, we propose two heuristics; the first is based on *integer programming* (IP). We create binary variables $\{x_{ri}\}$ where r represents the row number and i the robot. robot i is assigned to row r if $x_{ri} = 1$. Define single robot cost as $C_{ri}(\lambda) = \lambda|r - s_i.x| + (1 - \lambda)|r - g_i.x|$. We optimize the makespan lower bound of the first phase by letting $\lambda = 0$ or the third phase by letting $\lambda = 1$. The objective function and constraints are given by

$$\max_{r,i}\{C_{ri}(\lambda = 0)x_{ri}\} + \max_{r,i}\{C_{ri}(\lambda = 1)x_{ri}\} \tag{3.1}$$

$$\sum_{r} x_{ri} = 1, \text{for each robot } i \tag{3.2}$$

$$\sum_{g_i.y=t} x_{ri} \leq 1, \text{for each row } r \text{ and each color } t \tag{3.3}$$

$$\sum_{s_i.x=c} x_{ri} = 1, \text{for each column } c\text{and each row } r \tag{3.4}$$

Equation 3.1 is the summation of makespan lower bound of the first phase and the third phase. Note that the second phase can not be improved by optimizing the matching. Equation 3.2 requires that robot i be only present in one row. Equation 3.3 specifies that each row should contain robots that have different goal columns. Equation 3.4 specifies that each vertex (r, c) can only be assigned to one robot. The IP model represents a general assignment problem which is NP-hard in general. It has limited scalability but provides a way to evaluate how optimal the matching could be in the limit.

A second matching heuristic we developed is based on LBA [102], which takes polynomial time. LBA differs from the IP heuristic in that the bipartite graph is weighted. For the matching assigned to row r, the edge weight of the bipartite graph is computed greedily. If column c contains robots of color t, we add an edge (c, t) and its edge cost is

$$C_{ct} = \min_{g_i.y=t} C_{ri}(\lambda = 0) \tag{3.5}$$

We choose $\lambda = 0$ to optimize the first phase. Optimizing the third phase ($\lambda = 1$) would give similar results. After constructing the weighted bipartite graph, an $O(\frac{m_1^{2.5}}{\log m_1})$ LBA algorithm [102] is applied to get a minimum bottleneck cost matching for row r. Then we remove the assigned robots and compute the next minimum bottleneck cost matching for the next row. After getting all the matchings $\mathcal{M}_r$, we can further use LBA to assign $\mathcal{M}_r$ to a different row r' to get a smaller makespan lower bound. The cost for assigning matching $\mathcal{M}_r$ to row r' is defined as

$$C_{\mathcal{M}_r r'} = \max_{i \in \mathcal{M}_r} C_{r'i}(\lambda = 0) \tag{3.6}$$

The total time complexity of using LBA heuristic for matching is $O(\frac{m_1^{3.5}}{\log m_1})$.

We denote GRH with IP and LBA heuristics as GRHip and GRHlba, respectively. We mention that GRM, which uses the line swap motion primitive, can also benefit from these heuristics to re-assign the goals within each group. This can lower the bottleneck path length and improve optimality.

3.5.2 Path Refinement

Final paths from GRA-based algorithms are concatenations of paths from multiple planning phases. This means robots are forced to "synchronize" at phase boundaries, which causes unnecessary idling for robots finishing a phase early. Noticing this, we de-synchronize the planning phases, which yields significant gains in makespan optimality.

Our path refinement method does something similar to Minimal Communication Policy (Minimal Communication Policy (MCP)) [103], a multi-robot execution policy proposed to handle unexpected delays without stopping unaffected robots. During execution, MCP lets robots execute their next non-idling move as early as possible while preserving the relative execution orders between robots, e.g., when two robots need to enter the same vertex at different times, that ordering is preserved.

We adopt the principle used in MCP to refine the paths generated by GRA-based algorithms as shown in algorithm 3, with algorithm 4 as a sub-routine. The algorithms work as

follows. All idle states are removed from the initial robot but the order of visits for each vertex is kept (Line 2-3). Then we enter a loop executing the plans following the MCP principle (Line 8-10). In algorithm 4, if i is the next robot that enters vertex v_i according to the original order, we check if there is a robot currently at v_i. If there is not, we let i enter v_i. If another robot j is currently occupying v_j, we examine if j is moving to its next vertex v_j in the next step by recursively calling the function `Move`. We check if there is any cycle in the robot movements in the next original plan. If i is in a cycle, we move all the robots in this cycle to their next vertex recursively (Line 20-28). If no cycle is found and j is to enter its next vertex v_j, we let i also move to its next vertex v_i. Otherwise, i should wait at u_i. The algorithm is *deadlock-free* by construction [103].

Algorithm 3: Path Refinement

```
   Input: Paths 𝒫 generated by GRA
1  Function Refine(𝒫):
2      foreach v ∈ V, VOrder[v] ← Queue()
3      Preprocess(InitialPlans, VOrder)
4      while true do
5          for i = 1...n do
6              Moved ← Dict()
7              CycleCheck ← Set()
8              Move(i)
9              if AllReachedGoals()=true then
10                 break
```

Let T be the makespan of the initial paths, the makespan of the solution obtained by running `Refine` is clearly no more than T. In each loop of `Refine`, we essentially run DFS on a graph that has n nodes and traverse all the nodes, for which the time complexity is $O(n)$, Therefore the time complexity of the path refinement is bounded by $O(nT)$.

Other path refinement methods, such as [64, 104], can also be applied in principle, which iteratively chooses a small group of robots and re-plan their paths holding other robots as dynamic obstacles. However, in dense settings that we tackle, re-planning for a small group of robots has little chance of finding better paths this way.

Algorithm 4: Move

```
1  Function Move(i):
2      if i in Moved then
3          return Moved[i]
4      u_i ← current position of i
5      v_i ← next position of i
6      if i = VOrder[v_i].front() then
7          j ← the robot currently at v_i
8          if i is in CycleCheck then
9              MoveAllRobotsInCycle(i)
10             return true
11         CycleCheck.add(i)
12         if Move(j) = true or j =None then
13             let i enter v_i
14             VOrder[v_i].popfront()
15             Moved[i] ←true
16             return true
17     let i wait at u_i
18     Moved[i]=false
19     return false
20 Function MoveAllRobotsInCycle(i):
21     Visited ← Set()
22     j ← i
23     while j is not in visited do
24         let j enter its next vertex v_j
25         Visited.add(j)
26         VOrder[v_j].popfront()
27         Moved[j] ← true
28         j ← the robot currently at vertex v_j
```

3.6 Generalization to 3D

We now explore the 3D setting. To keep the discussion focused, we mainly show how to generalize GRH to 3D, but note that GRM and GRLM can also be similarly generalized. High-dimensional GRP [93] is defined as follows.

Problem 2 (**Grid Rearrangement in kD (GRPk)**). *Let M be an $m_1 \times \ldots \times m_k$ table, $m_1 \geq \ldots \geq m_k$, containing $\prod_{i=1}^{k} m_i$ items, one in each table cell. In a* shuffle *operation, the items in a single column in the i-th dimension of M, $1 \leq i \leq k$, may be permuted arbitrarily. Given two arbitrary configurations S and G of the items on M, find a sequence of shuffles that take M from S to G.*

We may solve Grid Rearrangement Problem in 3D (GRP3D) using GRA2D as a subroutine by treating GRP as a GRP2D, which is straightforward if we view a 2D slice of GRA2D as a "wide" column. For example, for the $m_1 \times m_2 \times m_3$ grid, we may treat the second and third dimensions as a single dimension. Then, each wide column, which is itself an $m_2 \times m_3$ 2D problem, can be reconfigured by applying GRA2D. With some proper counting, we obtain the following 3D version of Theorem 3.2.2 as follows.

Theorem 3.6.1 (**Grid Rearrangement Theorem, 3D**). *An arbitrary Grid Rearrangement problem on an $m_1 \times m_2 \times m_3$ table can be solved using $m_1m_2 + m_3(2m_2 + m_1) + m_1m_2$ shuffles.*

Although [93] offers a broad framework for addressing GRPk, it is not directly applicable to multi-robot routing in high-dimensional spaces. To overcome this limitation, we have developed the high-dimensional MRPP algorithm by incorporating and elaborating on its underlying principles similar to the 2D scenarios. Denote the corresponding algorithm for Theorem 3.6.1 as Grid Rearrangement Algorithm in 3D (GRA3D), we illustrate how it works on a $3\times3\times3$ table (Figure 3.9). GRA operates in three phases. First, a bipartite graph $B(T, R)$ is constructed based on the initial table where the partite set T are the colors of

items representing the desired (x, y) positions, and the set R are the set of (x, y) positions of items (Figure 3.2(b)). Edges are added as in the 2D case. From $B(T, R)$, m_3 *perfect matchings* are computed. Each matching contains $m_1 m_2$ edges and connects all of T to all of R. processing these matchings yields an intermediate table (Figure 3.9(c)), serving a similar function as in the 2D case. After the first phase of $m_1 m_2$ z-shuffles, the intermediate table (Figure 3.9(c)) that can be reconfigured by applying GRA2D with m_1 x-shuffles and $2m_2$ y-shuffles. This sorts each item in the correct x-y positions (Figure 3.2(d)). Another $m_1 m_2$ z-shuffles can then sort each item to its desired z position (Figure 3.2(e)).

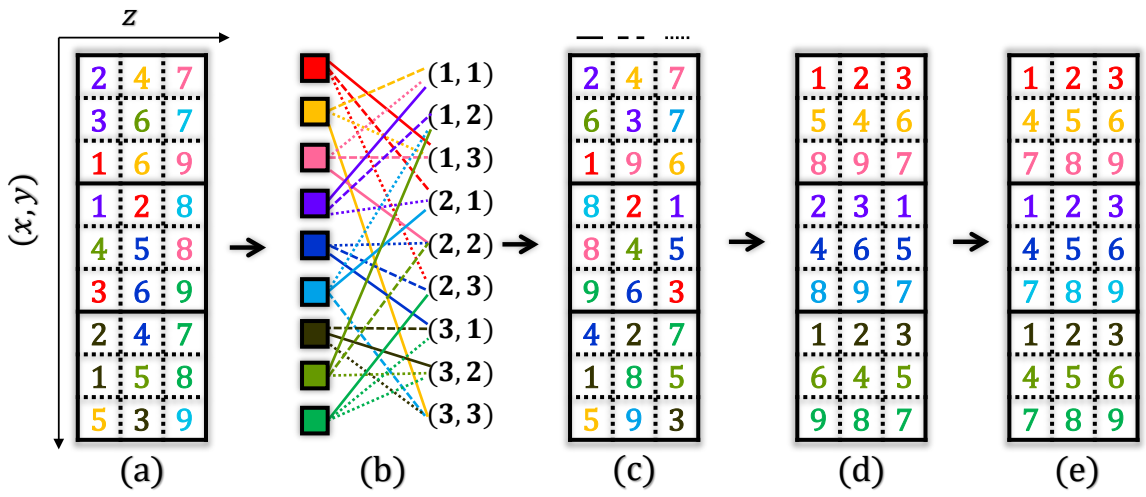


Figure 3.9: Illustration of applying *GRA3D*. (a) The initial $3 \times 3 \times 3$ table with a random arrangement of 27 items that are colored and labeled. Color represents the (x, y) position of an item. (b) The constructed bipartite graph. The right partite set contains all the possible (x, y) positions. It contains 3 perfect matchings, determining the 3 columns in (c). (c) Applying z-shuffles to (a), according to the matching results, leads to an intermediate table where each x-y plane has one color appearing exactly once. (d) Applying wide shuffles to (c) correctly places the items according to their (x, y) values (or colors). (e) Additional z-shuffles fully sort the labeled items.

3.6.1 Extending GRH to 3D

The algorithm 5, which calls algorithm 6 and algorithm 7, outlines the high-level process of extending GRH to 3D. In each x-y plane, $\mathcal{G}$ is partitioned into 3×3 cells (e.g.,Figure 3.7). Without loss of generality, we assume that m_1, m_2, m_3 are multiples of 3 and there are no

obstacles. First, to make GRA3D applicable, we convert the arbitrary start/goal configurations to intermediate *balanced* configurations S_1 and G_1, treating robots as unlabeled, as we have done in GRH, wherein each 2D plane, each 3×3 cell contains no more than 3 robots (Figure 3.10).

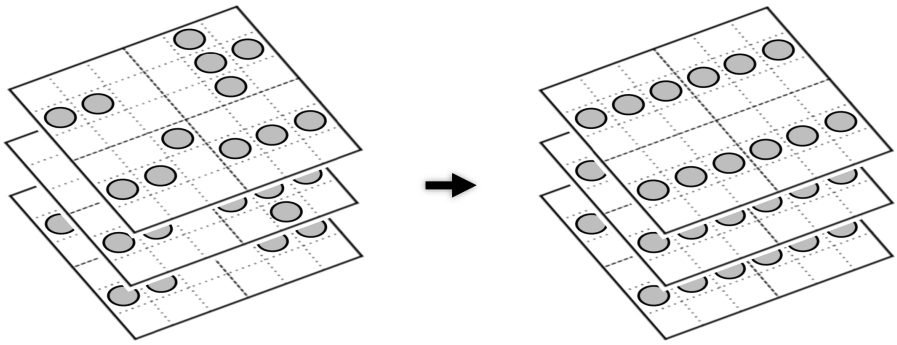


Figure 3.10: Applying unlabeled MRPP to convert a random configuration to a balanced centering one on $6 \times 6 \times 3$ grids.

GRA3D can then be applied to coordinate the robots moving toward their intermediate goal positions G_1. Function `MatchingXY` finds a feasible intermediate configuration S_2 and routes the robots to S_2 by simulating shuffle operations along the z axis. Function `XY-Fitting` apply shuffle operations along the x and y axes to route each robot i to its desired x-y position $(g_{1i}.x, g_{1i}.y)$. In the end, the function `Z-Fitting` is called, routing each robot i to the desired g_{1i} by performing shuffle operations along the z axis and concatenating the paths computed by unlabeled MRPP planner `UnlabeledMRPP`.

Algorithm 5: Grid Rearrangement with Highways in 3D (GRH3D)

Input: Start and goal vertices $S = \{s_i\}$ and $G = \{g_i\}$

```
1 Function GRH3D(S, G):
2     S_1, G_1 ←UnlabeledMRPP(S, G)
3     MatchingXY()
4     XY-Fitting()
5     Z-Fitting()
```

We now explain each part of GRH3D. `MatchingXY` uses an extended version of GRA to find perfect matching that allows feasible shuffle operations. Here, the “item color" of

an item i (robot) is the tuple $(g_{1i}.x, g_{1i}.y)$, which is the desired x-y position it needs to go. After finding the m_3 perfect matchings, the intermediate configuration S_2 is determined. Then, shuffle operations along the z direction can be applied to move the robots to S_2. The robots in each x-y plane will be reconfigured by applying x-shuffles and y-shuffles.

Algorithm 6: MatchingXY

Input: Balanced start and goal vertices $S_1 = \{s_{1i}\}$ and $G_1 = \{g_{1i}\}$

1 **Function** `MatchingXY(`S_1, G_1`)`**:**
2 $A \leftarrow [1, ..., n]$
3 $\mathcal{T} \leftarrow$ the set of (x, y) positions in S_1
4 **for** $(r, t) \in \mathcal{T} \times \mathcal{T}$ **do**
5 **if** $\exists i \in A$ *where* $(s_{1i}.x, s_{1i}.y) = r \wedge (g_{1i}.x, g_{1i}.y) = t$ **then**
6 add edge (r, t) to $B(T, R)$
7 remove i from A
8 compute matchings $\mathcal{M}_1, ..., \mathcal{M}_{m_3}$ of $B(T, R)$
9 $A \leftarrow [1, ..., n]$
10 **foreach** $\mathcal{M}_c$ *and* $(r, t) \in \mathcal{M}_c$ **do**
11 **if** $\exists i \in A$ *where* $(s_{1i}.x, s_{1i}.y) = r \wedge (g_{1i}.x, g_{1i}.y) = t$ **then**
12 $s_{2i} \leftarrow (s_{1i}.x, s_{1i}.y, c)$ and remove i from A
13 mark robot i to go to s_{2i}
14 perform simulated z-shuffles in parallel

We need to apply GRA2D for these robots in each plane, as demonstrated in algorithm 7. In GRH, for each 2D plane, the “item color" for robot i is its desired x position $g_{1i}.x$. For each plane, we compute the m_2 perfect matchings to determine the intermediate position g_2. Then each robot i moves to its g_{2i} by applying y-shuffle operations. In Line 18, each robot is routed to its desired x position by performing x-shuffle operations. In Line 19, each robot is routed to its desired y position by performing y-shuffle operations. After all the robots reach the desired x-y positions, another round of z-shuffle operations in `Z-Fitting` can route the robots to the balanced goal configuration computed by an unlabeled MRPP planner. In the end, we concatenate all the paths as the result.

GRLM and GRM can be extended to 3D in similar ways by replacing the solvers in 2D planes. For GRM, there is no need to use unlabeled MRPP for balanced reconfiguration,

Algorithm 7: XY-Fitting

Input: Current positions S_2 and balanced goal positions G_1

Function `XY-Fitting()`**:**
  for $z \leftarrow [1, ..., m_3]$ **do**
    $A \leftarrow \{i | s_{2i}.y = z\}$
    `GRH2D(`A, z`)`

Function `GRH2D(`A, z`)`**:**
  $\mathcal{T} \leftarrow$ the set of x positions of S_2
  for $(r, t) \in \mathcal{T} \times \mathcal{T}$ **do**
    if $\exists i \in A$ *where* $s_{2i}.x = r \wedge g_{1i}.x = t$ **then**
      if *robot* i *is not assigned* **then**
        add edge (r, t) to $B(T, R)$
        mark i assigned
    compute matchings $\mathcal{M}_1, ..., \mathcal{M}_{m_2}$ of $B(T, R)$
  $A' \leftarrow A$
  foreach $\mathcal{M}_c$ *and* $(r, t) \in \mathcal{M}_c$ **do**
    if $\exists i \in A'$ *where* $s_{2i}.x = c \wedge g_{1i}.x = t$ **then**
      $g_{2i} \leftarrow (s_{2i}.x, c, z)$ and remove i from A'
      mark robot i to go to g_{2i}
  route each robot $i \in A$ to g_{2i}
  route each robot $i \in A$ to $(g_{2i}.x, g_{1i}.y, z)$
  route each robot $i \in A$ to $(g_{1i}.x, g_{1i}.y, z)$

which yields a makespan upper bound of $4m_1 + 8m_2 + 8m_3$ (as a direct combination of Theorem 3.3.1 and Theorem 3.6.1). For arbitrary instances under half density in 3D, the makespan guarantee in Theorem 3.4.5 updates to $3m_1 + 4m_2 + 4m_3 + o(m_1)$.

3.6.2 Properties of GRH3D

First, we introduce two important lemmas about obstacle-free 3D grids which have been proven in [29].

Lemma 3.6.1. *On an $m_1 \times m_2 \times m_3$ grid, any unlabeled MRPP can be solved using $O(m_1 + m_2 + m_3)$ makespan.*

Lemma 3.6.2. *On an $m_1 \times m_2 \times m_3$ grid, if the underestimated makespan of an MRPP instance is d_g (usually computed by ignoring the inter-robot collisions), then this instance can be solved using $O(d_g)$ makespan.*

We proceed to analyze the time complexity of GRH3D on $m_1 \times m_2 \times m_3$ grids. The dominant parts of GRH3D are computing the perfect matchings and solving unlabeled MRPP. First we analyze the running time for computing the matchings. Finding m_3 "wide column" matchings runs in $O(m_3 m_1^2 m_2^2)$ deterministic time or $O(m_3 m_1 m_2 \log(m_1 m_2))$ expected time. We apply GRA2D for simulating "wide column" shuffle, which requires $O(m_3 m_1^2 m_2)$ deterministic time or $O(m_1 m_2 m_3 \log m_1)$ expected time. Therefore, the total time complexity for Rubik Table part is $O(m_3 m_1^2 m_2^2 + m_1^2 m_2 m_3)$. For unlabeled MRPP, we can use the max-flow based algorithm [105] to compute the makespan-optimal solutions. The max-flow portion can be solved in $O(n|E|T) = O(n^2(m_1 + m_2 + m_3))$ where $|E|$ is the number of edges and $T = O(m_1 + m_2 + m_3)$ is the time horizon of the time expanded graph [101]. Note that any unlabeled MRPP can be applied, for example, the algorithm in [106] is distance-optimal but with convergence time guarantee.

Note that the choice of 2D plane can also be x-z plane or y-z plane. In addition, one can also perform two "wide column" shuffles plus one z shuffle, which yields $2(2m_1 + m_2) + m_3$

number of shuffles and $O(m_1 m_2 m_3^2 + m_3 m_1 m_2^2)$. This requires more shuffles but shorter running time.

Next, we derive the optimality guarantee.

Proposition 3.6.1 (Makespan Upper Bound)**.** *GRH3D returns solution with worst makespan* $3m_1 + 4m_2 + 4m_3 + o(m_1)$.

Proof. The Rubik Table portion has a makespan of $(2m_3 + 2m_2 + m_1) + o(m_1)$. For the unlabeled MRPP portion, we use $m_1 + m_2 + m_3$, which is the maximum grid distance between any two nodes on the $m_1 \times m_2 \times m_3$ grid, as a conservative makespan upper bound. Therefore, the total makespan upper bound is $2(m_1 + m_2 + m_3) + (2m_3 + 2m_2 + m_1) + o(m_1) = 3m_1 + 4m_2 + 4m_3 + o(m_1)$. When $m_1 = m_2 = m_3$ for cubic grids, it turns out to be $11m_3 + o(m_3)$. □

We note that the unlabeled MRPP upper bound is actually a very conservative estimation. In practice, for such dense instances, the unlabeled MRPP usually requires much fewer steps. Next, we analyze the makespan when start and goal configurations are uniformly randomly generated based on the well-known *minimax grid matching* result.

Theorem 3.6.2 (Multi-dimensional Minimax Grid Matching [107])**.** *For* $k \geq 3$, *consider* N *points following the uniform distribution in* $[0, N^{1/k}]^k$. *Let* $\mathcal{L}$ *be the minimum length such that there exists a perfect matching of the* N *points to the grid points that are regularly spaced in the* $[0, N^{1/k}]^k$ *for which the distance between every pair of matched points is at most* $\mathcal{L}$. *Then* $\mathcal{L} = O(\log^{1/k} N)$ *with high probability.*

Proposition 3.6.2 (Asymptotic Makespan)**.** *GRH3D returns solutions with* $m_1 + 2m_2 + 2m_3 + o(m_1)$ *asymptotic makespan for MRPP instances with* $\frac{m_1 m_2 m_3}{3}$ *random start and goal configurations on 3D grids, with high probability. Moreover, if* $m_1 = m_2 = m_3$, *GRH3D returns* $5m_3 + o(m_3)$ *makespan-optimal solutions.*

Proof. First, we point out that the theorem of minimax grid matching (Theorem 3.6.2) can be generalized to any rectangular cuboid as the matching distance mainly depends on

the discrepancy of the start and goal distributions, which does not depend on the shape of the grid. Using the minimax grid matching result, the matching distance from a random configuration to a centering configuration, which is also the underestimated makespan of unlabeled MRPP, scales as $O(\log^{1/3} m_1)$. Using the Lemma 3.6.2, it is not difficult to see that the matching obtained this way can be readily turned into an unlabeled MRPP plan without increasing the maximum per robot travel distance by much, which remains at $o(m)$. Therefore, the asymptotic makespan of GRH3D is $m_1 + 2m_2 + 2m_3 + O(\log^{1/3} m_1) = m_1 + 2m_2 + 2m_3 + o(m_1)$ with high probability. If $m_1 = m_2 = m_3$, the asymptotic makespan is $5m_3 + o(m_3)$, with high probability. □

Proposition 3.6.3 (Asymptotic Makespan Lower Bound)**.** *For MRPP instances on* $m_1 \times m_2 \times m_3$ *grids with* $\Theta(m_1 m_2 m_3)$ *random start and goal configurations on 3D grids, the makespan lower bound is asymptotically approaching* $m_1+m_2+m_3$*, with high probability.*

Proof. We examine two opposite corners of the $m_1 \times m_2 \times m_3$ grid. At each corner, we examine an $\alpha m_1 \times \alpha m_2 \times \alpha m_3$ sub-grid for some constant $\alpha \ll 1$. The Manhattan distance between a point in the first sub-grid and another point in the second sub-grid is larger than $(1 - 6\alpha)(m_1 + m_2 + m_3)$. As $m \to \infty$, with $\Theta(m^3)$ robots, a start-goal pair falling into these two sub-grids can be treated as an event following binomial distribution $B(k, \alpha^6)$ when $m \to \infty$. The probability of having at least one success trial among n trials is $p = 1 - (1 - \alpha^6)^n$, which goes to one when $m \to \infty$.

Because $(1 - x)^y < e^{-xy}$ for $0 < x < 1$ and $y > 0$, $p > 1 - e^{-\alpha^6 n}$. Therefore, for arbitrarily small α, we may choose m_1 such that p is arbitrarily close to 1. The probability of a start-goal pair falling into these two sub-grids is asymptotically approaching one , meaning that the makespan goes to $(1 - 6\alpha)(m_1 + m_2 + m_3)$. □

Corollary 3.6.1 (Asymptotic Makespan Optimality Ratio)**.** *GRH3D yields asymptotic* $1 + \frac{m_2+m_3}{m_1+m_2+m_3}$ *makespan optimality ratio for MRPP instances with* $\Theta(m_1 m_2 m_3) \leq \frac{m_1 m_2 m_3}{3}$ *random start and goal configurations on 3D grids, with high probability.*

Proof. If $n < \frac{m_1 m_2 m_3}{3}$, we add virtual robots with randomly generated start and use the same for the goal, until we reach $n = \frac{m_1 m_2 m_3}{3}$ robots, which then allows us to invoke Proposition 3.6.2. In viewing Proposition 3.6.2 and Proposition 3.6.3, solution computed by GRH3D guarantees a makespan optimality ratio of $\frac{m_1+2m_2+2m_3}{m_1+m_2+m_3} = 1 + \frac{m_2+m_3}{m_1+m_2+m_3}$ as $m_3 \to \infty$. Moreover, if $m_1 = m_2 = m_3$, GRH3D returns $\frac{5}{3}$ makespan-optimal solution. □

Corollary 3.6.2 (Asymptotic Optimality, Fixed Height)**.** *For an MRPP on an $m_1 \times m_2 \times K$ grid, $m_1 > m_2 \gg K$, and $\frac{1}{3}$ robot density, the GRH3D algorithm yields $1 + \frac{m_1}{m_1+m_2}$ optimality ratio, with high probability. If $m_1 = m_2$, the asymptotic makespan optimality ratio is 1.5.*

Theorem 3.6.3. *Consider an k-dimensional cubic grid with grid size m. If robot density is less than $1/3$ and start and goal configurations are uniformly distributed, generalizations to GRA2D can solve the instance with asymptotic makespan optimality being $\frac{2^{k-1}+1}{k}$.*

Proof. By theorem Theorem 3.6.2, the unlabeled MRPP takes $o(m)$ makespan (note for $k = 1, 2$, the minimax grid matching distance is still $o(m)$ [97]). Extending proposition 3.6.3 to k-dimensional grid, the asymptotic lower bound is mk. We now prove that the asymptotic makespan $f(k)$ is $(2^{k-1}+1)m+o(m)$ by induction. The Rubik Table algorithm solves a d-dimensional problem by using two 1-dimensional shuffles and one $(k-1)$-dimensional "wide column" shuffle. Therefore, we have $f(k) = 2m + f(k-1)$. It's trivial to see $f(1) = m + o(m), f(2) = 3m + o(m)$, which yields that $f(k) = 2^{k-1}m + m + o(m)$ and makespan optimality ratio being $\frac{2^{k-1}+1}{k}$. □

3.7 Simulation Experiments

We thoroughly evaluated GRA-based algorithms and compared them with many similar algorithms. We mainly highlight comparisons with Explicit Estimation Conflict-Based Search (EECBS)(w=1.5) [61], Lazy Constraints Addition Search for Multi-Agent Pathfind-

ing (LaCAM) [65] and Data-Driven MRPP Algorithm (DDM) [62]. These two methods are, to our knowledge, two of the fastest near-optimal MRPP solvers. Beyond EECBS and DDM, we considered a state-of-the-art polynomial algorithm, push-and-swap [56], which gave fairly sub-optimal results: the makespan optimality ratio is often above 100 for the densities we examine.

As a reader's guide to this section, in subsection 3.7.1, as a warm-up, we show the 2D performance of all GRA-based algorithms at their baseline, i.e., without any efficiency-boosting heuristics mentioned in section 3.5. In subsection 3.7.4, for 2D square grids, we show the performance of all GRA2D-based algorithms with and without the two heuristics discussed in section 3.5. We then thoroughly evaluate the performance of all versions of the GRA2D algorithm at $\frac{1}{3}$ robot density in subsection 3.7.2. Some special 2D patterns are examined in subsection 3.7.3. 3D settings are briefly discussed in subsection 3.7.5.

All experiments are performed on an Intel® Core™ i7-6900K CPU at 3.2GHz. Each data point is an average of over 20 runs on randomly generated instances unless otherwise stated. A running time limit of 300 seconds is imposed over all instances. The optimality ratio is estimated as compared to conservatively estimated makespan lower bounds. All the algorithms are implemented in C++. We choose Gurobi [108] as the mixed integer programming solver and ORtools [109] as the max-flow solver.

3.7.1 Optimality of Baseline Versions of GRA-Based Methods

First, we provide an overall evaluation of the optimality achieved by basic versions of GRM, GRLM, and GRH over randomly generated 2D instances at their maximum designed robot density. That is, these methods do not contain the heuristics from section 3.5. We test over three $m_1 : m_2$ ratios: $1 : 1$, $3 : 2$, and $5 : 1$. For GRM, different sub-grid sizes for dividing the $m_1 \times m_2$ grid are evaluated. The result is plotted in Figure 3.11. Computation time is not listed; we provide the computation time later for GRH; the running times of GRM, GRLM, and GRH are all similar. The optimality ratio is computed as the ratio

between the solution makespan and the longest Manhattan distance between any pair of start and goal, which is conservative.

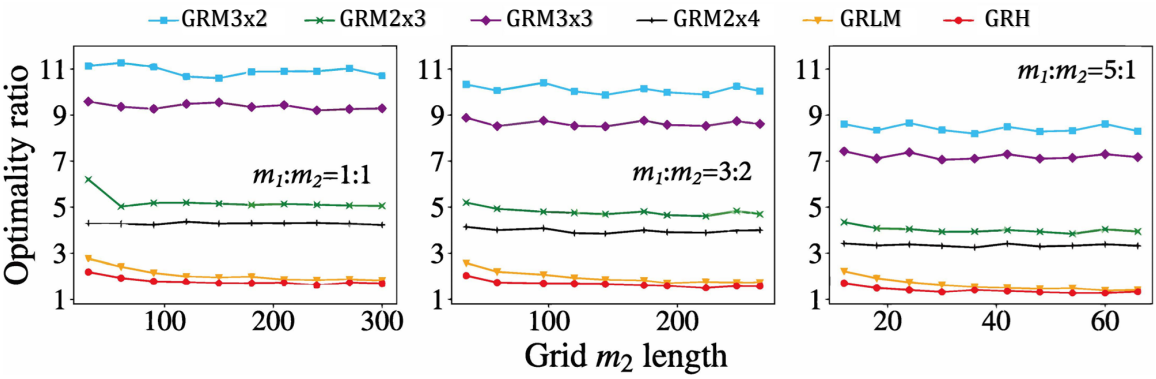


Figure 3.11: Makespan optimality ratio for GRM (3x2, 2x3, 3x3, 2x4), GRLM, and GRH at their maximum design density, for different m_2 values and $m_1 : m_2$ ratios. The largest GRM problems have $90,000$ robots on a 300×300 grid.

GRM does better and better on the optimality ratio as the sub-grid size ranges from 3×2, 3×3, 2×3, and 2×4, dropping to just above 3. In general, using "longe" sub-grids for the shuffle will decrease the optimality ratio because there are opportunities to reduce the overhead. However, the time required for computing the solutions for all possible configurations grows exponentially as the size of the sub-grids increases.

On the other hand, both GRLM and GRH achieve a sub-2 optimality ratio in most test cases, with the result for GRH dropping below 1.5 on large grids. For all settings, as the grid size increases, there is a general trend of improvement of optimality across all methods/grid aspect ratios. This is due to two reasons: (1) the overhead in the shuffle operations becomes relatively smaller as grid size increases, and (2) with more robots, the makespan lower bound becomes closer to $m_1 + m_2$. Lastly, as $m_1 : m_2$ ratio increases, the optimality ratio improves as predicted. For many test cases, the optimality ratio for GRH at $m_1 : m_2 = 5$ is around 1.3.

# of Robots	5	10	15	20	25
Optimality Gap	1	1.0025	1.004	1.011	1.073

Table 3.2: Optimality gap investigation on 5×5 grids

The exploration of optimality gaps is conducted on 5×5 grids, as shown in Table 3.2. For every specified number of robots, we create 100 random instances and employ the ILP solver to solve them. The optimality gap is then assessed by calculating the average ratio between the optimal makespan and the makespan lower bound. The optimality gap widens with higher robot density.

3.7.2 Evaluation and Comparative Study of GRH

3.7.2.1 Impact of Grid Size

For our first detailed comparative study of the performance of GRA,GRLM and GRH at 100%, $\frac{1}{2}$ and $\frac{1}{3}$ density respectively, we set $m_1 : m_2 = 3 : 2$ in terms of computation time and makespan optimality ratio. We compare with DDM [62], EECBS ($w = 1.5$) [61], Push and Swap[56], LaCAM [65] in Figure 3.12-Figure 3.14. For EECBS, we turn on all the available heuristics and reasonings.

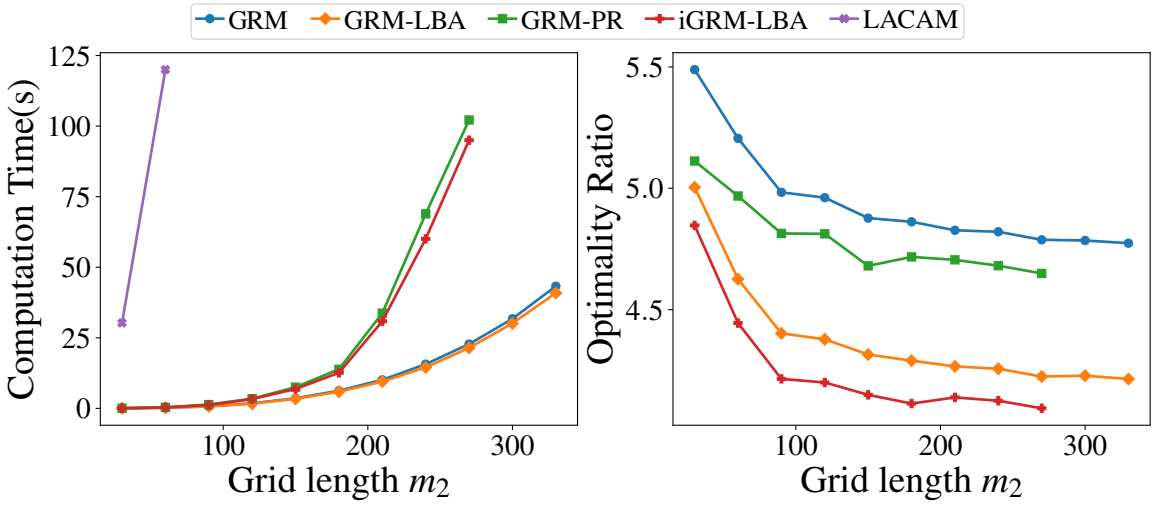


Figure 3.12: Computation time and optimality ratios on $m_1 \times m_2$ grids of varying sizes with $m_1 : m_2 = 3 : 2$ and robot density at 100% density.

When at 100% robot density, GRM methods can solve huge instances, e.g., on 450×300 grids with $135,000$ robots in about 40 seconds while none of the other algorithms can. LaCAM can only solve instances when $m_2 = 30$, though the resulting makespan optimality

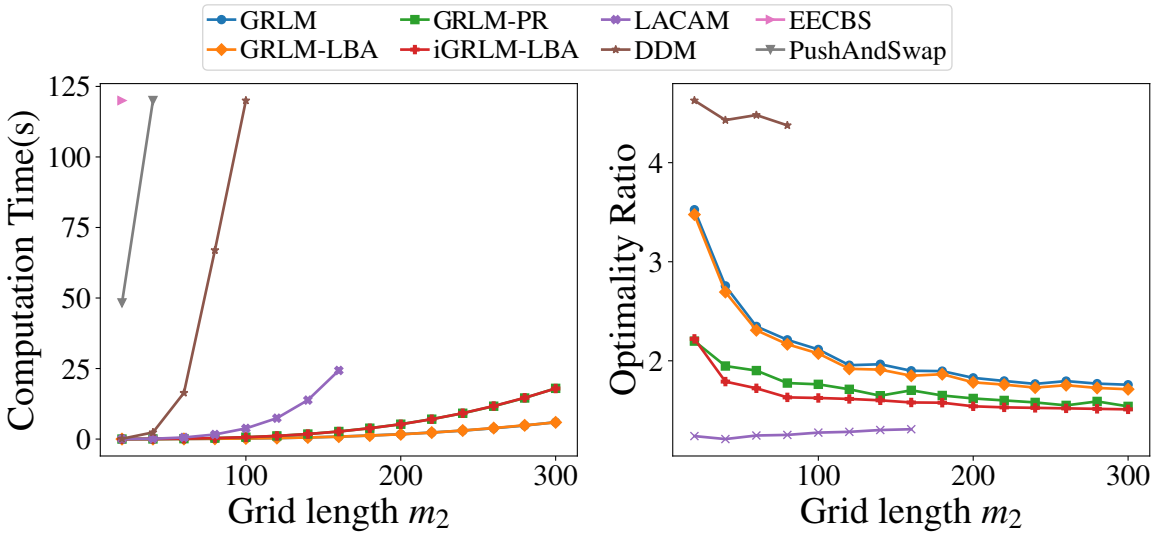


Figure 3.13: Computation time and optimality ratios on $m_1 \times m_2$ grids of varying sizes with $m_1 : m_2 = 3 : 2$ and robot density at $\frac{1}{2}$ density.

is around 2073 and thus is not shown in the figure. When at $\frac{1}{2}, \frac{1}{3}$ robot density, GRLM and GRH still are the fastest methods among all, scaling to 45,000 robots in 10 seconds while the optimality ratio is close to 1.5 and 1.3 respectively. LaCAM also achieves great scalability and is able to solve problems when $m_2 \leq 270$. However, after that, LaCAM faces out-of-memory error. This is due to the fact that LaCAM is a search algorithm on joint configurations, and the required memory grows exponentially as the number of robots. In contrast, GRH, GRLM do not have the issue and solve the problems consistently despite the optimality being worse than LaCAM. EECBS, stopped working after $m_2 = 90$ at $\frac{1}{3}$ robot density and cannot solve any instance within the time limit at 100% and $\frac{1}{2}$ robot density, while DDM stopped working after $m_2 = 180$. Push and Swap also performed poorly, and its optimality ratio is more than 30 in those scenarios and thus is not presented in the figure.

3.7.2.2 Handling Obstacles

GRH can also handle scattered obstacles and is especially suitable for cases where obstacles are regularly distributed. For instance, problems with underlying graphs like that in

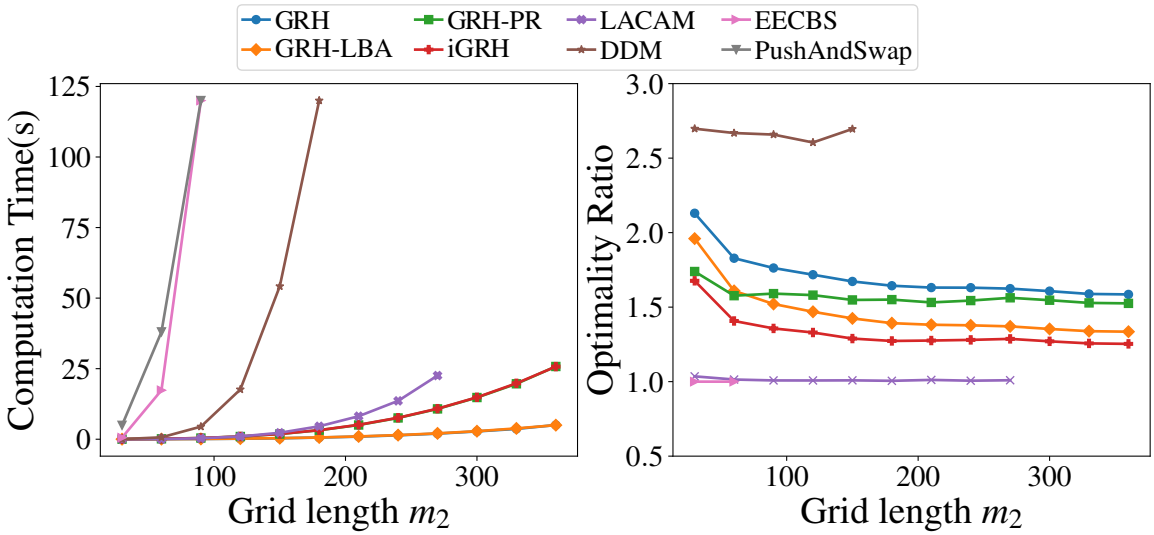


Figure 3.14: Computation time and optimality ratios on $m_1 \times m_2$ grids of varying sizes with $m_1 : m_2 = 3 : 2$ and robot density at $1/3$ density.

Fig. Figure 3.1(b), where each 3×3 cell has a hole in the middle, can be natively solved without performance degradation. Such settings find real-world applications in parcel sorting facilities in large warehouses [36, 110]. For this parcel sorting setup, we fix the robot density at $\frac{2}{9}$ and test EECBS, DDM, GRH, GRHlba, GRHpr, iGRH on graphs with varying sizes. The results are shown in Fig Figure 3.15. Note that DDM can only apply when there is no narrow passage. So we added additional "borders" to the map to make it solvable for DDM. The results are similar as before; we note that iGRH reaches a conservative optimality ratio estimate of 1.26.

3.7.2.3 Impact of Grid Aspect Ratios

In this section, we fix $m_1 m_2 = 90000$ and vary the $m_2 : m_1$ ratio between 0 (nearly one dimensional) and 1 (square grids). We evaluated four algorithms, two of which are GRH and iGRH. Now recall that GRP on an $m_1 \times m_2$ table can also be solved using $2m_2$ column shuffles and m_1 row shuffles. Adapting GRH and iGRH with $m_1 + 2m_2$ shuffles gives the other two variants we denote as GRH-LL and iGRH-LL, with "LL" suggesting two sets of longer shuffles are performed (each set of column shuffle work with columns of length

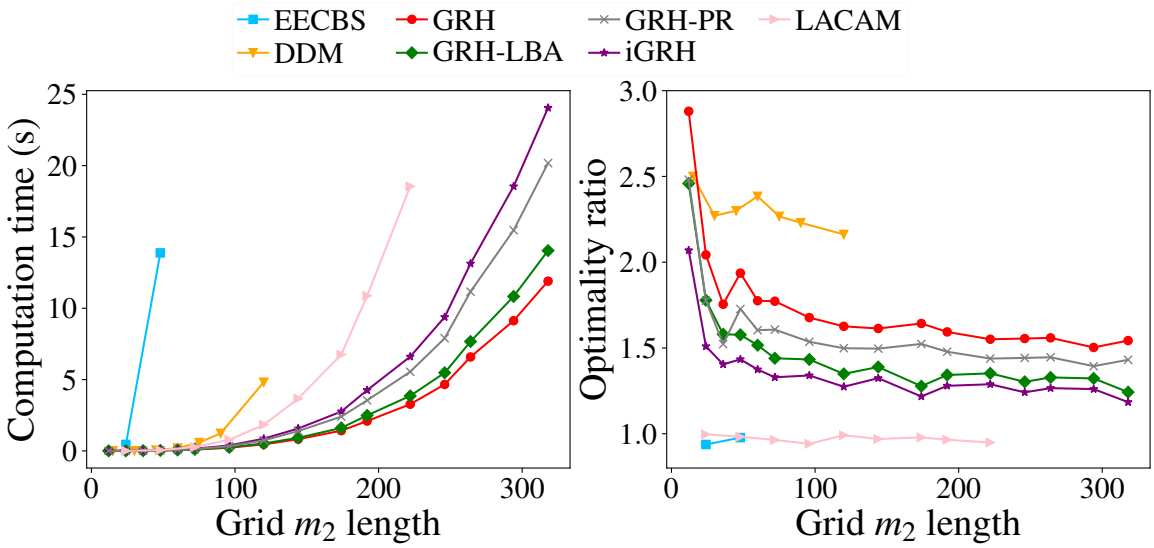


Figure 3.15: Computation time and optimality ratios on environments of varying sizes with regularly distributed obstacles at $\frac{1}{9}$ density and robots at $\frac{2}{9}$ density. $m_1 : m_2 = 3 : 2$.

m_1). The result is summarized in Figure 3.16.

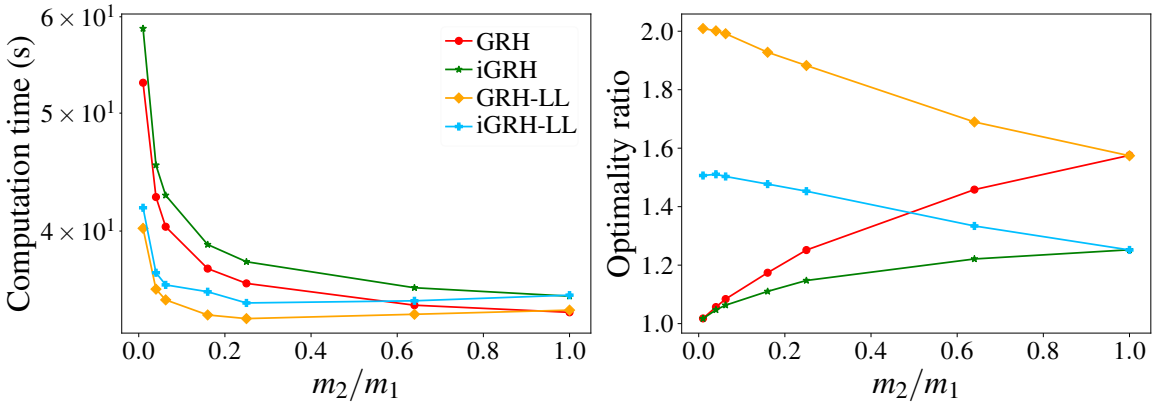


Figure 3.16: Computation time and optimality ratios on rectangular grids of varying aspect ratio and $\frac{1}{3}$ robot density.

Interestingly but not surprisingly, the result clearly demonstrates the trade-offs between computation effort and solution optimality. GRH and iGRH achieve better optimality ratio in comparison to GRH-LL and iGRH-LL but require more computation time. Notably, the optimality ratio for GRH and iGRH is very close to 1 when $m_2 : m_1$ is close to 0. As expected, iGRH does much better than GRH across the board.

3.7.3 Special Patterns

We experimented iGRH on many "special" instances, two are presented here (Figure 3.17). For both settings, we set $m_1 = m_2$. In the first, the "squares" setting, robots form concentric square rings and each robot and its goal are centrosymmetric. In the second, the "blocks" setting, the grid is divided into smaller square blocks (not necessarily 3×3) containing the same number of robots. Robots from one block need to move to another randomly chosen block. iGRH achieves optimality that is fairly close to 1.0 in the square setting and 1.7 in the block setting. The computation time is similar to that of Figure 3.15; EECBS performs well in terms of optimality, but its scalability is limited, working only on grids with $m \leq 90$. On the other hand, LaCAM exhibits excellent scalability and good optimality for block patterns, although its optimality is comparatively worse for square patterns. Other algorithms are excluded from consideration due to significantly poorer optimality; for example, DDM's optimality exceeds 9, and Push&Swap's optimality is greater than 40.

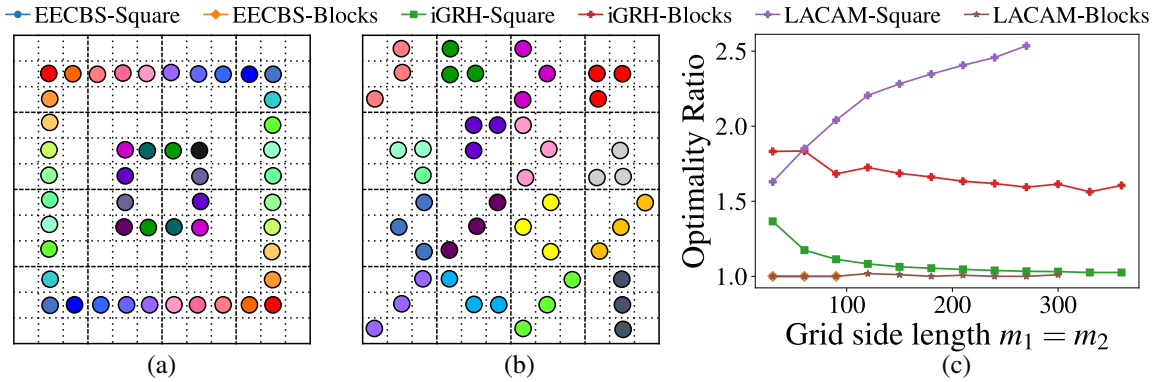


Figure 3.17: (a) An illustration of the "squares" setting. (b) An illustration of the "blocks" setting. (c) Optimality ratios for the two settings for EECBS, LaCAM, and GRHlba.

3.7.4 Effectiveness of Matching and Path Refinement Heuristics

In this subsection, we evaluate the effectiveness of heuristics introduced in section 3.5 in boosting the performance of baseline GRA-based methods (we will briefly look at the IP heuristic later). We present the performance of GRM, GRLM, and GRH at these methods' maximum design density. For each method, results on all 4 combinations with the heuristics

are included. For a baseline method X, X-LBA, X-PR, and iX mean the method with the LBA heuristic, the path refinement heuristic, and both heuristics, respectively. In addition to the makespan optimality ratio, we also evaluated *sum-of-cost* (SOC) optimality ratio, which may be of interest to some readers. The sum-of-cost is the sum of the number of steps taking individual robots to reach their respective goals. We tested the worst-case scenario, i.e., $m = m_1 = m_2$. The result is shown in Fig. Figure 3.18.

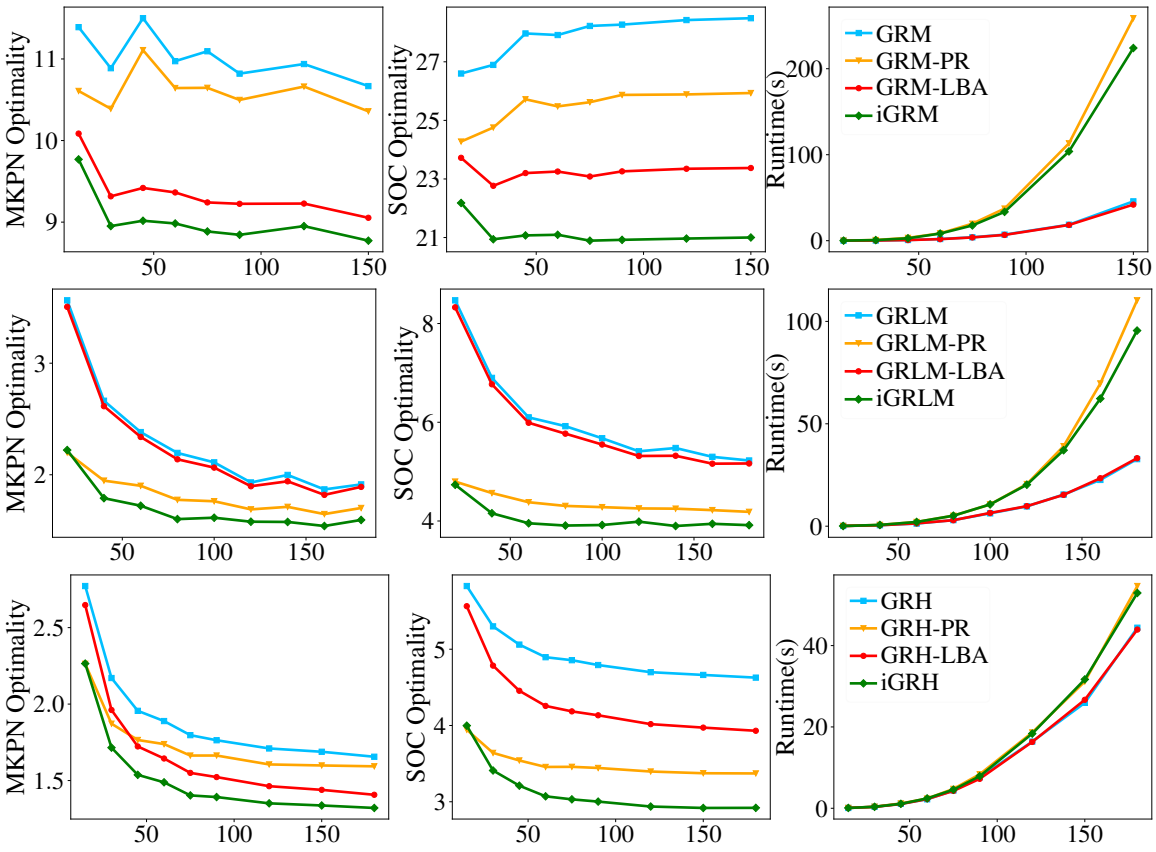


Figure 3.18: Effectiveness of heuristics in boosting GRA-based algorithms on $m \times m$ grids. For all figures, the x-axis is the grid side length m. Each row shows a specific algorithm (GRM, GRLM, and GRH). From left to right, makespan(MKPN) optimality ratio, SOC optimality ratio, and the computation time required for the path refinement routine are given.

We make two key observations based on Figure 3.18. First, both heuristics provide significant individual boosts to nearly all baseline methods (except GRLM-LBA), delivering around 10%–20% improvement on makespan optimality and 30%–40% improvement on SOC optimality. Second, the combined effect of the two heuristics is nearly additive, con-

firming that the two heuristics are orthogonal two each other, as their designs indicate. The end result is a dramatic overall cross-the-board optimality improvement. As an example, for GRH, for the last data point, the makespan optimality ratio dropped from around 1.8 for the based to around 1.3 for iGRH. In terms of computational costs, the LBA heuristic adds negligible more time. The path refinement heuristic takes more time in full and $\frac{1}{2}$ density settings but adds little cost in the $\frac{1}{3}$ density setting.

3.7.5 Evaluations on 3D Grids

In the first evaluation, we fix the aspect ratio $m_1 : m_2 : m_3 = 4 : 2 : 1$ and $\frac{1}{3}$ robot density, and examine the performance GRA methods on obstacle-free grids with varying size. Start and goal configurations are randomly generated; the results are shown in Figure 3.19. Integer Linear Programming (ILP) with 16-split heuristic and Enhanced Conflict Based Search (ECBS) computes solution with better optimality ratio but have poor scalability. In contrast, GRH3D and Grid Rearrangement with Highways in 3D using LBA (GRH3D-LBA) readily scale to grids with over $370,000$ vertices and $120,000$ robots. Both of the optimality ratio of GRH3D and GRH3D-LBA decreases as the grid size increases, asymptotically approaching ~ 1.7 for GRH3D and ~ 1.5 for GRH3D-LBA.

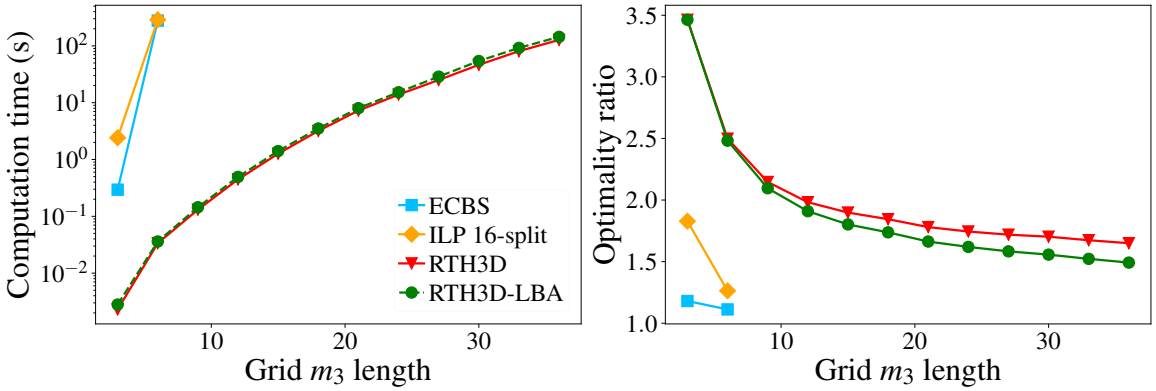


Figure 3.19: Computation time and optimality ratio on grids with varying grid size and $m_1 : m_2 : m_3 = 4 : 2 : 1$.

In a second evaluation, we fix $m_1 : m_2 = 1 : 1, m_3 = 6$ and robot density at $\frac{1}{3}$, and vary m_1. As shown in Figure 3.20, GRH3D and GRH3D-LBA show exceptional scalability and

the asymptotic 1.5 makespan optimality ratio, as predicted by 3.6.2 (m_3 is a constant and $m_1 = m_2$).

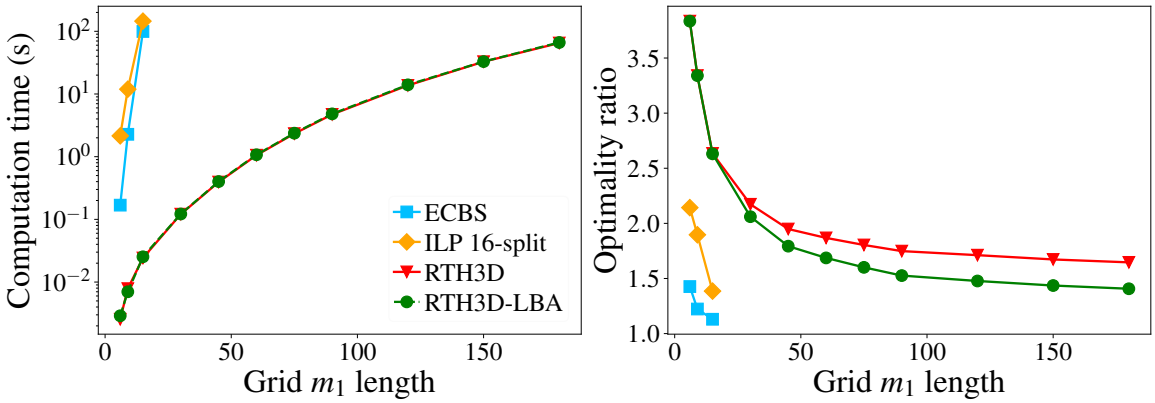


Figure 3.20: Computation time and optimality ratio on grids with $m_1 : m_2 = 1$ and $m_3 = 6$.

GRH3D and GRH3Dlba support scattered obstacles where obstacles are small and regularly distributed. As a demonstration of this capability, we simulate setting with an environment containing many tall buildings, a snapshot of which is shown Fig. Figure 3.21. These "tall building" obstacles are located at positions $(3i + 1, 3j + 1)$, which corresponds

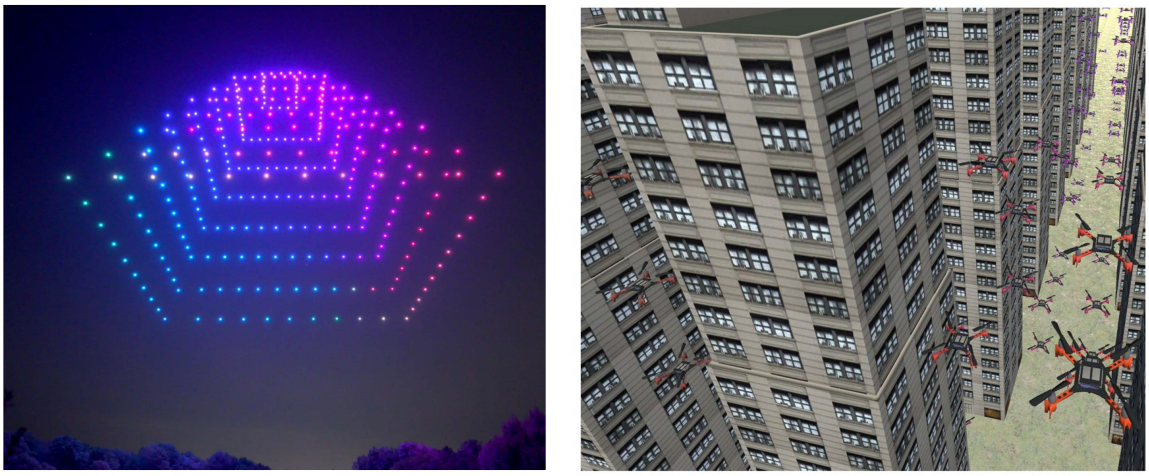


Figure 3.21: [left] A real-world drone light show where the UAVs form a 3D grid like pattern [**drone-flight**]. [right] Our simulated large UAV swarm in the Unity environment with many tall buildings as obstacles. A video of the simulations and experiments can be found at https://youtu.be/v8WMkX0qxXg

to an obstacle density of over 10%. UAV swarms have to avoid colliding with the tall buildings and are not allowed to fly higher than those buildings. We fix the aspect ratio

at $m_1 : m_2 : m_3 = 4 : 2 : 1$ and robot density at $\frac{2}{9}$. The evaluation result is shown in Figure 3.22.

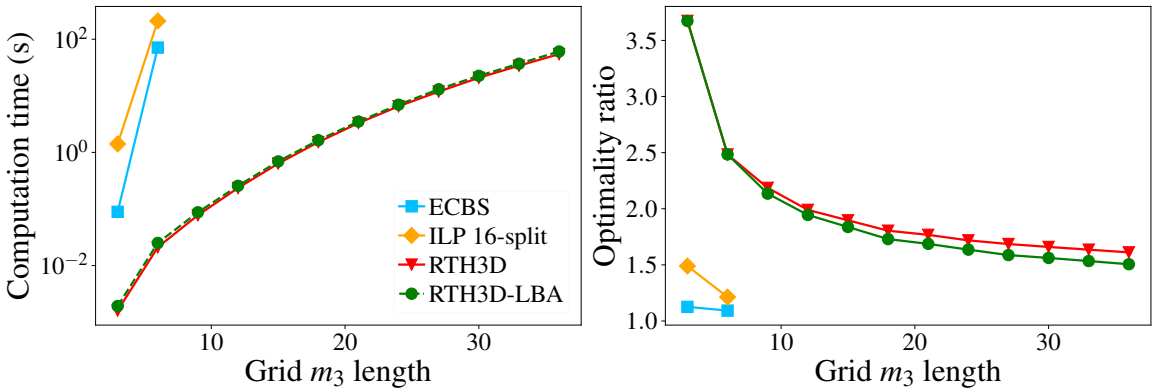


Figure 3.22: Computation time and optimality ratio on 3D environments with obstacles. $m_1 : m_2 : m_3 = 4 : 2 : 1$.

3.7.6 Impact of Robot Density

Next, we vary robot density, fixing the environment as a $120 \times 60 \times 6$ grid. For robot density lower than $\frac{1}{3}$, we add virtual robots with random start and goal configurations for perfect matching computation. When applying the shuffle operations and evaluating optimality ratios, virtual robots are removed. Therefore, the optimality ratio is not overestimated. The result is plotted in Figure 3.23, showing that robot density has little impact on the running time. On the other hand, as robot density increases, the lower bound and the makespan required by unlabeled MRPP are closer to theoretical limits. Thus, the makespan optimality ratio is actually better.

3.7.7 Special Patterns

In addition to random instances, we also tested instances with start and goal configurations forming special patterns, e.g., 3D “ring” and “block” structures (Figure 3.24). For both settings, $m_1 = m_2 = m_3$. In the first setting, “rings”, robots form concentric square rings in each x-y plane. Each robot and its goal are centrosymmetric. In the “block" setting, the grid is divided to 27 smaller cubic blocks. The robots in one block need to move to another

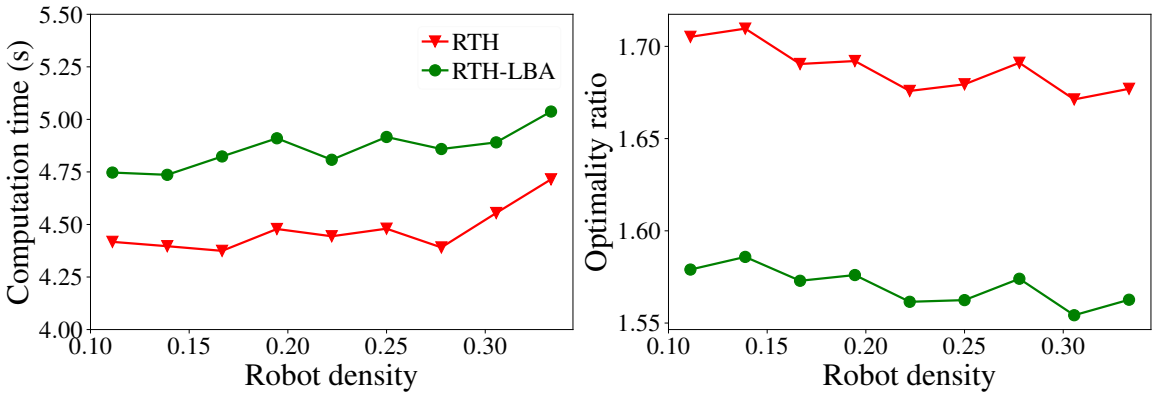


Figure 3.23: Computation time and optimality ratio on a $120 \times 60 \times 6$ grid with varying robot density.

random chosen block. Figure 3.24(c) shows the optimality ratio results of both settings on grids with varying size. Notice that the optimality ratio for the ring approaches 1 for large environments.

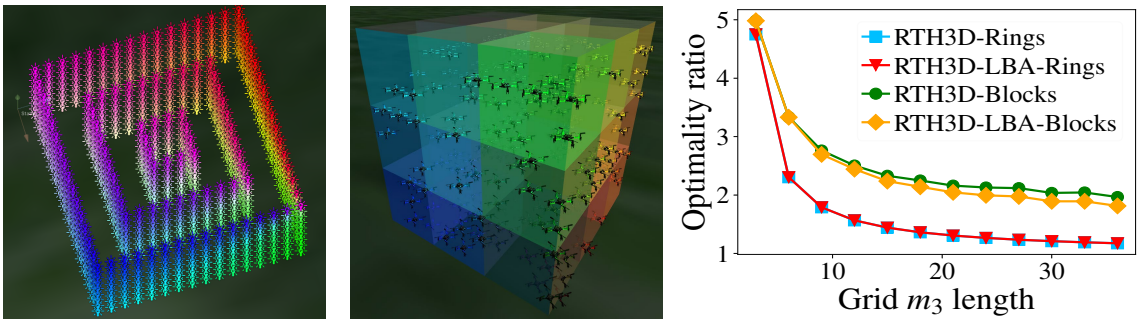


Figure 3.24: Special patterns and the associated optimality ratios.

3.7.8 Crazyswarm Experiment

Paths planned by GRA can also be readily transferred to real UAVs. To demonstrate this, 10 Crazyflie 2.0 nano quadcopters are choreographed to form the "A-R-C-R-U" pattern, letter by letter, on $6 \times 6 \times 6$ grids (Figure 3.26).

Figure 3.25: Crazyflie 2.0 nano quadcopter.

The discrete paths are computed by GRH3D. Due to relatively low robot density, shuffle operations are further optimized to be more effi-

cient. Continuous trajectories are then computed based on GRH3D plans by applying the method described in [92].

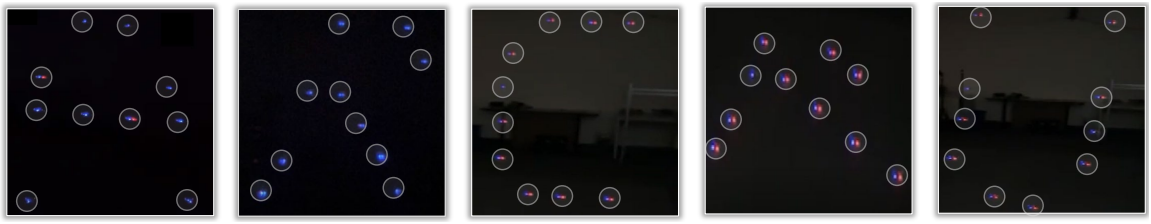


Figure 3.26: A Crazyswarm [7] with 10 Crazyflies following paths computed by GRA on $6 \times 6 \times 6$ grids, transitioning between letters ARC-RU. The figure shows five snapshots during the process. We note that some letters are skewed in the pictures which is due to non-optimal camera angles when the videos were taken.

3.8 Conclusion and Discussion

In this study, we propose to apply Grid Rearrangements [93] to solve MRPP. A basic adaptation of GRA, with a more efficient line shuffle routine, enables solving MRPP on grids at maximum robot density, in polynomial time, with a previously unachievable optimality guarantee. Then, combining GRA, a highway heuristic, and additional matching heuristics, we obtain novel polynomial time algorithms that are provably asymptotically $1 + \frac{m_2}{m_1+m_2}$ makespan-optimal on $m_1 \times m_2$ grids with up to $\frac{1}{3}$ robot density, with high probability. Similar guarantees are also achieved with the presence of obstacles and at an robot density of up to one-half. These results in 2D are then shown to readily generalize to 3D and higher dimensions. In practice, our methods can solve problems on 2D graphs with over 10^5 number of vertices and 4.5×10^4 robots to 1.26 makespan-optimal (which can be better with a larger $m_1 : m_2$ ratio). Scalability is even better in 3D. To our knowledge, no previous MRPP solvers provide dual guarantees on low-polynomial running time and practical optimality.

Limitation. While the GRA excels in achieving reasonably good optimality within polynomial time for dense instances, it does have limitations when applied to scenarios with a small number of robots or instances that are inherently easy to solve. In such cases,

although GRA remains functional, its performance may lag behind other algorithms, and its solutions might be suboptimal compared to more specialized or efficient approaches. The algorithm's strength lies in its ability to efficiently handle complex, densely populated scenarios, leveraging its unique methodology. However, users should be mindful of its relative performance in simpler instances where alternative algorithms may offer superior solutions. This limitation underscores the importance of considering the specific characteristics of the robotic system and task at hand when choosing an optimization algorithm, tailoring the selection to match the intricacies of the given problem instance.

Our study opens the door for many follow-up research directions; we discuss a few here.

New line shuffle routines. Currently, GRLM and GRH only use two/three rows to perform a simulated row shuffle. Among other restrictions, this requires that the sub-grids used for performing simulated shuffle be well-connected (i.e. obstacle-free or the obstacles are regularly spaced so that there are at least two rows that are not blocked by static obstacles in each motion primitive to simulate the shuffle). Using more rows or irregular rows in a simulated row shuffle, it is possible to accommodate larger obstacles and/or support density higher than one-half.

Better optimality at lower robot density. It is interesting to examine whether further optimality gains can be realized at lower robot density settings, e.g., $\frac{1}{9}$ density or even lower, which are still highly practical. We hypothesize that this can be realized by somehow merging the different phases of GRA so that some unnecessary robot travel can be eliminated after computing an initial plan.

Consideration of more realistic robot models. The current study assumes a unit-cost model in which an robot takes a unit amount of time to travel a unit distance and allows turning at every integer time step. In practice, robots will need to accelerate/decelerate and also need to make turns. Turning can be especially problematic and cause a significant increase in plan execution time if the original plan is computed using the unit-cost model

mentioned above. We note that GRH returns solutions where robots move in straight lines most of the time, which is advantageous compared to all existing MRPP algorithms, such as ECBS and DDM, which have many directional changes in their computed plans. it would be interesting to see whether the performance of GRA-based MRPP algorithms will further improve as more realistic robot models are adapted.

Remark 1. *Currently, our GRA-based MRPP solves are limited to a static setting whereas e-commerce applications of multi-robot motion planning often require solving life-long settings [111]. The metric for evaluating life-long MRPP is often the* throughput*, namely the number of goals reached per time step. We note that GRH also provides optimality guarantees for such settings, e.g., for the setting where $m_1 = m_2 = m$, the direct application of GRH to large-scale life-long MRPP on square grids yields an optimality ratio of $\frac{2}{9}$ on throughput.*

We may solve life-long MRPP using GRH in batches*. For each batch with n robots, GRH takes about $3m$ steps; the throughput is then $\mathcal{T}_{RTH} = \frac{n}{3m}$. As for the lower bound estimation of the throughput, the expected Manhattan distance in an $m \times m$ square, ignoring inter-robot collisions, is $\frac{2m}{3}$. Therefore, the lower bound throughput for each batch is $\mathcal{T}_{lb} = \frac{3n}{2m}$. The asymptotic optimality ratio is $\frac{\mathcal{T}_{RTH}}{\mathcal{T}_{lb}} = \frac{2}{9}$. The $\frac{2}{9}$ estimate is fairly conservative because GRH supports much higher robot densities not supported by known life-long MRPP solvers. Therefore, it appears very promising to develop an optimized Grid Rearrangement-inspired algorithms for solving life-long MRPP problems.*

CHAPTER 4
WELL-CONNECTED SET AND ITS APPLICATION TO MULTI-ROBOT PATH PLANNING

4.1 Introduction

Designing infrastructures that accommodate many mobile entities (e.g., vehicles, robots, and so on) without causing frequent congestion or deadlock is critical for improving system throughput in real-world applications, e.g., in an autonomous warehouse where many robots roam around. A good design generally entails good *environment connectivity* in some sense. This paper captures the intuition of "good connectivity" with the concept of Well-Connected Set (WCS), presents a comprehensive study on computing Largest Well-Connected Set (LWCS), and highlights the application of LWCS in MRPP.

To illustrate what a WCS is, let's consider a parking garage. It is essential to design it so vehicles can park without blocking each other, and retrieving a parked vehicle doesn't require moving other vehicles. Roughly speaking, the parking spots satisfying these requirements form a WCS, and finding an LWCS is instrumental in determining maximum parking capacity while minimizing congestion. Well-formed infrastructures based on WCSs are encountered in a broad array of real-world scenarios, including fulfillment warehouses, parking structures, storage systems [14, 112, 113, 114], and so on. These infrastructures, designed properly, efficiently facilitate the movement of the enclosed entities, ensuring smooth operations and avoiding blockages.

WCS is especially relevant to MRPP, which involves finding collision-free paths for many mobile robots [115, 116, 117, 118, 111, 66, 75]. Here, the challenge lies in finding feasible paths connecting each robot's start and target positions. The concept of WCS

becomes crucial, as it ensures that each robot can reach its destination without traversing other robots' positions, thereby guaranteeing a deadlock-free solution for easily realized prioritized planners.

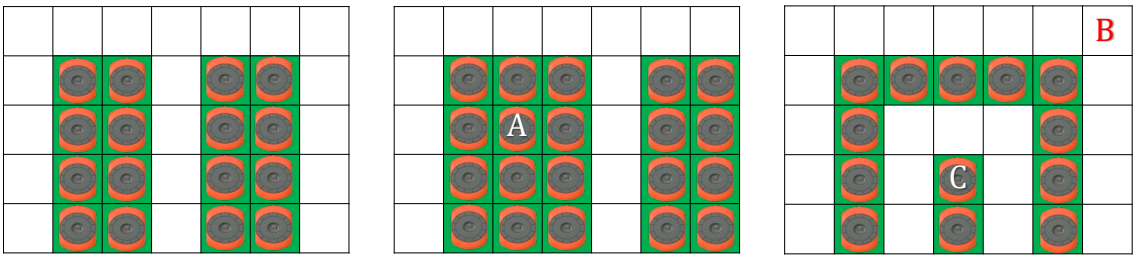
In this paper, we provide a rigorous formulation for the LWCS problem and establish its computational intractability. We then propose two algorithms for tackling this challenging combinatorial optimization problem. The first algorithm is an exact optimal approach that guarantees finding a largest well-connected vertex set, while the second algorithm offers a suboptimal but highly efficient solution for large-scale instances. As an application, LWCS readily provides prioritized MRPP with completeness guarantees.

(a) (b)

Figure 4.1: Examples of well-formed infrastructures. (a) Amazon fulfillment warehouse. (b) A typical parking lot.

Related work. The concept of well-connected vertex sets is inspired by well-formed infrastructures [75]. In well-formed infrastructures, such as parking lots and fulfillment warehouses [14], the endpoints are designed to allow multiple robots (vehicles) to move between them without completely obstructing each other, where the endpoint can be a parking slot, a pickup station, or a delivery station. Many real-world infrastructures are built in this way to benefit pathfinding and collision avoidance. Planning collision-free paths that move robots from their start positions to target positions, known as multi-robot path planning or MRPP, is generally NP-hard to optimally solve [27, 3]. In real applications, prioritized planning [15, 58] is one of the most popular methods used to find collision-free paths for

multiple moving robots where the robots are ordered into a sequence and planned one by one such that each robot avoids collisions with the higher-priority robots. The method performs well in uncluttered settings but is generally incomplete and can fail due to deadlocks in dense environments. Prior studies [75, 66] show that prioritized planning with arbitrary ordering is guaranteed to find deadlock-free paths in well-formed environments. When a problem is not well-formed, it may be possible to find a solution using prioritized planning with a specific priority ordering, as proposed in [66]. However, finding such a priority order can be time-consuming or even impossible. To our knowledge, no previous studies investigated how to efficiently design a well-formed layout that fully utilizes the workspace.



(a) (b) (c)

Figure 4.2: (a) The green cells form a WCS. Any robot parked at one of these cells does not block others' move. (b) An example of a non-WCS. Retrieving one robot, A, requires moving some other robots. (c) An example of a SWCS. B has no access to a robot at C without moving others.

Organization. We provide detailed problem formulations in section 4.2. In section 4.3, we investigate the theoretical properties and establish the proof of NP-hardness. Next, in section 4.4, we present our algorithms for finding WCS while optimizing the size of the set. In section 4.5, we explore the application of WCS in multi-robot navigation. In section 4.6, we evaluate the effectiveness of our algorithms on various maps. Finally, we conclude the paper in section 4.7 and discuss future directions for research.

4.2 Problem Formulation

4.2.1 Well-Connected Set

Let $G(V, E)$ be a connected undirected graph representing the environment with vertex set V and edge set E the edge set. A well-connected set (WCS) is defined as follows.

Definition 1 (Well-Connected Set (WCS))**.** *On a graph $G(V, E)$, a vertex set $M \subset V$ is well-connected if (i). $\forall u, v \in M, u \neq v$, there exists a path connecting u, v without passing through any $w \in M - \{u, v\}$, (ii) the induced subgraph of G by the vertex subset $V - M$ is connected.*

WCS enforces a stronger connectivity requirement. If a vertex set M satisfies (i) but violates (ii), we call M a Semi-Well-Connected Set (SWCS). Any WCS is a SWCS set but the opposite is not true (see, e.g., Fig. Figure 4.2(c)).

For a given G, there are many WCSs. We are particularly interested in computing a largest such set.

Toward that, We introduce two related concepts: the Maximal Well-Connected Set (MWCS) and the LWCS.

Definition 2 (Maximal Well-Connected Set (MWCS))**.** *A WCS M is maximal if for any $v \in V - M$, $\{v\} \bigcup M$ is not a well-connected set.*

Definition 3 (Largest Well-Connected Set (LWCS))**.** *A largest well-connected set M is a WCS with maximum cardinality.*

By definition, a LWCS is also a MWCS; the opposite is not necessarily true. In this paper, we focus on maximizing the “capacity" of well-formed infrastructures, or in other words, maximizing the cardinality of the WCS. Besides capacity, we also introduce the *path efficiency ratio* (Path Efficiency Ratio (PER)) for evaluating how good a layout is from the path-length perspective.

Definition 4 (Well-Connected Path (WCP)). *Let M be a WCS. A path $p = (p_0, ..., p_k)$ is a well-connected path connecting p_0 and p_k if its subpath $(p_1, ..., p_{k-1})$ does not pass through any vertex in M.*

If M is a WCS, a WCP connects any two vertices $u, v \in M$. We denote $d_w(u, v)$ as the shortest WCP distance between u, v and $d(u, v)$ as the shortest path distance.

Definition 5 (Path Efficiency Ratio). *Let M be a WCS of G and $u \in M$ as the reference point (i.e. I/O port), the* path efficiency ratio *w.r.t vertex u is defined as $\frac{\sum_{v \in M} d(u,v)}{\sum_{v \in M} d_w(u,v)}$.*

4.3 Theoretic Study

In this section, we investigate the property of WCS and prove that finding LWCS is NP-hard.

A vertex in an undirected connected graph is an *articulation point* (or cut vertex) if removing it (and edges through it) disconnects the graph or increases the number of connected components. We observe if a WCS contains a node v, then v should not be an articulation point so as not to violate property (ii) in the definition of WCS.

Proposition 4.3.1. *If M is a WCS, for any $M' \subseteq M, v \in M'$, v is not an articulation point of the subgraph induced by $V - M' + \{v\}$.*

Next, we investigate the property of the node in WCS and its neighbors.

Proposition 4.3.2. *Let M be a WCS. For any $v \in M$, if $|M| > |N(v)| + 1$ where $N(v)$ denotes the set of neighbors of v, then at least one of its neighbors is not in M.*

Proof. Assume $N(v) \subset M$. Because $|M| > |N(v)| + 1$, $M - (N(v) \bigcup \{v\}) \neq \emptyset$. Let $w \in M - (N(v) \bigcup \{v\})$, then every path from w to v has to pass through a neighboring node of v, which contradicts property (i) of WCS. □

We now prove that finding a LWCS is NP-hard.

Theorem 4.3.1 (Intractability)**.** *Finding the LWCS is NP-hard.*

Proof. We give the proof by using a reduction from *3SAT* [119]. Let (X, C) be an arbitrary instance of 3SAT with $|X| = n$ variables $x_1, ..., x_n$ and $|C| = m$ clauses $c_1, ..., c_m$, in which $c_j = l_j^1 \vee l_j^2 \vee l_j^3$. Without loss of generality, we may assume that the set of all literals, l_j^k's, contain both unnegated and negated forms of each variable x_i.

From the 3SAT instance, a LWCS instance is constructed as follows. For each variable x_i, we create three nodes, one node is for x_i, one node is for its negation $\bar{x}_i$, and y_i which is a gadget node. The three nodes are connected with each other with edges and form a triangle gadget. For each clause variable $c_i = l_j^1 \vee l_j^2 \vee l_j^3$, we create a node and connect it to the three nodes that are associated with $\bar{l}_j^1, \bar{l}_j^2, \bar{l}_j^3$. Finally, we create an auxiliary node z and add an edge between z and each literal node. Fig. Figure 4.3 gives the complete graph constructed from the 3CNF formula $(x_1 \vee x_2 \vee x_3) \wedge (x_2 \vee \bar{x}_3 \vee \bar{x}_4) \wedge (\bar{x}_1 \vee x_3 \vee x_4) \wedge (x_1 \vee \bar{x}_2 \vee \bar{x}_4)$.

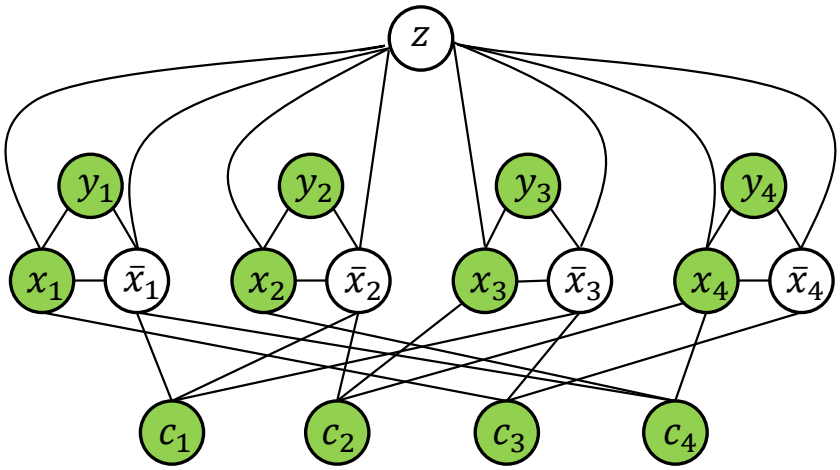


Figure 4.3: The graph derived from the 3SAT instance $(x_1 \vee x_2 \vee x_3) \wedge (x_2 \vee \bar{x}_3 \vee \bar{x}_4) \wedge (\bar{x}_1 \vee x_3 \vee x_4) \wedge (x_1 \vee \bar{x}_2 \vee \bar{x}_4)$. The green vertices form a LWCS of size 12. By setting the literals of green vertices to true, the 3SAT instance is satisfied.

To find a LWCS, we observe that for each triangle gadget formed by $(x_i, \bar{x}_i, y_i)$, the node $x_i, \bar{x}_i$ should not be selected at the same time. Otherwise, if node y_i is not selected, it would be completely isolated, which violates Proposition 4.3.1. If y_i is also selected, then every path to y_i from other nodes must always pass through either node x_i or node

$\bar{x}_i$, violating WCS definition. This implies that for each triangle, at most two nodes can be selected and added to the set, and one of the nodes must be y_i. A second observation is if the clause node $c_j = l_j^1 \vee l_j^2 \vee l_j^3$ is selected as a vertex of the WCS, the nodes $\{\bar{l}_j^1, \bar{l}_j^2, \bar{l}_j^3\}$ connected to c_j cannot be selected simultaneously. Otherwise, node c_i would be blocked by the three nodes.

If the 3SAT instance is satisfiable, let $\widetilde{X} = \{\widetilde{x}_1, ..., \widetilde{x}_n\}$ be an assignment of the truth values to the variables. Based on these observations, the MWCS of size $2n + m$ is constructed as $\widetilde{X} \bigcup C \bigcup Y$, where $Y = \{y_1, ..., y_n\}$. On the other hand, if the reduced graph has a WCS of size $2n + m$, for each triangle gadget, we can only choose two, and one of them must be the gadget node. Thus, all the gadget nodes should be selected and this contributes $2n$ nodes. For the remaining m nodes, we hope that all nodes in C are selected. To not violate the well-connectedness, for each clause node $c_j = l_j^1 \vee l_j^2 \vee l_j^3$ to be selected, at least one of the node from $(\bar{l}_j^1, \bar{l}_j^2, \bar{l}_j^3)$ should not be selected. This is equivalent to ensuring that $c_j = l_j^1 \vee l_j^2 \vee l_j^3$ is true. Therefore, if the node $\widetilde{x}_i$ is selected for each i, we set $\widetilde{x}_i = true$, then the resulting assignment $\widetilde{X}$ satisfies the 3SAT instance. □

Remark 2. *Although finding a LWCS is generally NP-hard, for certain special classes of graphs, a LWCS can be obtained easily. For example, in a complete graph, the LWCS is the set of all nodes in the graph. In a tree graph, the LWCS is the set that contains all the nodes of degree one. In a complete bipartite graph $B(U, V)$, the LWCS is formed by choosing any $|U| - 1$ vertices from U and any $|V| - 1$ vertices from V. Finding the maximum well-semi-connected set is also NP-hard, which can be proven by reduction from $3SAT$ in a similar manner to the proof for the LWCS.*

Next, we investigate the upper bound for the LWCS denoted as M^*.

Proposition 4.3.3. *Denote Δ as the maximum degree of the node of G. We have $|M^*| \leq \max(\frac{\Delta-1}{\Delta}|V|, \Delta + 1)$.*

Proof. For each $v \in M^*$, at least one of its neighbors u should be in $V - M^*$. Charge

v to u. A node $u \in V - M^*$ can be charged at most $\Delta - 1$ times. Hence, we have $|V - M^*| \geq \frac{|M^*|}{\Delta - 1}$. Since every node is either in M^* or $V - M^*$ we have $|M^*| + |V - M^*| = |V| \geq (1 + \frac{1}{\Delta - 1})|M^*|$. On the other hand, it is possible that $M^* = N(v) \bigcup \{v\}$ for a node v of degree Δ. Therefore, $|M^*| \leq \max(\frac{\Delta - 1}{\Delta}|V|, \Delta + 1)$. □

4.4 Algorithms for finding well-connected sets

4.4.1 Algorithm for Finding a MWCS

We first present an algorithm to find a MWCS in a graph, which begins by initializing two sets: M and P. Set M contains vertices currently included in the WCS, while P contains those available for adding to the set. Initially, M is empty, and P contains all the vertices of G. The algorithm selects a vertex from P and adds it to M while maintaining it as a WCS. At each step, we use Tarjan's algorithm [120] to compute the set of articulation vertices for the subgraph induced by $V - M$ and remove all the articulation vertices from P. Assume $v \in M$ and $u \in N(v)$, u is called an orphan neighbor of v if $u \notin M$ and $N(v) - u \subset M$. We iterate over all the neighboring vertices of M to remove all orphan neighbors from P, according to Proposition 4.3.2. If $|P| = 0$, the algorithm cannot add another vertex to M while keeping it well-connected. To ensure that M is indeed maximal, we check if a $v \in M$ exists, such as $M \subset N(v) \bigcup \{v\}$. If so, we add the remaining neighbor of v to M so that M is maximal. Finding the set of articulation vertices using Tarjan's algorithm takes $O(|V| + |E|)$. As a result, the algorithm for finding a MWCS takes $O(|M|(|V| + |E|)) = O(|V|(|V| + |E|))$.

Next, we establish lower bounds for the MWCS algorithm.

Proposition 4.4.1. *Denote W as the set of terminal nodes that contains nodes of degree one. Then $W \subseteq M$.*

Proof. Every terminal node u is neither an orphan neighbor nor an articulation point of the induced subgraph $G'(V - M + \{u\})$. Thus $W \subseteq M$. □

Algorithm 8: Maximal Well-Connected Set

Function MAXIMALWVS*(G)*:
  $M \leftarrow \{\}, P \leftarrow V$
  while $|P|! = 0$ **do**
    $AP \leftarrow \texttt{Tarjan}(G'(V - M))$
    $NB \leftarrow \texttt{FindOrphanNeighbors}(M)$
    $P \leftarrow P - (AP \bigcup NB)$
    $u \leftarrow \texttt{ChooseOne}(P)$
    $M.add(u)$ and $P.pop(u)$
  $M \leftarrow \texttt{AdditionalCheck}(M)$
  return M

Proposition 4.4.2. *Denote L as the length of the longest induced path of G. Then $|M| \geq \frac{|V|}{L}$.*

Proof. Let $u \in V - M$. Then u is either an orphan neighbor or an articulation point of $G'(V - M)$. For every $v \in M$, there exists a WCP p_v connecting u and v. Every vertex in $V - M$ should be passed through by at least one of those WCPs. Otherwise, let w be the vertex that is not passed through by any of the WCPs. Clearly, w cannot be an articulation point or an orphan neighbor. Then w can be added to M, which contradicts that M is maximal. Let $p'_v = p_v - v$. Then $\bigcup_{v \in M} p'_v = V - M$. Meanwhile, $|p_v| \leq diam(G(V - M)) \leq L$, where $diam(G(V - M))$ is the diameter of the induced subgraph by $V - M$. We have $|V - M| \leq \sum_{v \in M} |p'_v| \leq |M|(L - 1)$. On the other hand $|V - M| + |M| = |V|$. Hence $|M| \geq |V|/L$. □

4.4.2 Exact Search-Based Algorithm

We now establish a complete search algorithm to find a LWCS.

A naive algorithm for finding the LWCS starts by initializing a variable k to 1. The algorithm then searches for a WCS of size k in the graph. If such a set exists, it is considered a candidate for the LWCS. The algorithm then proceeds to search for a WCS of size $k + 1$. If such a set is found, it is taken as the new candidate for the LWCS. If no WCS of size k is found, the algorithm terminates and returns the WCS of size $k - 1$ as the LWCS. This

algorithm uses a simple iterative approach and keeps track of the size of the LWCS found so far. By searching for larger WCS and updating the candidate accordingly, it ensures that it finds the LWCS. When searching a well-connected vertex set of size k, there are $\binom{|V|}{k}$ possible subsets. For each subset M with $|M| = k$, we could run k times of BFS to check if the subgraph induced by $V - M + \{v\}$ is connected for each $v \in M$, in order to examine if it is a WCS. The naive algorithm runs in $O(2^{|V|}|V|(|V| + |E|))$. To improve the efficiency of the algorithm, we propose a search-based algorithm.

The algorithm is shown in algorithm 9. The algorithm employs a depth-first search (DFS) approach to exhaustively explore all possible vertex sets and select the one with the maximum size and the highest path efficiency ratio (PER). The algorithm starts by defining three empty sets, M, M^*, and $visited$, where M represents the current set of vertices being explored, M^* represents the set with the maximum size and highest PER found so far, and $visited$ stores all previously explored vertex sets to avoid repeated explorations. Then, the DFS search is initiated by calling the DFS function with the current set M, the set with the maximum size and highest PER found so far M^*, and the set of visited vertex sets visited as parameters. The DFS function starts by checking if the current set M has already been explored before. If it has, the function returns immediately. Otherwise, M is added to the visited set. The function then checks if the current set M is larger than the set with the maximum size and highest PER found so far M^*. If it is, then we update M as the current best M^*. If M has the same size as M^*, then the function compares their PER values, and the set with the higher PER becomes M^*.

Otherwise, the function generates a list of candidate vertices P that can be added to the current set M without violating the WCS condition. The function then loops over the candidate vertices and recursively calls the DFS function with the current set M unioned with the current candidate vertex and the same M^* and visited sets.

The algorithm continues until all possible vertex sets have been explored, or the condition for returning early is met. Similar to the algorithm for the MWCS, we also perform

additional checks. For each $v \in M$, we check if $N(v) \bigcup \{v\}$ can be larger than the current solution found to ensure that the final vertex set is the maximum one. The algorithm's time complexity is dependent on the size and density of the graph G, as well as the number of candidate vertices generated at each recursive call. In the worst-case scenario, the algorithm has a time complexity of $O(2^{|V|}(|V| + |E|))$, where V is the number of vertices in the graph G. However, the early termination condition in the DFS function helps avoid exploring unnecessary vertex sets and can significantly reduce the algorithm's execution time.

Algorithm 9: Largest Well-Connected Set

Function DFSSEARCH(G)**:**
  $M, M^*, visited \leftarrow \emptyset, \emptyset, \emptyset$
  DFS($G, M, M^*, visited$)
  $M^* \leftarrow$ AdditionalCheck(M^*)
  return M^*
Function DFS*($G, M, M^*, visited$)*:
  if $M \in visited$ **then return**
  $visited.add(M)$
  if $|M| > |M^*|$ *or* ($|M| = |M^*|$ *and* PER(M) $<$ PER(M^*)) **then** $M^* \leftarrow M$
  $AP \leftarrow$ Tarjan($G'(V - M)$)
  $NB \leftarrow$ FindOrphanNeighbor(M)
  $P \leftarrow P - (AP \bigcup NB)$
  if $|P| + |M| < |M^*|$ **then return**
  foreach v *in* P **do** DFS($G, M \bigcup \{v\}, M^*, visited$)

4.5 Applications in multi-robot navigation

We demonstrate how WCS benefits prioritized multi-robot path planning (MRPP) on graphs. In a legal move, a robot may cross an edge if the edge is not used by another robot during the same move and the target vertex is not occupied by another robot at the end of the move. The task is to plan paths with legal moves for all robots to reach their respective goals. The makespan (the time for all robots to reach their goals) and the total arrival time are two common criteria to evaluate the solution quality. Previous studies [75, 66] have established

the completeness of prioritized planning in well-formed infrastructures. Building on the foundation, we provide algorithms with completeness guarantees for non-well-formed environments.

Definition 6 (Well-Formed MRPP)**.** *An MRPP instance is well-formed if, for any robot i, a path connects its start and goal without traversing any other robots' start or goal vertex.*

Theorem 4.5.1. *Well-formed MRPP is solvable using prioritized planning with any total priority ordering [75, 66].*

When addressing non-well-formed MRPP, we adopt a simple, effective strategy similar to [115] to convert start and goal configurations to intermediate well-connected configurations so that the resulting problems are guaranteed to be solvable by prioritized planning with any total priority ordering. We call the algorithm Unlabeled Multi-Robot Path Planning (UNPP) - **un**labeled **p**rioritized **p**lanning.

We compute a MWCS/LWCS M offline. The first step in the algorithm is to assign the $2n$ vertices, $\mathcal{S}'', \mathcal{G}''$ in M as the intermediate start vertices and goal vertices. This is done by solving a min-cost matching problem using the Hungarian algorithm [121]. Collision-free paths are easy to plan in the unlabeled setting with optimality guarantees on makespan and total distance [105, 106]. The output of this function is a set of collision-free paths for the robots that route them to a well-formed configuration and the intermediate labeled starting and goal positions $\mathcal{S}', \mathcal{G}'$. The PrioritizedPlanning function is then called on the resulting intermediate starting and goal positions to generate a deadlock-free path for each robot. Finally, the paths generated by the Unlabeled Multi-Robot Path Planning and Prioritized Planning functions are concatenated to produce a final solution.

Theorem 4.5.2. *Denote n_c as the size of the LWCS of graph G, for any MRPP instance with number of robots less than $n_c/2$, regardless of the distribution of starts and goals, UNPP is complete with respect to any priority ordering.*

Algorithm 10: UNPP

Input: Starts $\mathcal{S}$, goals $\mathcal{G}$, LWCS/MWCS M

Function UNPP$(\mathcal{S}, \mathcal{G})$**:**

- $\mathcal{S}'', \mathcal{G}'' \leftarrow \texttt{Assignment}(\mathcal{S}, \mathcal{G}, M)$
- $P_s, \mathcal{S}' \leftarrow \texttt{UnlabeledMRPP}(\mathcal{S}, \mathcal{S}'')$
- $P_g, \mathcal{G}' \leftarrow \texttt{UnlabeledMRPP}(\mathcal{G}, \mathcal{G}'')$
- $P_m \leftarrow \texttt{PrioritizedPlanning}(\mathcal{S}', \mathcal{G}')$
- $solution \leftarrow \texttt{Concat}(P_s, P_g, P_m)$
- **return** $solution$

Proof. When the number of robots $n \leq n_c/2$, it is always possible to select such $\mathcal{S}', \mathcal{G}'$ where $\mathcal{S}' \bigcap \mathcal{G}' = \emptyset$. Since $\mathcal{S}' \bigcup \mathcal{G}' \subseteq M$, by the definition of WCS, $\mathcal{S}' \bigcup \mathcal{G}'$ is also a WCS. Therefore, the resulting MRPP problem which requires routing robots from $\mathcal{S}'$ to $\mathcal{G}'$ is well-formed. By Theorem. Theorem 4.5.1, prioritized planning is guaranteed to solve the subproblem using any priority ordering. □

UNPP runs in polynomial time for MRPP instances with $n \leq n_c/2$. We can use the max-flow-based algorithm [105] to solve the unlabeled MRPP, which takes $O(n|E|D(G))$ where $D(G)$ is the diameter of the graph G, if we use [101] (faster max-flow algorithm can also be used here) to solve the max-flow problem. In the worst case, an unlabeled MRPP requires $n + |V| - 1$ makespan to solve [106]. For the prioritized planning applied on well-formed instances, the makespan is upper bounded by $nD(G)$. As we use spatiotemporal A* to plan the individual paths while avoiding collisions with higher-priority robots on a time-expanded graph with edges no more than $n|E|D(G)$ and such a solution is guaranteed to exist, the worst time complexity is $O(n^2|E|D(G))$. In summary, UNPP yields worst case time complexity of $O(n^2|E|D(G))$ and its makespan is upper bounded by $2(n + |V| - 1) + nD(G)$.

For $\frac{n_c}{2} < n < n_c$, arbitrary priority ordering does not guarantee a solution. For some robots, its intermediate start vertex in $\mathcal{S}'$ has to be the intermediate goal vertex in $\mathcal{G}'$ of another robot when assigned from the WCS M. The resulting subproblem is not well-formed. However, it is possible to solve such an instance by breaking it into several sub-

problems and using specific priority ordering to solve it. To do this, we can first establish a dependency graph to determine the priority ordering of robots. The dependency graph consists n nodes representing the robots. If $s'_i = g'_j$, we add a directed edge from node i to node j, meaning that j should have higher priority than i. If the resulting dependency graph is a DAG, topological sort can be performed on it to get the priority ordering. When encountering a cycle, as $n < n_c$, we can break the cycle by moving one of the robots in this cycle to a buffer vertex in $C - \{\mathcal{S}' \bigcup \mathcal{G}'\}$ and perform the topological sort on the remaining robots.

Finally, we briefly illustrate the application of WCS in multi-robot pickup and delivery (Multi-Agent Pickup and Delivery (MAPD)) [111]. In well-formed MAPD, each robot can rest in a non-task endpoint forever without preventing other robots from going to their task endpoints, i.e. pickup stations. The layout of the endpoints forms a WCS of the graph of the environment. While it is desirable to increase the number of robots and the endpoints as many as possible to maximize space utilization, it is also important to keep a well-connected layout so that the robots will not block each other for better pathfinding. Thus, the maximum number of endpoints is equal to n_c, and at most $n_c - 1$ robots can be used in the MAPD.

4.6 Experiments

In our evaluation, we first test algorithms that compute WCS for grids and a set of benchmark maps and then perform evaluations on MRPP problems. Since the grids and maps used in our experiments are either 4-connected or 8-connected, the solution we find without additional checks will always be larger than the number of neighbors of a vertex v plus one (i.e., $|N(v)| + 1$). Therefore, we can safely omit the additional checks in our solution. All experiments are performed on an Intel® CoreTM i7-6900K CPU at 3.2GHz with 32GB RAM in Ubuntu 18.4 LTS and implemented in C++.

Side	4-Connected Grid									8-Connected Grid								
	Rand			Greedy			DFS			Rand			Greedy			DFS		
	$\|M\|$	PER	T	$\|M\|$	PER	T	$\|M\|$	PER	T	$\|M\|$	PER	T	$\|M\|$	PER	T	$\|M\|$	PER	T
5	14	0.89	0	11	0.89	0	14	0.89	18	20	0.97	0	20	0.93	0	20	0.97	336
10	55	0.60	0	52	0.69	0	**60**	0.67	600	72	0.52	0	73	0.96	0	**74**	0.85	600
15	124	0.62	0.02	122	0.73	0.04	**138**	0.71	600	161	0.60	0.04	160	0.85	0.06	**162**	0.85	600
20	220	0.73	0.07	238	0.74	0.2	**242**	0.67	600	283	0.46	0.1	280	0.82	0.3	**285**	0.85	600
25	238	0.53	0.2	372	0.64	0.7	**378**	0.72	600	**447**	0.46	0.3	434	0.86	0.9	445	0.64	600
30	487	0.53	0.4	561	0.62	2.0	**561**	0.68	600	**645**	0.47	0.7	622	0.84	2.3	642	0.54	600
35	667	0.35	0.8	765	0.58	4.7	**765**	0.71	600	**875**	0.62	1.3	842	0.79	5.3	872	0.62	600
40	872	0.47	1.4	992	0.72	9.8	**992**	0.70	600	1137	0.56	2.4	1097	0.79	11.0	**1139**	0.49	600
45	1098	0.55	2.4	1245	0.57	18.9	**1245**	0.69	600	1441	0.60	4.1	1384	0.82	22.2	**1443**	0.36	600
50	1360	0.33	3.9	1588	0.64	35.9	**1588**	0.66	600	1778	0.51	6.5	1705	0.82	39.8	**1785**	0.50	600

Table 4.1: Grid Experiment on 4/8-connected square grids. Red values are optimal DFS solutions.

4.6.1 Grid Experiments

We test the algorithms on $m \times m$ 4/8-connected grids with varying side lengths m. The result is shown in Table 4.1. In “Random" and “Greedy", we run the MWCS algorithm 50 times and return the set with the maximum size. Random randomly chooses a node from the candidates P and adds it to the set. In Greedy, to select the next candidate to add to the current WCS, we sort the candidate nodes in ascending order based on their total shortest distance from any node in the current WCS. We then select the candidate with the smallest total shortest distance as the next node to add to the WCS. To evaluate the running time of Random and Greedy, we take the average of the total execution time over the 50 runs of the algorithm. To evaluate the path efficiency of the maximum(maximal) WCS found by each algorithm, we treat each node in V as the reference point and compute the PER for each node. We then take the average of the PER values to obtain a measure of the overall path efficiency of the algorithms. In DFS, we set a time limit of 600 seconds to search for a solution. If the time limit is reached before a solution is found, we report the best solution found so far.

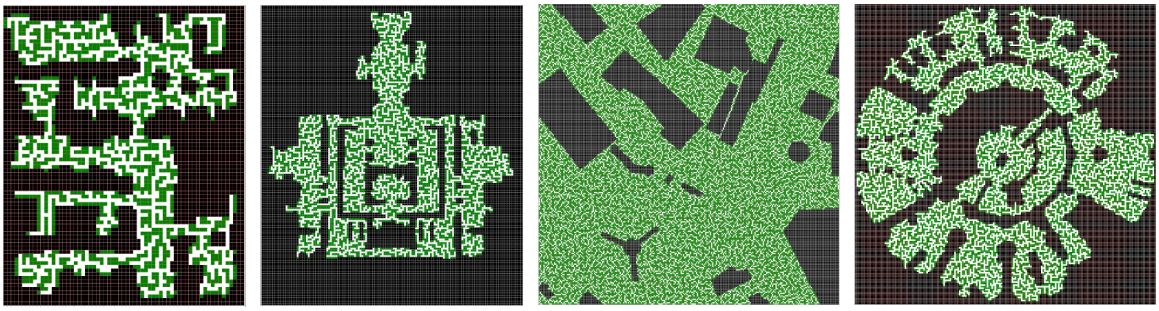
Though DFS only finds guaranteed optimal solutions on small grids, the final vertex

size it returned is usually larger than the other two methods and has better PER. Greedy finds larger WCS on 4-connected grids than Random. Interestingly, on 8-connected grids, the MWCS found by Greedy is smaller. PER of Greedy is generally better than Random.

Through linear regression, we found that on grids, the size of the maximum(maximal) WCS $|M|$ founded by these algorithms is linearly related to the number of vertices $|V|$. Specifically, on 4-connected grids, $|M| \sim 0.63|V|$, and on 8-connected grids, $|M| \sim 0.72|V|$. This means that in a square parking lot (or other well-formed infrastructures), if it is considered a 4-connected grid, at most about 63% of the space can be used for parking.

4.6.2 Benchmark Maps

We select several maps from 2D Pathfinding Benchmarks [122]. Here, we use Greedy to compute the suboptimal LWCS as most of the maps are too large to perform DFS search. For maps that are not connected, the largest connected component is used. The result is presented in Table 4.2. And some examples are shown in Fig. Figure 4.4. Our algorithm is efficient on large and complex maps with tens of thousands of vertices. The computed vertex set size is roughly 50%-60% of $|V|$ for 4-connected graphs, and 60%-70% for 8-connected graphs.



(a) (b) (c) (d)

Figure 4.4: Examples of the computed MWCS (colored in green) in different 4-connected grid maps. (a) den312d. (b) ht_chantry. (c) Shanghai_0_256. (d) lak503d. Zoom in on the digital version to see more details.

Map Name	Grid Size	$\|\mathbf{V}\|$	$\|\mathbf{E_4}\|$	$\mathbf{Time_4}$	$\|\mathbf{M_4}\|$	$\mathbf{PER_4}$	$\|\mathbf{E_8}\|$	$\mathbf{Time_8}$	$\|\mathbf{M_8}\|$	$\mathbf{PER_8}$
arena	49×49	2,054	3,955	2.33	1,113	0.68	7,813	3.76	1,455	0.52
brc202d	481×530	43,151	81,512	1,685	22,659	0.61	160,277	2,668	29,973	0.63
den001d	80×211	8,895	16,980	20.1	4,859	0.77	33,392	92.55	6,233	0.51
den020d	118×89	3,102	7,412	4.48	1,599	0.89	10,869	9.04	2,104	0.699
den312d	81×65	2,445	4,391	3.18	1,247	0.701	8,464	5.11	1,663	0.708
hrt002d	50×49	754	1,300	0.24	377	0.87	2,489	0.38	510	0.68
ht_chantry	141×162	7,461	13,963	38.87	3,889	0.45	27,222	60.95	5,183	0.37
lak103d	49×49	859	1,509	0.32	438	0.84	2,869	0.51	584	0.58
lak503d	194×194	17,953	33,781	258.89	9,484	0.58	66,734	415.60	12,482	0.48
lt_warehouse	130×194	5,534	10,397	18.67	2,895	0.87	20,306	31.72	3,858	0.63
NewYork_0_256	256×256	48,299	94,068	2,000	26,025	0.40	186,935	3,469	34,054	0.35
orz201d	45×47	745	1,342	0.23	389	0.73	2,604	0.39	513	0.61
ost003d	194×194	13,214	24,999	131.82	7,004	0.88	49,437	206.30	9,221	0.59
random-32-32-20	32×32	819	1,270	0.26	375	0.66	2,487	0.44	533	0.55
Shanghai_0_256	256×256	48,708	95,649	2,119	26,453	0.32	190,581	3,542	34,501	0.29

Table 4.2: Computed suboptimal LWCS size in selected 4/8-connected maps.

4.6.3 Evaluations of MRPP

Lastly, we examine the effectiveness of our proposed MRPP method on selected benchmarks. We compare our proposed method with two other prioritized planners, HCA*[58] and PIBT[63]. For each map, a maximal vertex set is precomputed using the Greedy method. To conduct the experiments, we randomly generate 50 instances for each map and the number of robots n. The results are shown in Fig. Figure 4.5-Figure 4.6. The experimental results demonstrate that our proposed method significantly improves the success rate compared to HCA* and PIBT. Furthermore, although the solution quality is not optimal, it is still reasonably good.

It should be emphasized that the proposed method applies to robots that consider kinodynamics. Specifically, in the case of car-like robots, a simple modification can be made by replacing the traditional A* algorithm with the hybrid A* algorithm for single-robot path planning. This adjustment enables the method to cater to car-like motion's unique dynamics and constraints.

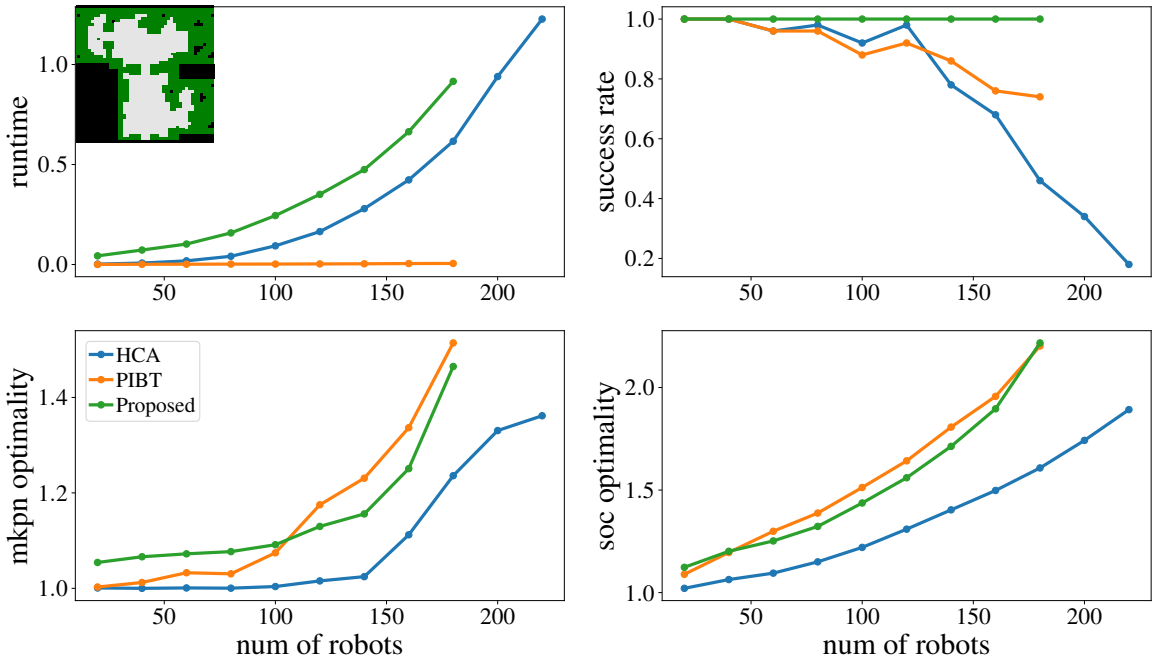


Figure 4.5: Experimental results for map orz201d, including computation time, success rate, makespan optimality, and soc optimality, for HCA, PIBT, and the proposed method.

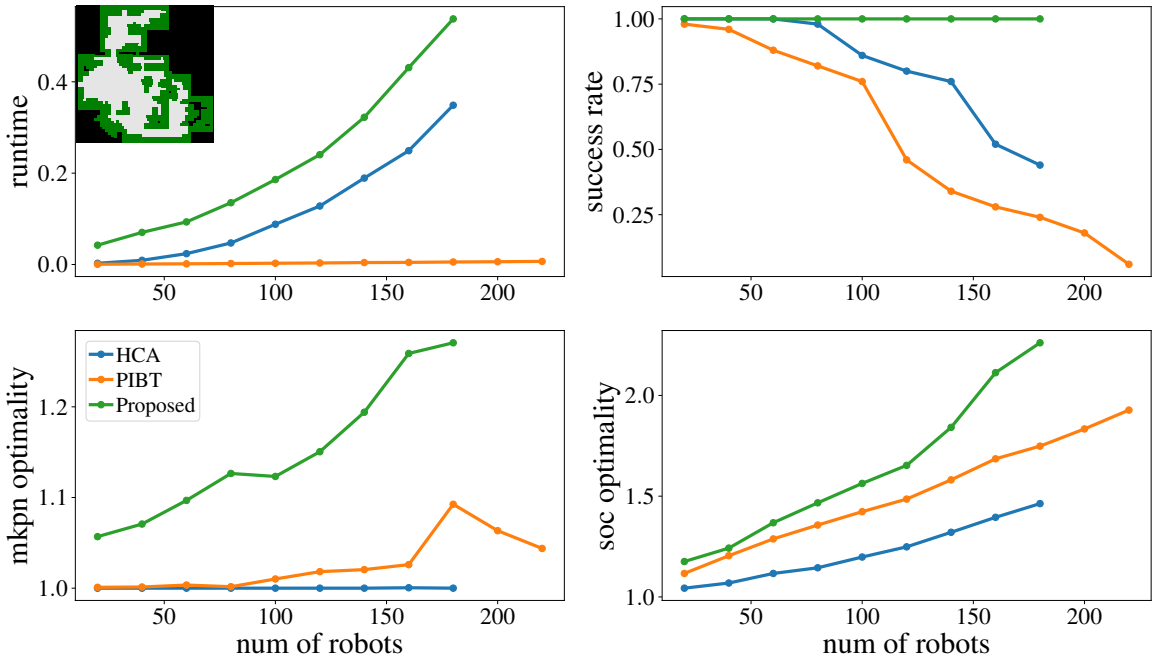


Figure 4.6: Experimental results for map hrt002d, including computation time, success rate, makespan optimality, and soc optimality, for HCA, PIBT, and the proposed method.

4.7 Conclusions

In this paper, we have presented a comprehensive study of the LWCS problem and its applications in multi-robot path planning. We provided a rigorous problem formulation and developed two algorithms, an exact optimal algorithm and a suboptimal algorithm, to solve the problem efficiently. We have shown that the problem has various real-world applications, such as parking and storage systems, multi-robot coordination, and path planning. Our algorithms have been evaluated on various maps to demonstrate their effectiveness in finding solutions. Moreover, we have integrated the LWCS problem with prioritized planning to plan paths for multi-robot systems without encountering deadlocks. Our study enhances comprehension of the relationship between multi-robot path planning complexity, the number of robots, and graph topology, laying a robust groundwork for future research in this domain. In future work, we plan to investigate the performance of our algorithms in more complex environments and explore their scalability in solving larger instances of the problem. Additionally, we aim to explore the potential of our algorithms in real-world applications and examine their robustness against uncertainties and disruptions.

CHAPTER 5
TOWARD EFFICIENT PHYSICAL AND ALGORITHMIC DESIGN OF AUTOMATED GARAGES

5.1 Introduction

The invention of automated parking systems (garages) helps solve parking issues in areas where space carries significant premiums, such as city centers and other heavily populated areas. Nowadays, parking space is becoming increasingly scarce and expensive; a spot in Manhattan could easily surpass $200,000$ USD. Developing garages supporting high-density parking that save space and are more convenient is thus highly attractive for economic/efficiency reasons.

In automated garages, human drivers only need to drop off (pick up) the vehicle in a specific I/O (Input/Output) port without taking care of the parking process. Vehicles in such a system do not require ambient space for opening the doors, and can thus be parked much closer. Moving a vehicle to a parking spot or a port is the key function for such systems. One of the solutions is to use robotic valets to move vehicles. Such systems are already commercially available, such as HKSTP [123] in Hong Kong. In such systems [124], vehicles are parked such that they may block each other, requiring multiple rearrangements to retrieve a specific vehicle. Unfortunately, little information can be found on how well these systems function, e.g., their parking/retrieval efficiency. Other solutions focus on parking for self-driving vehicles. In such an automated garage, vehicles are able to drive themselves to parking slots and ports. This makes the system more flexible but provides limited space-saving advantages, besides requiring autonomy from the vehicles.

Recently, many efficient multi-robot path planning algorithms have been proposed,

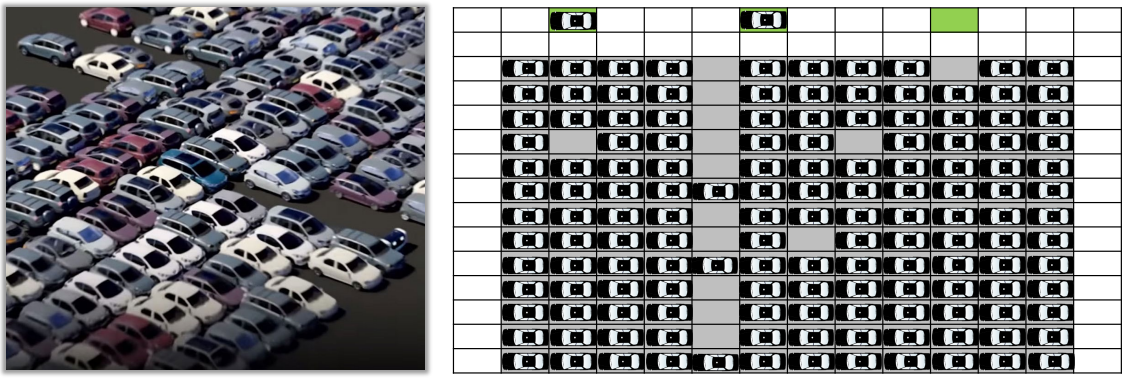


Figure 5.1: Left: an illustration of the density level of the envisioned automated garage system. Right: The grid-based abstraction with three I/O ports for vehicle dropoff and retrieval.

making it possible for lowering parking and retrieval cost using multiple robotic valets. In this chapter, we study multi-robot based parking and retrieval problem, proposing a complete automated garage design, supporting near 100% parking density, and developing associated algorithms for efficiently operating the garage.

Results and contributions. The main results and contributions of our work are as follows. In designing the automated garage, we introduce *batched vehicle parking and retrieval* (Batched Vehicle Parking and Retrieval (BVPR)) and *continuous vehicle parking and retrieval* (Continuous Vehicle Parking and Retrieval (CVPR)) problems modeling the key operations required by such a garage, which facilitate future theoretical and algorithmic studies of automated garage systems.

On the algorithmic side, we study a system that can support parking density as high as $(m_1 - 2)(m_2 - 2)/m_1 m_2$ on a $m_1 \times m_2$ grid map and allow multi-vehicle parking and retrieving, which approaches 100% parking density for large garages. Leveraging the regularity of the system, which is grid-like, we propose an optimal ILP-based method and a fast suboptimal algorithm based on sequential planning. Our suboptimal algorithm is highly scalable while maintaining a good level of solution quality, making it suitable for large-scale applications. We further introduce a shuffling mechanism to rearrange vehicles during off-peak hours for fast vehicle retrieval during rush hours, if the retrieval order can

be anticipated. Our rearrangement algorithm performs such shuffles with total time cost of $O(m_1 m_2)$ at near full garage density.

Related work. Researchers have proposed diverse approaches toward efficient high-density parking solutions. Many systems for self-driving vehicles have been studied [76, 77, 78], where vehicles are parked using a central controller and may be stacked in several rows and can block each other. These designs increase parking capacity by up to 50%. However, the retrieval becomes highly complex due to blockages and is heavily affected by the maneuverability of self-driving vehicles.

With most vehicles being incapable of self-driving, robotic valet based high-density parking systems could be a more appropriate choice. The Puzzle Based Storage (PBS) system or grid-based shuttle system, proposed originally by [79], is one of the most promising high-density storage systems. In such a system, storage units, which can be AGVs or shuttles, are movable in four cardinal directions. There must be at least one empty cell (escort). To retrieve a vehicle, one must utilize the escorts to move the desired vehicles to an I/O port. This is similar to the 15-puzzle, which is known to be NP-hard to optimally solve [80]. Optimal algorithms for retrieving one vehicle with a single escort and multiple escorts have been proposed in [79, 81]. However, these methods only consider retrieving one single vehicle at a time. Besides, the average retrieval time can be much longer than conventional aisle-based solutions. To achieve a trade-off between capacity demands and retrieval efficiency, we suggest using more escorts and I/O ports that allow retrieving and parking multiple vehicles simultaneously by utilizing recent advanced Multi-Robot Path Planning (MRPP) algorithms [82].

MRPP has been widely studied. In the static or one-shot setting [34], given a graph environment and a number of robots with each robot having a unique start position and a goal position, the task is to find collision-free paths for all the robots from start to goal. It has been proven that solving one-shot MRPP optimally in terms of minimizing either makespan or sum of costs is NP-hard [59, 3]. Solvers for MRPP can be categorized into *optimal* and

suboptimal. Optimal solvers either reduce MRPP to other well-studied problems, such as ILP[33], SAT[27] and ASP[55] or use search algorithms to search the joint space to find the optimal solution [53, 52]. Due to the NP-hardness, optimal solvers are not suitable for solving large problems. Bounded suboptimal solvers [60] achieve better scalability while still having a strong optimality guarantee. However, they still scale poorly, especially in high-density environments. There are polynomial time algorithms for solving large-scale MRPP [56], which are at the cost of solution quality. Other $O(1)$ time-optimal polynomial time algorithms [83, 125, 72, 29] are mainly focusing on minimizing the makespan, which is not very suitable for continuous settings.

Organization. The rest of the chapter is organized as follows. In section 5.2, we cover the preliminaries including garage design. In section 5.3- subsection 5.2.3, we provide the algorithms for operating the automated garage. We perform thorough evaluations and discussions of the garage system in section 5.4 and conclude with section 5.5.

5.2 Problem Definition

5.2.1 Garage Design Specification

In this study, the automated grid-based garage is a four-connected $m_1 \times m_2$ grid $\mathcal{G}(\mathcal{V}, \mathcal{E})$ (see Figure 5.1). There are n_o I/O ports (referred simply as *ports* here on) distributed on the top border of the grid for dropping off vehicles for parking or for retrieving a specific parked vehicle. A port can only be used for either retrieving or parking at a given time. Vehicles must be parked at a *parking spot*, a cell of the lower center $(m_1 - 2) \times (m_2 - 2)$ subgrid. Once a vacant spot is parked, it becomes a movable obstacle. $\mathcal{O} = \{o_1, ..., o_{n_o}\}$ is the set of ports and $\mathcal{P} = \{p_1, ..., p_{|\mathcal{P}|}\}$ is the set of parking spots.

5.2.2 Batched Vehicle Parking and Retrieval (BVPR)

Batched vehicle parking and retrieval, or BVPR, seeks to optimize parking and retrieval in a *batch* mode. In a single batch, there are n_p vehicles to park, and n_r vehicles to retrieve,

n_l parked vehicles to remain. Denote $\mathcal{C} = \mathcal{C}_p \cup \mathcal{C}_r \cup \mathcal{C}_l$ as the set of all vehicles. At any time, the maximum capacity cannot be exceeded, i.e., $|\mathcal{C}| < |\mathcal{P}|$. Time is discretized into timesteps and multiple vehicles carried by AGVs/shuttles can move simultaneously. In each timestep, each vehicle can move left, right, up, down or wait at the current position. Collisions among the vehicles should be avoided:

1. Meet collision. Two vehicles cannot be at the same grid point at any timestep: $\forall i, j \in \mathcal{C}, v_i(t) \neq v_j(t)$;

2. Head-on collision. Two vehicles cannot swap locations by traversing the same edge in the opposite direction: $\forall i, j \in \mathcal{C}, (v_i(t) = v_j(t+1) \wedge v_i(t+1) = v_j(t)) =$ false;

3. Perpendicular following collisions (see Figure 5.2). One vehicle cannot follow another when their moving directions are perpendicular. Denote $\hat{e}_i(t) = v_i(t+1) - v_i(t)$ as the moving direction vector of vehicle i at timestep t, then $\forall i, j \in \mathcal{C}, (v_i(t+1) = v_j(t) \wedge \hat{e}_i(t) \perp \hat{e}_j(t)) =$ false.

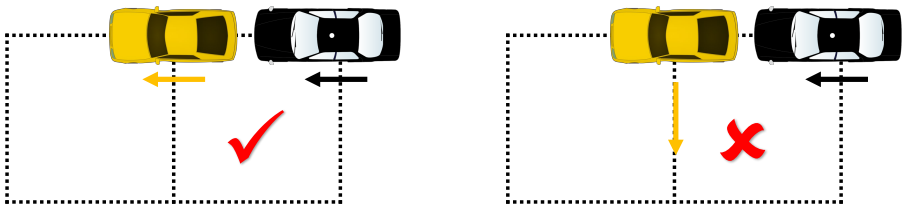


Figure 5.2: While parallel movement of vehicles in the same direction (left) is allowed, *perpendicular following* of vehicles (right) is forbidden.

Unlike certain MRPP formulations [34], we need to consider perpendicular following collisions, which makes the problem harder to solve. The task is to find a collision-free path for each vehicle, moving it from its current position to a desired position. Specifically, for a vehicle to be retrieved, its goal is a specified port. For other vehicles, the desired position is any one of the parking spots. The following criteria are used to evaluate the solution quality:

1. Average Parking and Retrieval Time (APRT): the average time required to retrieve or park a vehicle.

2. Makespan (MKPN): the time required to move all vehicles to their desired positions.

3. Average Number of Moves per task (ANM): the sum of the distance of all vehicles divided by the total number of vehicles in $\mathcal{C}_p \cup \mathcal{C}_r$.

In general, these objectives create a Pareto front [3] and it is not always possible to simultaneously optimize any two of these objectives.

5.2.3 Continuous Vehicle Parking and Retrieval (CVPR)

CVPR is the continuous version of the vehicle parking and retrieval problem. It inherits most of BVPR's structure, but with a few key differences. In this formulation, we make the following assumptions. When a vehicle $i \in \mathcal{C}_r$ arrives at its desired port, it would be removed from the environment. There will be new vehicles appearing at the ports that need to be parked (within capacity) and there would be new requests for retrieving vehicles. Besides, when a port is being used for retrieving a vehicle, other users cannot park vehicles at the port until the retrieval task is finished. Except for MKPN when the time horizon is infinite or fixed, the three criteria can still be used for evaluating the solution quality.

5.2.4 Vehicle Shuffling Problem (VSP)

In real-world garages, there are often off-peak periods (e.g., after the morning rush hours) where the system reaches its capacity and there are few requests for retrieval. If the retrieval order of the vehicles at a later time (e.g., afternoon rush hours) is known, then we can utilize the information to reshuffle the vehicles to facilitate the retrieval later. This problem is formulated as a one-shot MRPP. Given the start configuration X_I and a goal configuration X_G, we need to find collision-free paths to achieve the reconfiguration. The goal configuration is determined according to the retrieval time order of the vehicles; vehicles expected

to be retrieved earlier should be parked closer to the ports so that they are not blocked by other vehicles that will leave later.

5.3 Solving BCPR

5.3.1 Integer Linear Programming (ILP)

Building on network flow based ideas from [33, 99], we reduce BVPR to a multi-commodity max-flow problem and use integer programming to solve it. Vehicles in $\mathcal{C}_r$ have a specific goal location (port) and must be treated as different commodities. On the other hand, since the vehicles in $\mathcal{C}_p \cup \mathcal{C}_l$ can be parked at any one of the parking slot, they can be seen as one single commodity. Assuming an instance can be solved in T steps, we construct a T-step time-extended network as shown in Figure 5.3.

A T-step time-extended network is a directed network $\mathcal{N}_T = (\mathcal{V}_T, \mathcal{E}_T)$ with directed, unit-capacity edges. The network $\mathcal{N}_T$ contains $T+1$ copies of the original graph's vertices $\mathcal{V}$. The copy of vertex $u \in \mathcal{V}$ at timestep t is denoted as u_t. At timestep t, an edge (u_t, v_{t+1}) is added to $\mathcal{E}_T$ if $(u, v) \in \mathcal{E}$ or $v = u$. For $i \in \mathcal{C}_r$, we can give a supply of one unit of commodity type i at the vertex s_{i0} where s_i is the start vertex of i. To ensure that vehicle i arrives at its port, we add a feedback edge connecting its goal vertex g_i at T to its source node s_{i0}. For the vehicles that need to be parked, we create an auxiliary source node α and an auxiliary sink node β. For each $i \in \mathcal{C}_p \cup \mathcal{C}_l$, we add an edge of unit capacity connecting node α to its starting node s_{i0}. As the vehicles can be parked at any one of the parking slots, for any vertex $u \in \mathcal{P}$ we add an edge of unity capacity connecting u_T and β. A supply of $n_p + n_l$ unit of commodity of the type for the vehicles in $\mathcal{C}_p \cup \mathcal{C}_l$ can be given at the node α.

To solve the multi-commodity max-flow using ILP, we create a set of binary variables $X = \{x_{iuvt}\}, i = 0, ..., n_r, (u, v) \in \mathcal{E}$ or $u = v$, $0 \leq t \leq T$; a variable set to true means that the corresponding edge is used in the final solution. The ILP formulation is given as follows.

$$\text{Minimize} \quad \sum_{i,t,u\neq v} x_{iuvt} \tag{5.1}$$

$$\text{subject to} \quad \forall t, v, i \quad \sum_{u} x_{iuv(t-1)} = \sum_{w} x_{ivwt} \tag{5.2}$$

$$\forall t, i, v \quad \sum_{v} x_{iuvt} \leq 1 \tag{5.3}$$

$$\forall t, i, (u, v) \in \mathcal{E} \quad \sum_{i} (x_{iuvt} + x_{ivut}) \leq 1 \tag{5.4}$$

$$\forall t, i, (u, v) \perp (v, w) \sum_{i} (x_{iuvt} + x_{ivwt}) \leq 1 \tag{5.5}$$

$$\sum_{i=0}^{n_r-1} x_{ig_is_iT} + \sum_{u\in\mathcal{P}} x_{n_ru\beta T} = n_p + n_r + n_l \tag{5.6}$$

$$x_{iuvt} = \begin{cases} 0 & \text{if } i \text{ does not traverse edge } (u, v) \text{ at } t \\ 1 & \text{if } i \text{ traverses edge } (u, v) \text{ at } t \end{cases} \tag{5.7}$$

In Eq. (Equation 5.1), we minimize the total number of moves of all vehicles within the time horizon T. Eq. (Equation 5.2) specifies the flow conservation constraints at each grid point. Eq. (Equation 5.3) specifies the vertex constraints to avoid meet-collisions. In Eq. (Equation 5.4), the vehicles are not allowed to traverse the same edge in opposite directions. Eq. (Equation 5.5) specifies the constraints that forbid perpendicular following conflicts. If the programming for T-step time-expanded network is feasible, then the solution is found. Otherwise, we increase T step by step until there is a feasible solution. The smallest T for which the integer programming has a solution is the minimum makespan. As a result, the ILP finds a makespan-optimal solution minimizing the total number of moves as a secondary objective.

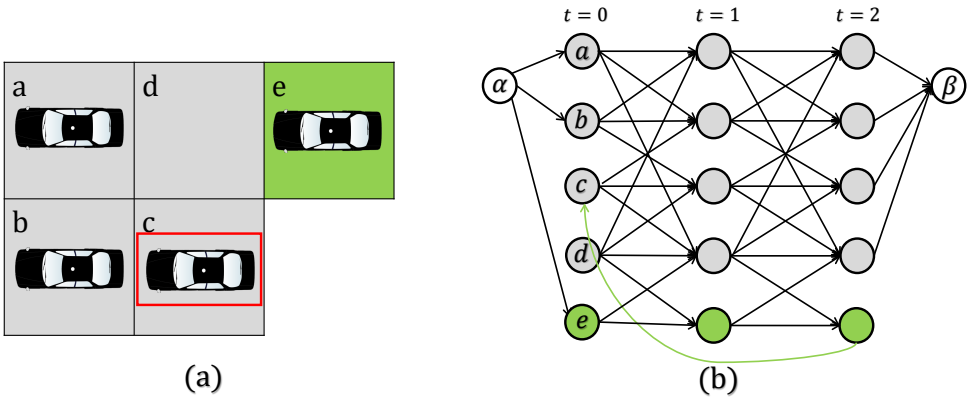


Figure 5.3: (a) A illustrative BVPR instance with 4 parking spots and one port. The vehicles within the red rectangle need to be retrieved while other vehicles should be parked in the gray areas. (b) The 2-step flow network reduced from the BVPR instance.

5.3.2 Efficient Heuristics for High-Density Planning

ILP can find makespan-optimal solutions but it scales poorly. We seek a fast algorithm that is able to quickly solve dense BVPR instances at a small cost of solution quality. The algorithm we propose is built on a single-vehicle motion primitive of retrieving and parking.

5.3.2.1 Motion Primitive for Single-Vehicle Parking/Retrieving

Regardless of the solution quality, BVPR can be solved by sequentially planning for each vehicle in $\mathcal{C}_r \cup \mathcal{C}_p$. Specifically, for each round we only consider completing one single task for a given vehicle $i \in \mathcal{C}_r \cup \mathcal{C}_p$, which is either moving i to its port or one of the parking spots. After vehicle i arrives at its destination, the next task is solved. The examples in Figure 5.4 and Figure 5.5, where the maximum capacity is reached, illustrate the method's operations.

For the retrieval scenario in Figure 5.4, the vehicle marked by the red rectangle is to be retrieved. For realizing the intuitive path indicated by the dashed lines on the left, vehicles blocking the path should be cleared out of the way, which can be easily achieved by moving those blocking vehicles one step to the left or to the right, utilizing the two empty columns.

Such a motion primitive can always successfully retrieve a vehicle without deadlocks.

For the parking scenario in Figure 5.5, we need to park the vehicle in the green port. We do so by first searching for an empty spot (escort) greedily. Using the mechanism of parallel moving of vehicles, the escort can first be moved to the column of the parking vehicle in one timestep and then moved to the position right below the vehicle in one timestep. After that, the vehicle can move directly to the escort, which is its destination.

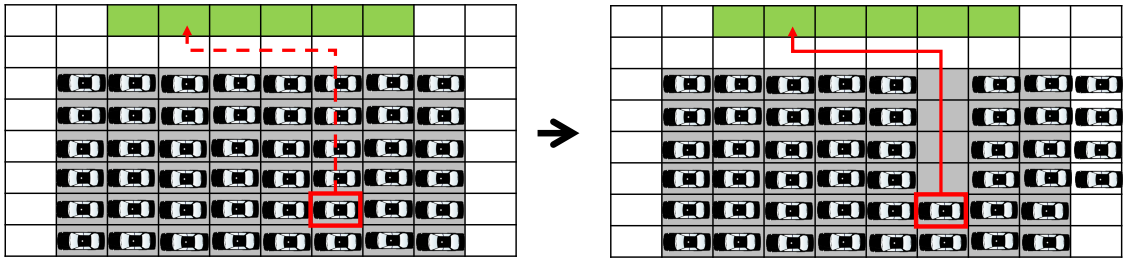


Figure 5.4: The motion primitive for retrieving a vehicle.

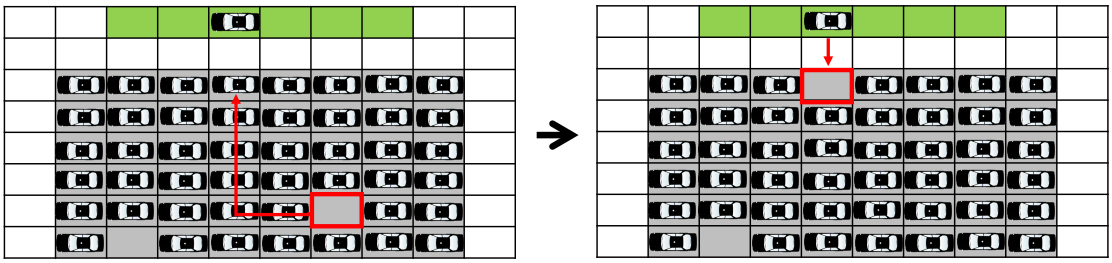


Figure 5.5: The motion primitive for parking a vehicle.

Sequential planning always returns a solution if there is one; however, when multiple parking and retrieval requests are to be executed, sequential solutions result in poor performance as measured by MKPN and ARPT.

5.3.2.2 Coupling Single Motion Primitives by MCP (Coupling Single Motion Primitives by MCP (CSMP))

MCP [103] is a robust multi-robot execution policy to handle unexpected delays without stopping unaffected robots. During execution, MCP preserves the order by which robots

visit each vertex as in the original plan. When a robot i is about to perform a move action and enter a vertex v, MCP checks whether robot i is the next to enter that vertex by the original plan. If a different robot, j, is planned to enter v next, then i waits in its current vertex until j leaves v.

We use MCP to introduce concurrency to plans found through sequential planning. The algorithm is described in algorithm 11 and algorithm 12. algorithm 11 describes the framework of CSMP. First, we find initial plans for all the vehicles in $\mathcal{C}_r \cup \mathcal{C}_p$ one by one (Line 4-6). After obtaining the paths, we remove all the waiting states and record the order of vehicle visits for each vertex in a list of queues (Line 7). Then we enter a loop executing the plans using MCP until all vehicles have finished the tasks and reach their destination (Line 8-15). In algorithm 12, if i is the next vehicle that enters vertex v_i according to the original order, we check if there is a vehicle currently at v_i. If there is not, we let i enter v_i. If another vehicle j is currently occupying v_j, we examine if j is moving to its next vertex v_j in the next step by recursively calling the function `MCPMove`. If j is moving to v_j in the next step and the moving directions of i, j are not perpendicular, we let vehicle i enter vertex v_i. Otherwise, i should wait at u_i. The algorithm is deadlock-free by construction; we omit the relatively straightforward proof due to the page limit.

Proposition 5.3.1. *CSMP is dead-lock free and always finds a feasible solution in finite time if there is one.*

5.3.2.3 Prioritization

Sequential planning can always find a solution regardless of the planning order. However, priorities will affect the solution quality. Instead of planning by a random priority order (Alg. algorithm 11 Line 4-6), when possible, we can first plan for parking since single-vehicle parking only takes two steps. After all the vehicles in $\mathcal{C}_p$ have been parked, we apply `SingleMP` to retrieve vehicles. Among vehicles in $\mathcal{C}_r$, we first apply `SingleMP`

Algorithm 11: CSMP

```
Function CSMP ():
    foreach v ∈ 𝒱, VOrder[v] ← Queue()
    InitialPlans ← {}
    for i ∈ 𝒞_p ∪ 𝒞_r do
        SingleMP(i, InitialPlans)
    Preprocess(InitialPlans, VOrder)
    while True do
        for i ∈ 𝒞 do
            mcpMoved ← Dict()
            MCPMove (i)
        if AllReachedGoal() =true then
            break
```

Algorithm 12: MCPMove

```
Function MCPMove (i):
    if i in mcpMoved then
        return mcpMoved[i]
    u_i ← current position of i
    v_i ← next position of i
    if i = VOrder[v_i].front() then
        j ← the vehicle currently at v_i
        if j=None or (MCPMove (j)=true and (u_i, v_i) ⊀ (u_j, v_j)) then
            move i to v_i
            VOrder[v_i].popfront()
            mcpMoved[i]=true
            return true
    let i wait at u_i
    mcpMoved[i]=false
    return false
```

for those vehicles that are closer to their port so that they can reach their targets earlier and will not block the vehicles at the lower row.

5.3.3 Complexity Analysis

In this section, we analyze the time complexity and solution makespan upper bound of the CSMP. In `SingleMP`, in order to park/retrieve one vehicle, we assume n_b vehicles may cause blockages and need to be moved out of the way. Clearly $n_b < n$ where $n = n_p + n_r + n_l$. Therefore, the complexity of computing the paths using `SingleMP` for all vehicles in $\mathcal{C}_r \cup \mathcal{C}_p$ is bounded by $(n_p + n_r)n$. The path length of each single-vehicle path computed by `SingleMP` is no more than $m_1 + m_2$. The makespan of the paths obtained by concatenating all the single-vehicle paths is bounded by $n_r(m_1 + m_2) + 2n_p$. This means that MCP will take no more than $n_r(m_1 + m_2) + 2n_p$ iterations. Therefore, the makespan of the solution is upper bounded by $n_r(m_1 + m_2) + 2n_p$. In each loop of MCP, we essentially run DFS on a graph that has $n = n_p + n_r + n_l$ nodes and traverse all the nodes, for which the time complexity is $O(n)$, Therefore the time complexity of CSMP is $O(n(n_r m_1 + n_r m_2 + 2n_p))$. In summary, the time complexity of CSMP is bounded by $O(n(n_r m_1 + n_r m_2 + 2n_p))$, while the makespan is upper bounded by $n_r(m_1 + m_2) + 2n_p$.

5.3.4 Extending CSMP to CVPR

CSMP can be readily adapted to solve CVPR. Similar to the BVPR version, we call `MCPMove` for each vehicle at each timestep. When a new request comes at some timestep, we compute the paths for the associated vehicles using the `SingleMP` and update the information of vertex visit order. That is, when we apply `SingleMP` on a vehicle $i \in \mathcal{C}_p \cup \mathcal{C}_r$, if vehicle j will visit vertex u at timestep t', then we push i to the queue $VOrder[u]$, where the queue is always sorted by the entering time of u. In this way, MCP will execute the plans while maintaining the visiting order. The previously planned vehicles will not be affected by the new requests and keep executing their original plan. The main drawback of

this method is that it usually has worse solution quality than replanning since the visiting order is fixed.

VSP is essentially solving a static/one-shot MRPP. On an $m_1 \times m_2$ grid, it can be solved by applying the Rubik Table algorithm [30], using no more than $2(m_2 - 2)$ column shuffles and $(m_1 - 2)$ row shuffles. As an example shown in Figure 5.6, we may use two nearby columns to shuffle the vehicles in a given column fairly efficiently, requiring only $O(m_1)$ steps [115]. The same applies to row shuffles. Depending on the number of parked vehicles, one or more multiple row/column shuffles may be carried out simultaneously. We have (straightforward proofs are omitted due to limited space)

Proposition 5.3.2. *VSP may be solved using $O(m_1 m_2)$ makespan at full garage capacity and $O(m_1 + m_2)$ makespan when the garage has $\Theta(m_1 m_2)$ empty spots and $\Omega(m_1 m_2)$ parked vehicles. In contrast, with $\Omega(m_1 m_2)$ parked vehicles, the required makespan for solving VSP is $\Omega(m_1 + m_2)$.*

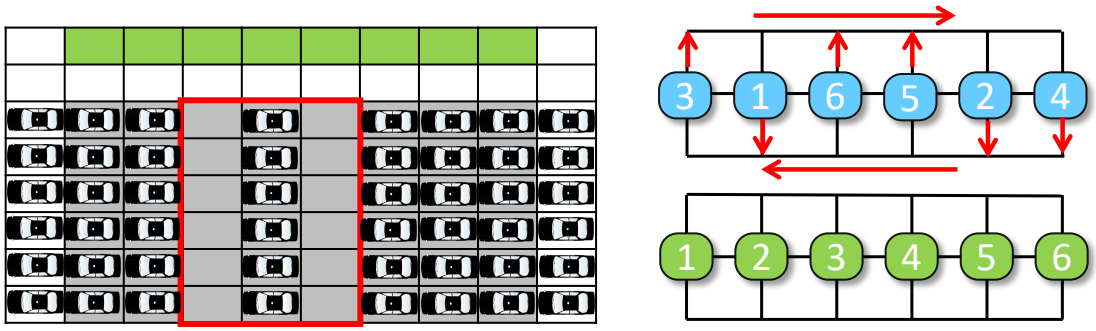


Figure 5.6: Illustration of the mechanism for "shuffling" a single row/column.

5.4 Evaluation

In this section, we evaluate the proposed algorithms. All experiments are performed on an Intel® Core™ i7-9700 CPU at 3.0GHz. Each data point is an average over 20 runs on randomly generated instances unless otherwise stated. ILP is implemented in C++ and other algorithms are implemented in CPython. A video of the simulation can be found at https://youtu.be/CZTaxnAS7TU.

5.4.1 Algorithmic Performance on BVPR

Varying grid sizes. In the first experiment, we evaluate the proposed algorithms on $m \times m$ grids with varying grid side length, under the densest scenarios: there are $(m-2)^2$ vehicles in the system and all the ports are used for either parking or retrieving ($n_p + n_r = n_o$). The result can be found in Figure 5.7. CONCAT is the method that simply concatenates the sing-vehicle paths. In rCSMP, we apply `SingleMP` on vehicles with random priority order, while in pCSMP, we apply the prioritization strategy.

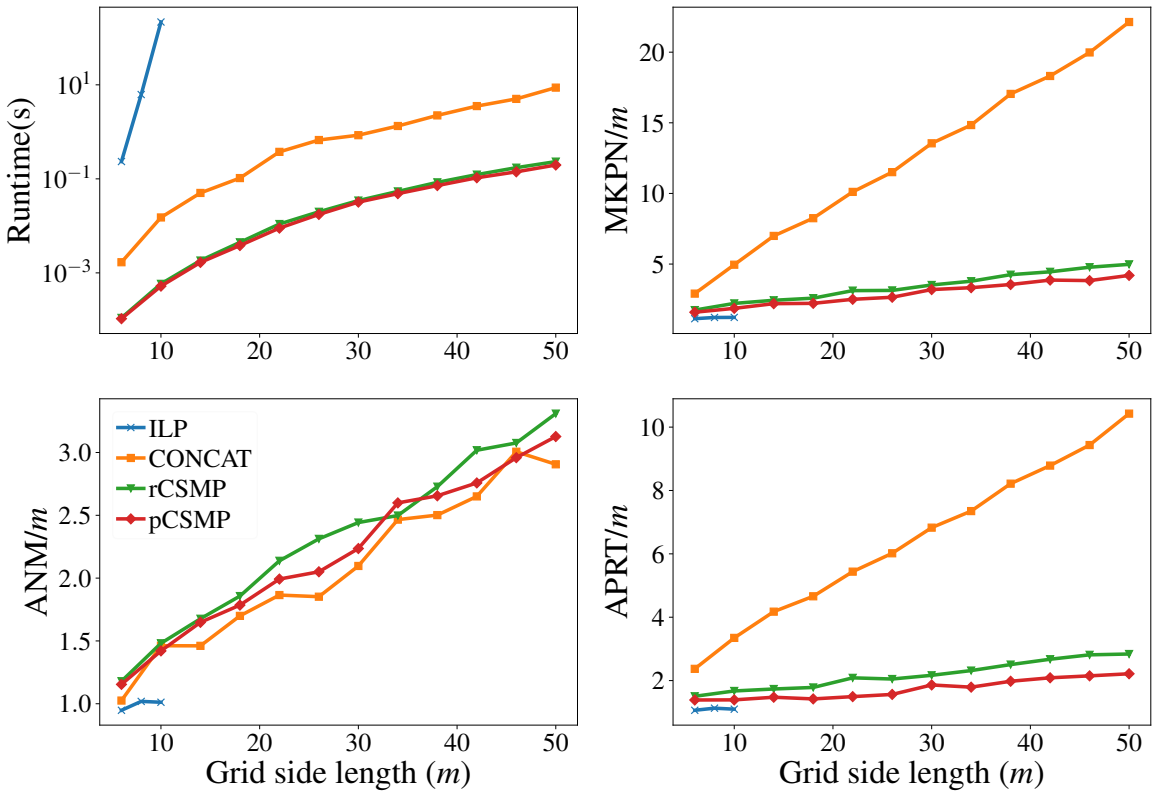


Figure 5.7: Runtime, MKPN, APRT, AVN data of the proposed methods on $m \times m$ grids under the densest scenarios.

Among the methods, ILP has the best solution quality in terms of MKPN, ANM, and APRT, which is expected since its optimality is guaranteed. However, ILP has the poorest scalability, hitting a limit with $m \leq 10$ and $n \leq 64$. CONCAT, rCSMP, and pCSMP are much more scalable, capable of solving instances on 50×50 grids with 2304 vehicles

in a few seconds. Since CONCAT just concatenates sing-vehicle paths, this results in very long paths compared to rCSMP and pCSMP; we observe that the MCP procedure greatly improves the concurrency, leading to much better solution quality. MKPN and APRT of paths obtained by CONCAT can be $10m$-$20m$ while the MKPN and APRT of the paths obtained by rCSMP and pCSMP are $2m$-$4m$. MKPN and APRT of pCSMP with prioritization strategy is about 20% lower than these of rCSMP.

Impact of vehicle density. In the second experiment, we examine the behavior of algorithms as vehicle density changes, fixing grid size at 20×20. We still let $n_p + n_r = n_o$. The result is shown in Figure 5.8. As in the previous case, ILP can only solve instances with a density below 20% in a reasonable time, while the other three algorithms can all tackle the densest scenarios. For all algorithms, vehicle density in $\mathcal{C}_l$ has limited impact on MKPN and APRT, where rCSMP and pCSMP have much better quality than CONCAT. In low-density scenarios, fewer vehicles need to move, which may cause blockages for retrieving/parking a vehicle. As a result, ANM increases as vehicle density increases.

5.4.2 Algorithmic Performance on CVPR

Random retrieving and parking. We test the continuous CSMP on a 12×12 grid with 10 ports. In each time step, if a port is available, there would be a new vehicle that need to be parked appearing at this port with probability p_p if it does not exceed the capacity. And with probability p_r this port will be used to retrieve a random parked vehicle if there is one. We simulate the following three scenarios:

(i). Morning rush hours. Initially, no vehicles are parked. There are many more requests for parking than retrieving: $p_p = 0.6, p_r = 0.01$.

(ii). Workday hours. Initially, the garage is full. Request for parking and retrieval are equal: $p_p = p_r = 0.05$.

(iii). Evening rush hours. Initially, the garage is full. Retrieval requests dominate parking: $p_p = 0.01, p_r = 0.6$.

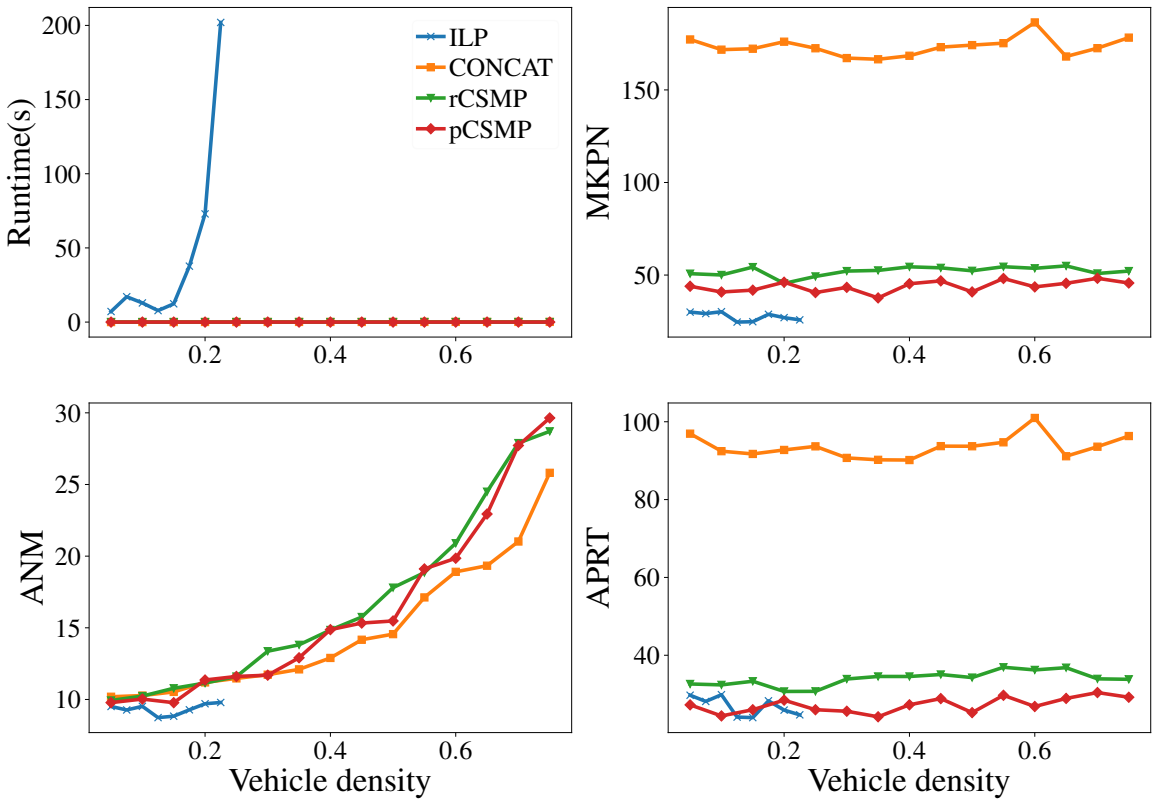


Figure 5.8: Runtime, MKPN, APRT, ANM data of the proposed methods on 20×20 grids with varying vehicle density.

The maximum number of timesteps is set to 500. We evaluate the average retrieval time, average parking time, and total number of moves under these scenarios. The result is shown in Figure 5.9. Online CSMP achieves the best performance in the morning due to fewer retrievals. On the other hand, the average retrieval time in all three scenarios is less than $2m$ and parking time is less than m. This shows that the algorithm is able to plan paths with good solution quality even in the densest scenarios and rush hours.

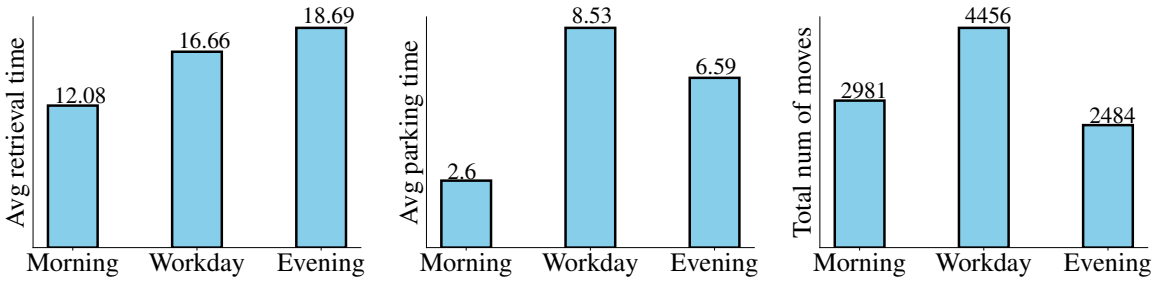


Figure 5.9: CVPR performance statistics under three garage traffic patterns.

Benefits of shuffling. In this experiment, we examine the effect of the shuffling (for solving VSP). We assume that each vehicle is assigned a retrieval priority order, as to be expected in the evening rush hours when some people go home earlier than others. We perform the *column* shuffle operations on the vehicles to facilitate the retrieval. The makespan, average number of moves, and computation time of the column shuffle operations on $m \times m$ grids with different grid sizes under the densest settings are shown in Figure 5.10(a)-(c). While the paths of shuffling can be computed in less than 1 second, the makespan of completing the shuffles scales linearly with respect to m^2 and the average number of moves scales linearly with respect to m.

After shuffling, continuous CSMP with $p_r = 1, p_p = 0$ is applied to retrieve all vehicles and compared to the case where no shuffling is performed. The outcome makespan, average number of moves per vehicle, and average retrieval time per vehicle are shown in Figure 5.10(d). Compared to unshuffled configuration, CSMP is able to retrieve all the vehicles with 30%-50% less number of moves and retrieval time (note that logarithmic scale is used to fit all data), showing that rearranging vehicles in anticipation of rush hour retrieval

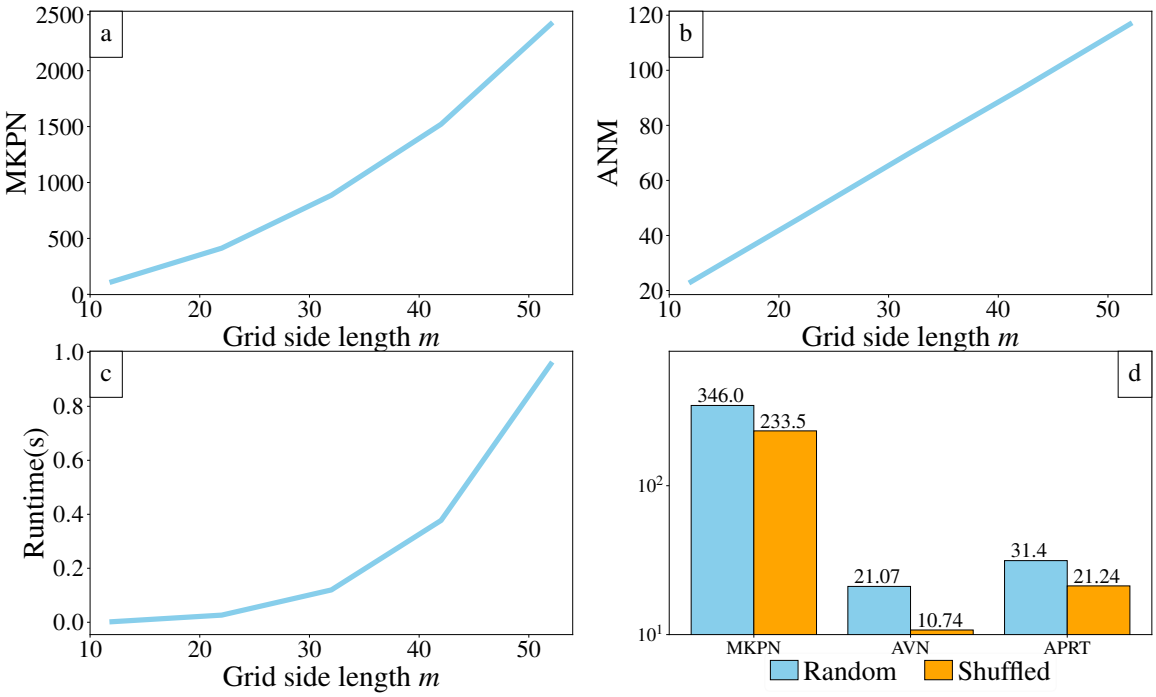


Figure 5.10: Statistics of performing the column shuffle operations on $m \times m$ grids with varying grid size.

provides significant benefits.

5.5 Conclusion and Discussions

In this chapter, we present the complete physical and algorithmic design of an automated garage system, aiming at allowing the dense parking of vehicles in metropolitan areas at high speeds/efficiency. We model the retrieving and parking problem as a multi-robot path planning problem, allowing our system to support nearly 100% vehicle density. The proposed ILP algorithm can provide makespan-optimal solutions, while CSMP algorithms are highly scalable with good solution quality. Also clearly shown is that it can be quite beneficial to perform vehicle rearrangement during non-rush hours for later ordered retrieval operations, which is a unique high-utility feature of our automated garage design.

For future work, we intend to further improve CSMP's scalability and flexibility, possibly leveraging the latest advances in MRPP/MAPF research. We also plan to extend the automated garage design from 2D to 3D, supporting multiple levels of parking. Finally, we

would like to build a small-scale test beds realizing the physical and algorithmic designs.

CHAPTER 6
DECENTRALIZED LIFELONG PATH PLANNING FOR MULTIPLE ACKERMANN CAR-LIKE ROBOTS

6.1 Introduction

The rapid development of robotics technology in recent years has made possible many revolutionary applications. One such area is multi-robot systems, where many large-scale systems have been successfully deployed, including, e.g., in warehouse automation for general order fulfillment [14], grocery order fulfillment [35], and parcel sorting [36]. However, upon a closer look at such systems, we readily observe that the robots in such systems largely live on some discretized grid structure. In other words, while we can effectively solve multi-robot coordination problems in grid-like settings, we do not yet see applications where many non-holonomic robots traverse smoothly in continuous domains due to a lack of good computational solutions. Whereas many factors contribute to this (e.g., state estimation), a major roadblock is the lack of efficient computational solutions tackling the lifelong motion planning for non-holonomic robots in continuous domains.

Toward clearing the above-mentioned roadblock, in this chapter, we proposed two algorithms specially designed to solve static/one-shot and lifelong path/motion planning tasks for Ackermann car-like robots. The basic idea behind our methods is straightforward: to enable the adaptation of discrete search strategies for car-like robots, we use a small set of fixed but representative motion primitives to transition the robots' states. These fixed motion primitives, when properly put together, yield near-optimal trajectories that "almost" connect the starts and goals for robots. Some local adjustments are then used to complete the full trajectory. The first algorithm and our main contribution in this research, *Priority-inherited Backtracking for Car-like Robots* (Priority Inheritance with Backtracking for Car-

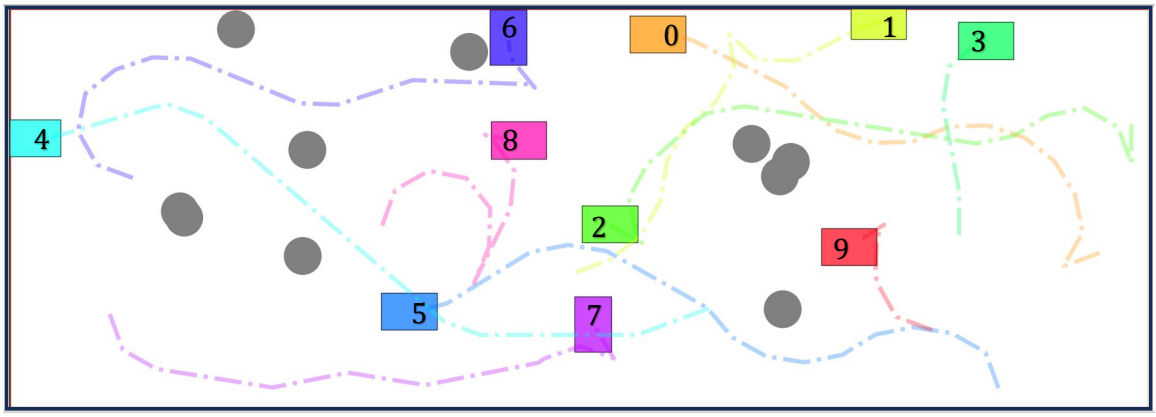


Figure 6.1: A simulated example on a 60×30 map with 10 obstacles and robots. The collision-free trajectories for the car-like robots are shown.

like Robots (PBCR)) adapts a decentralized strategy that leverages search-prioritization strategies from Priority Inheritance and Backtracking (PIBT) [63]. The second algorithm, *Enhanced Conflict-based search for Car-like Robots* (Enhanced Conflict Based Search for Car-like Robots (ECCR)), is a centralized method building on the principles of Enhanced Conflict-Based Search (ECBS) [60] and CL-CBS [85], the car-like robot extension of (basic) conflict-based search [53]. We further boost algorithms' success rates by introducing carefully designed, effective heuristics which also reduce the occurrence of deadlocks. Thorough simulation-based evaluations confirm that our methods deliver scalable SOTA performances on many key practical metrics. While our centralized methods tend to find shorter trajectories due to their access to global information, our decentralized method produces better scalability, yielding a higher success rate.

Related Work. Multi-Robot Path Planning (MRPP) has garnered extensive research interests in robotics and artificial intelligence in general. The graph-based MRPP variant, also known as Multi-Agent Path Finding (MAPF) [34], is to find collision-free paths for a set of robots within a given graph environment. Each robot possesses a distinct starting point, and a goal position, and the challenge lies in determining paths that ensure their traversal from start to goal without collisions. It has been proven many times over that optimally solving the graph-based problem to minimize objectives like makespan or cumulative

costs is NP-hard; see, e.g., [27, 3].

Computational methods for MRPP can be broadly classified into two categories: *centralized* and *decentralized*. Centralized solvers operate under the assumption that robot paths can be computed centrally and subsequently executed with minimal coordination errors. On the other hand, decentralized solvers capitalize on the autonomy of individual robots, enabling them to calculate paths independently while requiring coordination for effective decision-making. Centralized solutions often involve reducing MRPP to well-established problems [33, 27, 55], employing search algorithms to explore the joint solution space [53, 60, 52, 58, 65] or apply human-designed rules for coordination and collision-avoidance [56, 115]. Decentralized approaches [62, 63] leverage efficient heuristics to address conflicts locally, bolstering their success rates. Machine learning and reinforcement learning methods for MRPP have also started to emerge, with researchers putting forward data-driven strategies to directly learn decentralized policies for MRPP [68, 126]. Nevertheless, we note that the paths generated by graph-based MRPP algorithms cannot be directly applied to physical robots due to their disregard for robot kinematics.

As autonomous driving gains momentum, interest in path planning for multiple car-like vehicles is also steadily rising [84, 82]. This has led to the development of new methods, including adaptations of CBS to car-like robots (CL-CBS) [85], prioritized trajectory optimization [86], sampling-based techniques [87], and optimal control strategies [127]. Decentralized approaches have been introduced as well, such as B-ORCA [88] and ϵCCA [89], both stemming from adaptations of ORCA [18] designed for car-like robots. These methods offer faster computation compared to centralized methods. However, their reliance on local information means they cannot provide a guarantee that robots will successfully reach their destinations. Moreover, they are prone to deadlock issues and tend to achieve much lower success rates, particularly in densely populated environments.

Organization. The rest of the paper is organized as follows. Sec. section 6.2 covers the preliminaries, including the problem formulation. In Sec. section 6.3-section 6.4, we

demonstrate our algorithms in detail. In Sec. section 6.5, we conduct evaluations on the proposed methods in static settings and lifelong settings and discuss their implications. We conclude in Sec. section 6.6.

6.2 Problem Formulation

6.2.1 Multi-Robot Path Planning for Car-Like Robots

A static instance of this problem is specified as $(\mathcal{W}, \mathcal{S}, \mathcal{G})$, where $\mathcal{W}$ constitutes a map with dimensions $W \times H$, housing a set of obstacles $\mathcal{O} = \{o_1, ..., o_{n_o}\}$. $\mathcal{S} = \{s_1, ...s_n\}$ defines the initial configurations of n robots within the workspace, and $\mathcal{G} = \{g_1, ...g_n\}$ represents the corresponding goal configurations.

Each robot's $SE(2)$ configuration, denoted as $v_i = (x_i, y_i, \theta_i)$, comprises a 3-tuple of position coordinates and the yaw orientation θ_i. The car-like robot is modeled as a rectangular shape with length ℓ and width w. The subset of $\mathcal{W}$ occupied by a robot's body at a given state v is denoted by $\Gamma(v)$. The motion of the robot adheres to the Ackermann-steering kinematics (see Figure 6.2(a)):

$$\dot{x} = u\cos\theta, \quad \dot{y} = u\sin\theta, \quad \dot{\theta} = \frac{u}{\ell_b}\tan\phi, \tag{6.1}$$

in which $u \in [-u_m, u_m]$ is the linear velocity of the robot, $\phi \in [-\phi_m, \phi_m]$ is the steering angle. These are the control inputs. ℓ_b is the wheelbase length, i.e., the distance between the front and back wheels.

To make planning more tractable, we discretize into intervals Δt. The goal is to ascertain a feasible path for each robot i, expressed as a state sequence $P_i = \{p_i(0), ...p_i(t), ...p_i(T)\}$ that adheres to the following constraints: (i) $p_i(0) = s_i$ and $p_i(T) = g_i$; (ii) $\forall t \in [0, T], \forall i \neq j, \Gamma(p_i(t)) \bigcap \Gamma(p_j(t)) = \emptyset$; (iii) The path follows the Ackermann-steering kinematic model. The time interval is kept small during the discretization process. This choice ensures that the distance moved within each step remains smaller than the size of the robot. Conse-

quently, the need to account for swap conflicts [34] is eliminated.

Trajectory quality is evaluated using the following criteria:

- Makespan: $\max_{1 \leq i \leq n} len(P_i)/u_{\max}$;
- Flowtime: $\sum_{1 \leq i \leq n} len(P_i)/u_{\max}$

where $len(P_i)$ denotes the length of trajectory P_i.

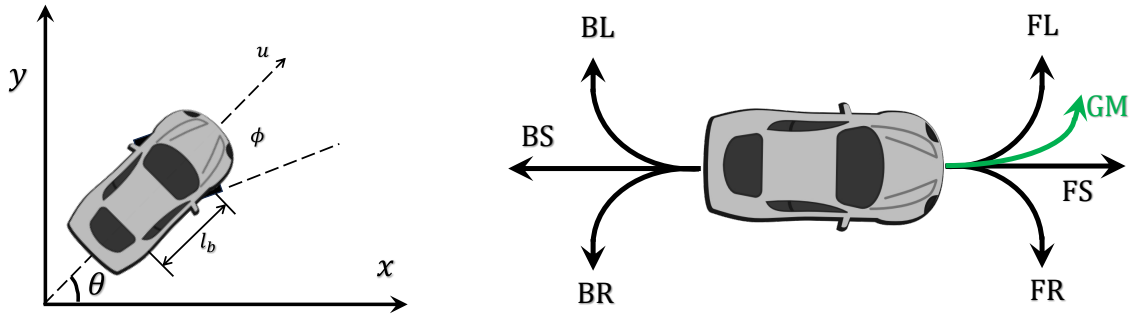
In a lifelong setting, we assume an infinite stream of tasks for each robot. Upon completing the current task, a robot immediately receives a new target. In this scenario, evaluating the system's efficiency often relies on throughput—the number of tasks accomplished (or goal states arrived) within a specified number of timesteps.

6.3 Priority-Inherited Backtracking for Car-Like Robots

In this section, we propose a *decentralized* method called PBCR for car-like robots, adapting the prioritization mechanism of the decentralized MRPP algorithm Priority Inheritance with Backtracking (PIBT) [63]. In conjunction with the introduction of PBCR, we also describe the general methodology we adopt to plan continuous trajectories for non-holonomic robots that is generally applicable, provided that the optimal trajectories connecting the robot can be compactly represented using a few motion primitives.

To render planning for the continuous system feasible, we constrain the robot's possible state transitions within the discretized time interval Δt, forming a set of *motion primitives*. Our motion primitives $\mathcal{M}$ are defined similarly as done in [85, 86], which contain a total of seven actions: forward max-left (FL), forward straight (FS), forward max-right (FR), backward max-left (BL), backward straight (BS), backward max-right (BR), and wait, as shown in Figure 6.2 (b). For the first six motion primitives, we assume that the robot maintains a constant velocity of u_m throughout a single time interval Δt. When the robot turns, we assume it is pivoting using the maximum steering angle ϕ_m and then tracing an arc with a turning radius r_m. This arc has a $u_m \Delta t$ length.

To ensure a robot can reach its goal state for PBCR, one additional *greedy motion primitive* (GM) is added to $\mathcal{M}$. This greedy motion primitive is derived by truncating the first segment of length $u_m \Delta t$ from the shortest path connecting the current state to the goal state for a single robot. In the absence of obstacles on the map, this shortest path corresponds to the (optimal) Reeds-Shepp [128] path while if obstacles are present, the path can be determined using the vanilla (non-spatio-temporal) hybrid A* algorithm. It's important to recognize that this greedy motion primitive could be identical to other motion primitives, resulting in the same subsequent state as those alternatives. In such cases, we opt to retain solely the greedy one.



(a) (b)

Figure 6.2: (a) Ackermann steering kinematic model. (b) The predefined motion primitives. We have at most 8 motion primitives, which are forward max-left (FL), forward straight (FS), forward max- right (FR), backward max-left (BL), backward straight (BS), backward max-right (BR), wait, and greedy motion primitive (GM)

6.3.1 Algorithm Skeleton

PBCR is outlined in algorithm 13-algorithm 14, at the core of which lies the `PIBTLoop`, which orchestrates the decision-making process for the robots at each time step t. The primary objective of `PIBTLoop` is to determine each robot's next motion and succeeding state without encountering collisions. `PIBTLoop` initializes two essential sets: UNDECIDED and OCCUPIED. The UNDECIDED set keeps track of robots that have yet to finalize their next actions, while the OCCUPIED set stores the information about occupied states, preventing multiple robots from attempting to occupy the same space simultaneously. At

the outset of each iteration, the algorithm updates the priorities of all robots based on relevant factors. The priorities are crucial in determining which robot takes precedence in decision-making. This step ensures that the higher-priority robots have their actions determined earlier, facilitating swift and efficient navigation. PBCR's decision-making process revolves around an iterative approach. The algorithm enters a loop that continues until all robots in the UNDECIDED set have their actions determined. The robot with the highest priority within the loop is selected from the UNDECIDED set. This robot is designated as robot i for the current iteration.

Algorithm 13: Priority selection at time step t

```
1 Function PIBTLOOP():
2     UNDECIDED ← R(t)
3     OCCUPIED ← ∅
4     update all priorities
5     while UNDECIDED! = ∅ do
6         i ← the robot with the highest priority in UNDECIDED
7         PIBT(i, NONE, NONE)
```

The `PIBT` function is invoked for the selected robot i, along with placeholders for two parameters: the parent robot j which robot i is inherited from, and the potential succeeding state v_j of the parent robot. This function serves as the primary interface for the robot's decision-making. It evaluates potential actions considering the robot's current state, goals, obstacles, and the presence of other robots. Based on these evaluations, the robot's next action is determined to ensure collision-free and goal-oriented navigation.

The `PIBT` function plays a pivotal role within PBCR, encapsulating the decision-making process for an individual robot. This function is called iteratively for each robot to compute its next action, considering its current state, goals, and interactions with neighboring robots. The function identifies the set UNDECIDED, which comprises all robots yet to finalize their actions, and marks robot i as decided temporarily. Additionally, NbrUndecided contains neighboring robots of i that remain undecided, and NbrOcc holds the occupied states of the neighboring robots that have already determined their actions. These sets pro-

Algorithm 14: PIBT function

```
1  Function PIBT(i, j, v_j):
2      UNDECIDED ← R(t) − i
3      NbrUndecided ← the neighboring robots of i that remain undecided
4      NbrOcc ← the occupied states of the neighboring decided robots
5      C ← {ValidSuccState(p_i(t), m_i) | m_i ∈ M_i}
6      while C ≠ ∅ do
7          v_i ← argmax_{v∈C} Q_i(v) and remove v_i from C
8          if FindCollision(v_i, NbrOcc ⋃ {p_j(t), v_j}) then
9              continue
10         OCCUPIED.add(v_i)
11         if ∃k ∈ NbrUndecided such that FindCollision(p_k(t), v_i)= true then
12             if PIBT(k, i, v_i) is valid then
13                 p_i(t + 1) ← v_i
14                 return valid
15             else
16                 OCCUPIED.remove(v_i)
17     UNDECIDED.add(i)
18     return invalid
```

vide crucial contextual information to facilitate informed decision-making. The function computes a set C of potential future states v_i for the current robot i. These states are generated based on all feasible actions m_i available to robot i at its current state $p_i(t)$. The goal is to explore various potential actions to lead to a successful next state for r_i. The function selects the potential state v_i within a loop that maximizes the robot's utility function $Q_i(v)$ from the set C. This action selection aims to optimize the robot's decision by choosing the most promising action according to the utility criterion. However, before finalizing the action, the function checks for collisions between the selected v_i and the occupied states of neighboring robots, as well as the state of the parent robot j (if it is not NONE) and its potential succeeding state v_j. If a collision is detected, the function continues to the next iteration of the loop, considering alternative potential actions.

If no collisions are found for the chosen v_i, v_i is added to the OCCUPIED set, indicating the robot intends to occupy this state. The function then checks if an undecided neighboring

robot k will collide with v_i. If such a robot k is found, the function recursively calls `PIBT` for robot k with the parent robot i and the tentative state v_i as the parameters. If the resulting action is valid, robot i updates its next state $p_i(t+1)$ to v_i, and the function returns "valid." If `PIBT` for robot k fails, the state v_i is removed from OCCUPIED. After evaluating all potential actions and considering interactions with neighboring robots, robot i remains in the UNDECIDED set. The function returns "invalid" if a valid action cannot be determined. Otherwise, it returns "valid" along with the updated next state $p_i(t+1)$ for robot i.

We note that PBCR's one-step planning and execution nature makes it applicable to static and lifelong scenarios.

6.3.2 Performance-Boosting Heuristics

Multiple heuristics are introduced to boost the performance of PBCR, while some are basic, others are involved.

Distance heuristic. We use the max of the holonomic cost with obstacles, the length of the shortest Reeds-Shepp path between two states, and 2D Euclidean distance as our distance heuristic function (`DistH`) similar to [129, 85]. This distance heuristic is admissible.

Priority heuristic. PBCR constitutes a single-step, priority-driven planning approach that necessitates the ongoing adjustment of each robot's priority at each time step. Initially, the priority assigned to each robot is determined based on the number of time steps that have transpired since its preceding task was updated. The robot with more time since its previous goal update receives a higher priority ranking. In static scenarios, the elapsed time is reset to zero whenever a robot reaches its designated goal state. This reset mechanism prevents robots that have achieved their goal state from obstructing the progress of other robots. When multiple robots share the same elapsed time, prioritization is resolved by favoring those robots with a larger heuristic distance value to their respective goal states. This heuristic principle finds widespread application in the realm of prioritized planning [130] to improve the success rate.

The Q-function. The Q-function is a critical tool for assessing the optimal choice of a motion primitive in guiding the robot from its current state toward its goal state. PIBT algorithm employs the single-agent shortest path length from the current vertex to its goal vertex, neglecting inter-agent collisions, as the basis for its evaluation function. This approach is feasible for discrete 2D graphs since these shortest path lengths can be pre-calculated and stored in a table. Nevertheless, this approach is impractical for the state space we're addressing with car-like robots. Firstly, the state space is infinite. Secondly, determining a single-robot shortest trajectory from a state to a goal state using hybrid A* for evaluating all the motion primitives is both time-intensive and incomplete. For these reasons, we consider employing the distance heuristic function discussed earlier as well as the action cost, denoted as

$$Q'_i(v) = -\texttt{DistH}(v, g_i) - \lambda \texttt{Cost}(p_i(t), v) \tag{6.2}$$

where λ is a weight and $\texttt{Cost}(p_i(t), v)$ returns the action cost from state $p_i(t)$ to v. We use the action cost similar to [85] where backward motions, turning, and changes of directions will receive additional penalty cost. However, it's important to note that the distance heuristic is not a "reachable" heuristic in the context of PBCR. A heuristic is labeled as "reachable" for $PIBT$-like algorithms if, while ignoring inter-robot collisions, a solitary robot is guaranteed reach its goal state by consistently choosing the "best" action with the maximum Q value at each timestep. In discrete MRPP, the single-agent shortest path length qualifies as a "reachable" heuristic. We mention that while Manhattan distance is a reliable "reachable" heuristic when the map is obstacle-free, this is not guaranteed when obstacles are present. In scenarios with obstacles, a robot directed by the Manhattan distance heuristic might become entrapped in local minima, obstructing its path to the goal state. Our case showed `DistH` is not a "reachable" heuristic either. Despite being admissible, it lacks the required precision. Notably, the assertion that a greedy motion primitive should yield the highest Q value among the possibilities isn't consistently upheld when

using $Q_i(v) = -\texttt{DistH}(v, g_i)$. This discrepancy easily leads the robot into a state of stagnation. Furthermore, we found that this greedy heuristic could also lead to deadlocks. An example can be seen in Figure 6.3.

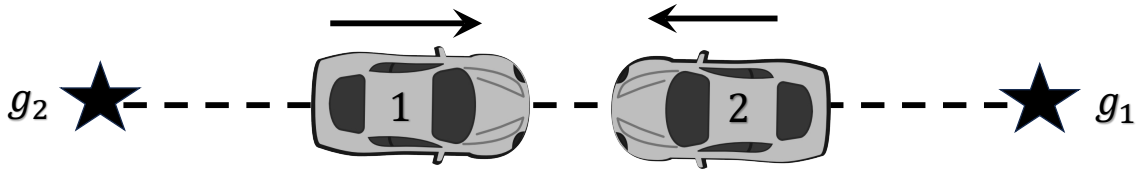


Figure 6.3: An illustrative case highlighting the potential occurrence of deadlocks due to the exclusion of the count-based heuristic is as follows: In this scenario, two robots, labeled as robot 1 and robot 2, travel in opposite directions along a linear path. When robot 2 yields to robot 1 in PBCR, a predicament arises. In this situation, the motion primitives FS, FR, and FL lead to collisions, compelling robot 2 to consistently opt for the motion primitive BS. This choice is driven by BL and BR, involving additional turning penalties. A dynamic shift occurs in priorities upon robot 1's successful arrival at its designated goal state. Robot 1 is assigned a lower priority than robot 2 and consequently yields to the latter. Similarly, robot 1 consistently selects BS using the greedy strategy. Consequently, an unending cycle emerges, entangling robot 1 and robot 2 in an indefinite sequence of movements

We present a novel count-based exploration heuristic to address the challenge, inspired by [131]. This heuristic is specifically designed to surmount the issue of local minima arising from inaccuracies in distance heuristics and deadlocks and to foster exploration of uncharted territories. To achieve this, we incorporate supplementary penalties for states that have been visited, aiming to encourage robots to break free from local minima and venture into unexplored regions. Each state $v = (x, y, \theta)$ is mapped to its nearest discrete state $\widetilde{v} = (\lfloor \frac{x}{\delta x} \rfloor, \lfloor \frac{y}{\delta y} \rfloor, \lfloor \frac{\theta}{\delta \theta} \rfloor)$. This mapping facilitates the tallying of occurrences using a hash table $\mathcal{H}_i$ for each robot, where δx, δy, and $\delta \theta$ delineate the state-space resolution.

When a robot i traverses state v at a given timestep t, the count $\mathcal{H}_i(\widetilde{v})$ is incremented by one. In addition, we introduce bonus rewards to incentivize the selection of the greedy motion primitive. The resultant Q-function is:

$$Q_i(v) = Q'_i(v) + \alpha Gr(v) - \beta \mathcal{H}_i(\widetilde{v}) \tag{6.3}$$

Here, α and β represent positive weight parameters. If the state v results from a greedy motion primitive, $Gr(v) = 1$; otherwise, $Gr(v) = 0$. To accommodate the requirement that robots should remain at their goal states upon arrival, we refrain from applying penalties to states near the goal states. As a result, when $v = g_i$ (the goal state of robot i), we set β to zero. We mention that such a Q function could be "reachable" if we properly choose the weight parameters. Moreover, by implementing the count-based heuristic, which compels robots to explore distinct motion patterns for collision avoidance, the deadlock issues can be nicely achieved.

6.4 Centralized ECCR

We enhance the Conflict Based Search for Car-like Robots (CL-CBS) algorithm by substituting the conventional low-level spatiotemporal hybrid state A* planner with focal hybrid state A* search [60]. This refined approach is named ECCR. In comparison to CL-CBS, the ECCR method guides the low-level planner to discover trajectories within a bounded suboptimality ratio while encountering fewer potential conflicts with other robots. This reduction in conflicts subsequently results in a significantly lower number of high-level expansions.

For the purpose of adapting ECCR to lifelong scenarios, characterized by the necessity for frequent online replanning, we incorporate the windowed version of focal search in the low-level planning process [132, 133]. Specifically, during path planning, we only address conflicts that arise within a window of ω steps. This adjustment effectively decreases the runtime of the ECCR algorithm during the planning phase. Replanning occurs at intervals of every ω steps and also when robots reach their current goals and receive new tasks.

6.5 Evaluation

In this section, we evaluate the proposed algorithms in static scenarios and lifelong scenarios. All methods are implemented in C++. The source code can be found in https:

//github.com/GreatenAnoymous/CarLikePlanning. All experiments are performed on an Intel® Core™ i7-6900K CPU at 3.0GHz in Ubuntu 18.04LTS. In the simulation, the following parameters are used: $w = 2$, $l = 3$, $l_b = 2$, $\delta x = \delta y = 2$, $\delta\theta = 40.1°$, $u_m = 2$, $\phi_m = 40.1°$, $r_m = 3$, $\Delta t = r_m \delta\theta / u_m$.

6.5.1 Static Scenarios

In this section, we assess the performance of the algorithms on a grid of dimensions 100×100, both with and without obstacles. For the scenarios involving obstacles, we create 50 circular obstacles with a radius of 1 unit length and place them randomly on the map. For each value of n, we generate 50 distinct instances, ensuring that the states of the robot do not overlap with each other and with the obstacles in start and goal configurations. A time limit of 60 seconds is imposed on each instance. The success rate is determined by tallying the instances each algorithm successfully solves within the time limit. Additionally, we evaluate the average runtime (in seconds), makespan, and flowtime across the solved instances. Our evaluation includes a comparison with two reference algorithms: firstly, the prior centralized algorithm known as CL-CBS, and secondly, the decentralized SHA* algorithm as described in [85]. We emphasize that all algorithms use identical predefined motion primitives, except that GM is exclusively used for PBCR.

A suboptimality ratio of 1.5 is selected for ECCR. Three variants of PBCR are evaluated. The first, PBCR(v0), employs the Q-function described in (Equation 6.3) without incorporating the count-based exploration heuristic. Both PBCR(v1) and PBCR(v2) leverage the count-based exploration heuristic to resolve local minima and deadlocks. In the case of PBCR(v1), we opt to clear the hash table $\mathcal{H}_i$ and reset the associated counts to zero whenever robot i reaches its designated goal state. Conversely, for PBCR(v2), the visiting history is not cleared. For all variants of PIBT, the maximum time step is constrained to 500. And we set $\lambda = 0.3$, $\alpha = \beta = \lambda \texttt{Cost}(p_i(t), v)$.

Detailed evaluation results are presented in Figure 6.4 through Figure 6.6. Compared

to CL-CBS, ECCR showcases significantly enhanced scalability and success rates due to its capability to expand a notably smaller number of high-level nodes. Notably, the solution quality of ECCR closely approaches that of CL-CBS. On the flip side, the PBCR variants excel in terms of runtime efficiency, resulting from their decentralized nature and one-step planning strategy. They can resolve instances involving 60 robots in as little as 4 seconds. Among these variants, PBCR featuring the count-based exploration heuristic achieves an elevated success rate. Conversely, without the count-based exploration heuristic, PBCR (v0) falters in instances with obstacles, revealing its limitations. However, PBCR(v2) successfully tackles 93% of cases involving 60 robots within 4 seconds, excelling in challenging scenarios—a substantial improvement over other variants and the previous decentralized SHA* approach.

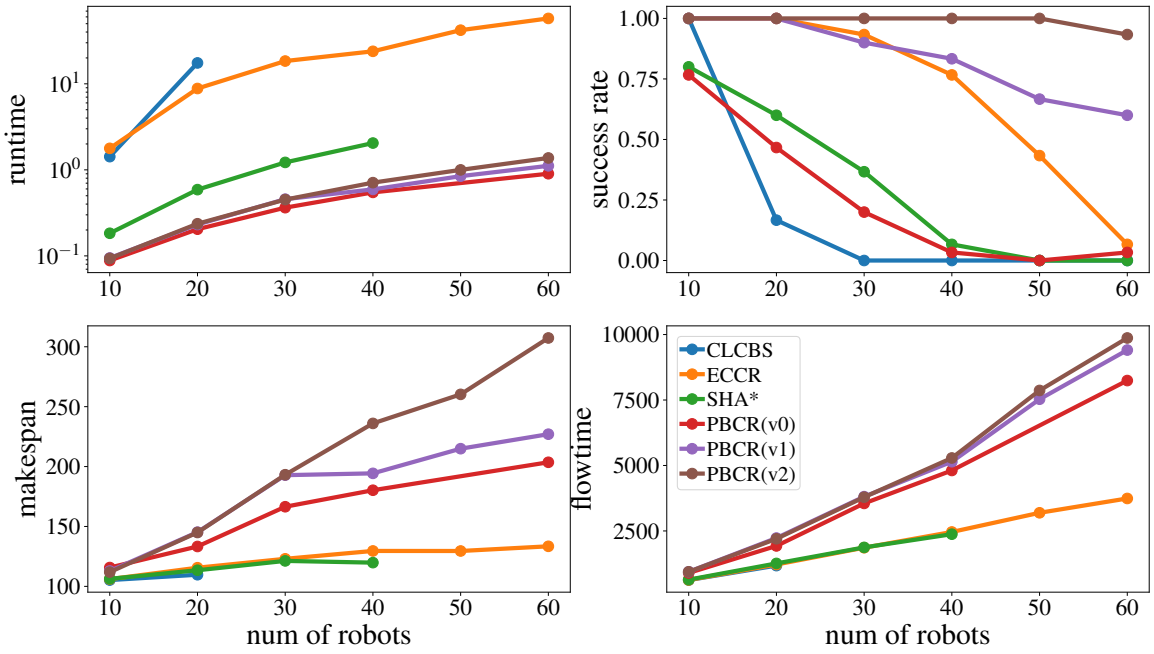


Figure 6.4: Evaluation results on a 100×100 empty map for a varying number of robots.

It's crucial to emphasize that all PBCR variants do not fail due to time constraints but rather due to surpassing the maximum timestep. For those failed instances, PBCR with count-based heuristic still can guide more than 90% of the robots to their goal states as shown in Figure 6.6. The contrast in success rates between PBCR(v2) and PBCR(v1)

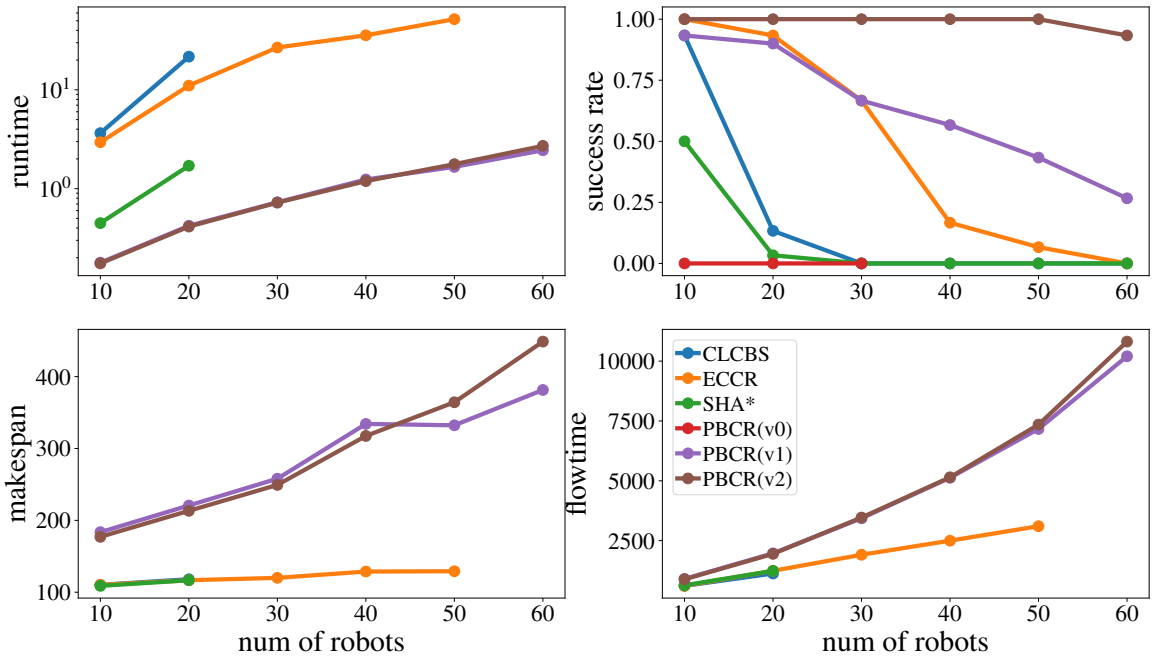


Figure 6.5: Evaluation data on a 100×100 map with 50 randomly placed obstacles for a varying number of robots.

implies that retaining the visiting history until completion, instead of resetting it upon each robot's arrival at its goal state, is more effective. This arises from the fact that in the context of PBCR, when a robot reaches its designated goal, it often has to temporarily vacate its goal position to accommodate other robots that must traverse it. This frequent shifting in and out of the goal state can potentially lead to repetitive cycles and hinder progress. Maintaining the visiting history mitigates this issue by preventing the robot from becoming trapped in an endless loop of entering and leaving the goal, consequently boosting the overall success rate. One drawback of PBCR lies in its trajectory quality. This stems from the absence of global information and the inherent nature of one-step planning, leading to longer planned trajectories compared to centralized algorithms such as ECCR and CL-CBS. This effect is particularly pronounced when dealing with a large number of robots.

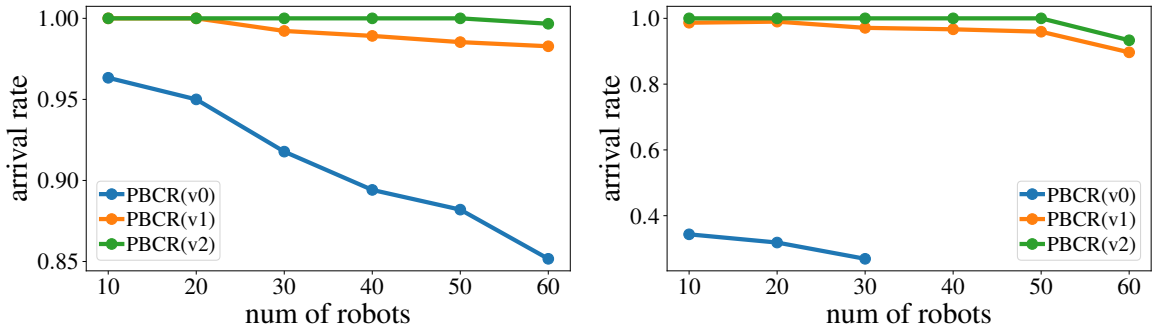


Figure 6.6: The average percentage of robots arrived at their goal state for each PBCR variant on the 100×100 maps for varying numbers of robots. (a) without obstacles (b) with obstacles.

6.5.2 Lifelong Scenarios

In this section, we subject both ECCR and PBCR to testing within lifelong settings. We adopt a 50×50 map configuration, both with and without obstacles. For scenarios involving obstacles, a set of 10 obstacles is distributed randomly. For each value of n, we randomly generate 20 unique instances. In each instance, both the initial configurations and 4000 goal states, randomly generated, are allocated to each robot. Throughout the simulation, we assume a lack of a priori knowledge on the part of the robots regarding their subsequent tasks. In each instance, we define a maximum of 5000 simulated steps and set a runtime limit of 600 seconds. Each robot has many tasks that cannot be feasibly completed within the designated maximum steps. Regarding ECCR, a suboptimality ratio of 1.5 is established, while a window size ω of 5 is employed. Given that new goals are allocated to robots upon completing current tasks, the visiting history is consistently cleared. This decision is driven by the fact that the visiting experience only relates to the robot's preceding task. PBCR(v0) is excluded from consideration owing to its poor performance in addressing issues related to deadlocks and stagnation. The outcome of our evaluations is depicted in Figure 6.7. PBCR achieves significantly lower execution times than ECCR, rendering it suitable for handling large-scale scenarios and real-time planning. Nonetheless, it accomplishes a smaller number of tasks.

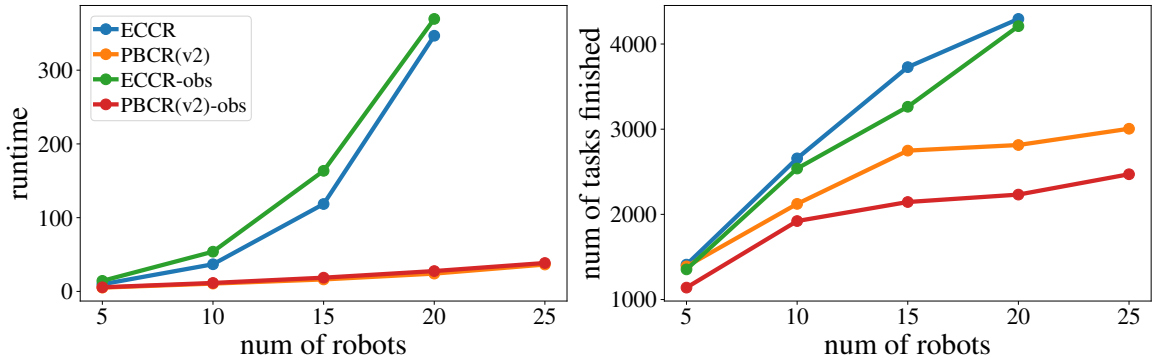


Figure 6.7: Runtime and the average number of tasks finished within given timesteps in lifelong scenarios for varying number of robots. The term "obs" is an abbreviation for scenarios with obstacles.

6.5.3 Real-Robot Experiments

We employed the portable multi-robot platform, microMVP [134], for conducting real-world experiments involving car-like robots to validate the execution of our algorithmic solutions on real hardware. Whereas the microMVP robots are differential drive robots, a software layer can be imposed to simulate car-like robots, which is what we did [1]. Figure 6.8 gives a visual representation of the setup. As evident from the attached video, paths generated by our planners can be successfully executed on the robots' controllers.

6.6 Conclusion and Discussions

In this study, we investigate path planning for multiple car-like robots. We present two distinct algorithms tailored to the specific demands of car-like robot navigation scenarios. The first and our main contribution, PBCR, employs an effective count-based exploration heuristic. Focusing on decentralized decision-making, PBCR demonstrates a notable advancement over previous decentralized approaches. The improvement is evident through significantly heightened success rates with some manageable optimality trade-offs of yielding longer trajectories. Opportunities for improving trajectory quality within PBCR remain;

[1] While achieving the same goal, the simulation makes the experiment more challenging than running directly on car-like robots.

Figure 6.8: Snapshot of a real robot experiment with 4 obstacles and 5 robots.

in particular, our approach employs a manually designed Q-function for action selection, which may be replaced with data-driven approaches for reaching optimal performance.

CHAPTER 7
CONCLUSION AND FUTURE WORK

MRPP is a fundamental research topic in robotics and artificial intelligence. It has been actively studied for a few decades due to its hardness and importance in applications, and will be continuously studied in the near future.

7.0.1 Contributions and Influences

In this dissertation, we have focused on multiple MRPP variations and proposed theoretically strong algorithms, and high performance heuristics to push the current MRPP algorithm design to new levels from a few directions. Furthermore, we investigated different potential applications of MRPP on well-formed infrastructures, high-density autonomous parking and retrieval systems as well as path planning for multiple car-like robots.

In chapter 3, we present a polynomial-time algorithm for MRPP on grids, achieving improved constant-factor optimality guarantees over existing methods. By integrating the Grid Rearrangement Algorithm with our parallel odd-even sorting approach, we attain a makespan of $4(m_1 + 2m_2)$ for fully occupied grids, significantly outperforming prior work. For grids with robot densities of $1/2$ and $1/3$, our highway and line-merge motion primitives yield an asymptotic makespan optimality ratio of $1 + \frac{m_2}{m_1+m_2}$ under uniform random start and goal distributions. This demonstrates that limiting robot density enables substantial performance gains. We also extend the 2D grid algorithms to k-dimensional grids and derive corresponding theoretical guarantees and optimality bounds. Additionally, we introduce a path refinement strategy using MCP, further enhancing solution quality. The work is pulished in [116, 83, 135].

In chapter 4, we systematically study well-formed infrastructures by formulating and analyzing the maximal and largest well-connected vertex set problem. Such layouts are

prevalent in real-world applications, including autonomous warehouses and conventional parking lots. Maximizing the well-connected vertex set size is crucial for optimizing space utilization. We first prove the intractability of finding the maximum well-connected vertex set by reducing the problem to 3SAT. To address this challenge, we propose efficient algorithms, including a greedy approach for computing maximal well-connected sets and a search-based algorithm for identifying the largest set. These methods facilitate more effective infrastructure design for applications such as warehouse automation. Furthermore, we highlight the relationship between well-connected sets and well-formed MRPP. This work is published in [136].

In chapter 5, we study the problem of automated vehicle parking and retrieval using multiple robotic valets, proposing a complete automated garage design that supports near 100% parking density and develops efficient algorithms for its operation. We introduce the BVPR and CVPR problems, which model key operations in high-density automated garages. These formulations establish a foundation for future theoretical and algorithmic research in this domain. We design a system that supports a parking density as high as $(m_1-2)(m_2-2)/m_1m_2$ on an $m_1 \times m_2$ grid, enabling multi-vehicle parking and retrieval. This density approaches 100% for large-scale garages. Leveraging the structured, grid-like nature of the parking environment, we propose an optimal ILP-based method and a highly scalable, fast suboptimal algorithm based on sequential planning. The suboptimal method maintains strong solution quality while being suitable for large-scale applications. We introduce a shuffling mechanism to rearrange vehicles during off-peak hours, improving retrieval efficiency during peak demand. Our algorithm executes these rearrangements in $O(m_1m_2)$ time at near-full garage density, significantly reducing retrieval delays. to address the growing need for space efficiency in urban environments, we introduced a puzzle-based automated parking and retrieval system capable of operating at near 100% robot density, along with specialized MRPP-based algorithms that ensure efficient vehicle storage and retrieval. This work is published in [112].

Finally, in chapter 6, we extended our research to continuous domains with non-trivial robot kinematics, specifically focusing on Ackerman-steering car-like robots. This work addresses the challenges of motion planning for non-holonomic robots, specifically Ackerman car-like robots, in continuous domains. The key contributions are as follows: We propose two novel algorithms designed to solve both static and lifelong path planning tasks for car-like robots. The first algorithm, PBCR, adapts a decentralized strategy leveraging PIBT. The second algorithm, ECCR, extends Conflict-Based Search (CBS) to car-like robots, offering a centralized solution. Our methods incorporate effective heuristics to enhance performance, reduce deadlocks, and improve success rates. Through thorough simulation-based evaluations, we demonstrate that our decentralized approach provides higher scalability and better success rates, while the centralized approach generates shorter trajectories by leveraging global information. These algorithms represent a significant step toward practical, scalable solutions for multi-robot systems in dynamic, real-world environments, especially in applications like warehouse automation and collaborative exploration.. This work is published in [137].

7.0.2 Limitations and Future Works

First, from a theoretical and algorithmic perspective on grid-based MRPP, our GRA-based algorithms face limitations, particularly in grids with randomly distributed obstacles. For such challenging scenarios, further development of novel heuristics and algorithms is required to solve them in polynomial time while maintaining optimal makespan guarantees.

Regarding the autonomous parking and retrieval problem, we plan to implement more advanced simulations that account for vehicle shapes and the movements of robots carrying the vehicles. Additionally, our current methods do not consider obstacles within the environment, and we aim to extend our solutions to scenarios involving obstacles in the parking and retrieval system.

In our study of well-connected sets and well-formed infrastructures, we assume that the

environment is represented as a graph, with each robot occupying a vertex. However, this assumption is not always valid, particularly in parking lot scenarios. Therefore, extending our algorithms for finding LWCS to account for robot shapes is one of our future research directions.

Lastly, while we have introduced PBCR, which outperforms previous decentralized algorithms in terms of success rate, our experiments indicate that there is still room for improvement in solution quality. Currently, we use a rule-based heuristic to reduce the likelihood of deadlock, but an adaptive, deep-learning-based method is a promising direction for future research.

REFERENCES

[1] J. E. Hopcroft, J. T. Schwartz, and M. Sharir, "On the complexity of motion planning for multiple independent objects; pspace-hardness of the" warehouseman's problem"," *The International Journal of Robotics Research*, vol. 3, no. 4, pp. 76–88, 1984.

[2] K. Solovey and D. Halperin, "On the hardness of unlabeled multi-robot motion planning," *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1750–1759, 2016.

[3] J. Yu and S. M. LaValle, "Structure and intractability of optimal multi-robot path planning on graphs," in *Twenty-Seventh AAAI Conference on Artificial Intelligence*, 2013.

[4] D. Halperin, J.-C. Latombe, and R. H. Wilson, "A general framework for assembly planning: The motion space approach," in *Proceedings of the fourteenth annual symposium on Computational geometry*, 1998, pp. 9–18.

[5] S. Rodriguez and N. M. Amato, "Behavior-based evacuation planning," in *2010 IEEE International Conference on Robotics and Automation*, IEEE, 2010, pp. 350–355.

[6] B. S. Smith, M. Egerstedt, and A. Howard, "Automatic generation of persistent formations for multi-agent networks under range constraints," *Mobile Networks and Applications*, vol. 14, pp. 322–335, 2009.

[7] J. A. Preiss, W. Hönig, G. S. Sukhatme, and N. Ayanian, "Crazyswarm: A large nano-quadcopter swarm," in *IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2017.

[8] D. Fox, W. Burgard, H. Kruppa, and S. Thrun, "A probabilistic approach to collaborative multi-robot localization," *Autonomous robots*, vol. 8, pp. 325–344, 2000.

[9] E. Griffith and S. Akella, "Coordinating multiple droplets in planar array digital microfluidic systems," *International Journal of Robotics Research*, vol. 24, no. 11, pp. 933–949, 2005.

[10] D. Rus, B. Donald, and J. Jennings, "Moving furniture with teams of autonomous robots," in *Proceedings IEEE/RSJ International Conference on Intelligent Robots & Systems*, 1995, pp. 235–242.

[11] R. Knepper and D. Rus, "Pedestrian-inspired sampling-based multi-robot collision avoidance," in *2012 IEEE RO-MAN: The 21st IEEE International Symposium*

on Robot and Human Interactive Communication, Proceedings of the IEEE, 2012, pp. 94–100.

[12] J. Jennings, G. Whelan, and W. Evans, "Cooperative search and rescue with a team of mobile robots," in *Proceedings IEEE International Conference on Robotics & Automation*, 1997.

[13] S. W. Feng, T. Guo, and J. Yu, "Optimal allocation of many robot guards for sweep-line coverage," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2023, pp. 3614–3620.

[14] P. R. Wurman, R. D'Andrea, and M. Mountz, "Coordinating hundreds of cooperative, autonomous vehicles in warehouses," *AI magazine*, vol. 29, no. 1, pp. 9–9, 2008.

[15] M. A. Erdmann and T. Lozano-Pérez, "On multiple moving objects," *Algorithmica*, vol. 2, no. 4, pp. 477–521, 1987.

[16] S. M. LaValle and S. A. Hutchinson, "Optimal motion planning for multiple robots having independent goals," *IEEE Transactions on Robotics and Automation*, vol. 14, no. 6, pp. 912–925, 1998.

[17] Y. Guo and L. Parker, "A distributed and optimal motion planning approach for multiple mobile robots," in *Proceedings IEEE International Conference on Robotics & Automation*, 2002, pp. 2612–2619.

[18] J. Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proceedings IEEE International Conference on Robotics & Automation*, 2008, pp. 1928–1935.

[19] T. Standley and R. Korf, "Complete algorithms for cooperative pathfinding problems," in *Proceedings International Joint Conference on Artificial Intelligence*, 2011, pp. 668–673.

[20] K. Solovey and D. Halperin, "K-color multi-robot motion planning," in *Proceedings Workshop on Algorithmic Foundations of Robotics*, 2012.

[21] M. Turpin, K. Mohta, N. Michael, and V. Kumar, "Capt: Concurrent assignment and planning of trajectories for multiple robots," *International Journal of Robotics Research*, vol. 33, no. 1, pp. 98–112, 2014.

[22] K. Solovey, J. Yu, O. Zamir, and D. Halperin, "Motion planning for unlabeled discs with optimality guarantees," *Robotics: Science and Systems*, 2015.

[23] G. Wagner and H. Choset, "Subdimensional expansion for multirobot path planning," *Artificial intelligence*, vol. 219, pp. 1–24, 2015.

[24] B. Araki, J. Strang, S. Pohorecky, C. Qiu, T. Naegeli, and D. Rus, "Multi-robot path planning for a swarm of robots that can both fly and drive," in *2017 IEEE International Conference on Robotics and Automation (ICRA*, IEEE, 2017, pp. 5575–5582.

[25] K. Vedder and J. Biswas, "X*: Anytime multiagent planning with bounded search," in *Proceedings of the 18th International Conference on Autonomous Agents and Mul- tiAgent Systems, International Foundation for Autonomous Agents and Multiagent Systems*, 2019, pp. 2247–2249.

[26] A. Khan, V. Kumar, and A. Ribeiro, "Large scale distributed collaborative unlabeled motion planning with graph policy gradients," *IEEE Robotics and Automation Letters*, 2021.

[27] P. Surynek, "An optimization variant of multi-robot path planning is intractable," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, 2010.

[28] E. D. Demaine, S. P. Fekete, P. Keldenich, H. Meijer, and C. Scheffer, "Coordinated motion planning: Reconfiguring a swarm of labeled robots with bounded stretch," *SIAM Journal on Computing*, vol. 48, no. 6, pp. 1727–1762, 2019.

[29] J. Yu, "Constant factor time optimal multi-robot routing on high-dimensional grids," in *2018 Robotics: Science and Systems*, 2018.

[30] M. Szegedy and J. Yu, "On rearrangement of items stored in stacks," in *The 14th International Workshop on the Algorithmic Foundations of Robotics*, 2020.

[31] H. Ma, C. Tovey, G. Sharon, T. Kumar, and S. Koenig, "Multi-agent path finding with payload transfers and the package-exchange robot-routing problem," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, 2016.

[32] M. Erdmann and T. Lozano-Perez, "On multiple moving objects," *Algorithmica*, vol. 2, no. 1, pp. 477–521, 1987.

[33] J. Yu and S. M. LaValle, "Optimal multirobot path planning on graphs: Complete algorithms and effective heuristics," *IEEE Transactions on Robotics*, vol. 32, no. 5, pp. 1163–1177, 2016.

[34] R. Stern *et al.*, "Multi-agent pathfinding: Definitions, variants, and benchmarks," in *Twelfth Annual Symposium on Combinatorial Search*, 2019.

[35] R. Mason, "Developing a profitable online grocery logistics business: Exploring innovations in ordering, fulfilment, and distribution at ocado," in *Contemporary Operations and Logistics*, Springer, 2019, pp. 365–383.

[36] Q. Wan, C. Gu, S. Sun, M. Chen, H. Huang, and X. Jia, "Lifelong multi-agent path finding in a dynamic environment," in *2018 15th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, IEEE, 2018, pp. 875–882.

[37] S.-H. Chan *et al.*, "The league of robot runners competition: Goals, designs, and implementation," in *ICAPS 2024 System's Demonstration track*, 2024.

[38] A. Dekhne, G. Hastings, J. Murnane, and F. Neuhaus, "Automation in logistics: Big opportunity, bigger uncertainty," in *McKinsey Q*, 2019, pp. 1–12.

[39] LogisticsIQ, *Warehouse automation market with post-pandemic (covid-19) impact by technology, by industry, by geography - forecast to 2026*, https://www.researchandmarkets.com/r/s6basv, Accessed: 2022-01-05, 2020.

[40] S. W. Feng, T. Guo, K. E. Bekris, and J. Yu, "Team rubot's experiences and lessons from the ariac," *Robotics and computer-integrated manufacturing*, vol. 70, p. 102 126, 2021.

[41] B. Huang, T. Guo, A. Boularias, and J. Yu, "Interleaving monte carlo tree search and self-supervised learning for object retrieval in clutter," in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 625–632.

[42] D. Kornhauser, G. Miller, and P. Spirakis, "Coordinating pebble motion on graphs, the diameter of permutation groups, and applications," in *Proceedings IEEE Symposium on Foundations of Computer Science*, 1984, pp. 241–250.

[43] A. Botea, D. Bonusi, and P. Surynek, "Solving multi-agent path finding on strongly biconnected digraphs," *Journal of Artificial Intelligence Research*, vol. 62, pp. 273–314, 2018.

[44] O. Goldreich, "Finding the shortest move-sequence in the graph-generalized 15-puzzle is np-hard," in *Studies in complexity and cryptography. Miscellanea on the interplay between randomness and computation*, Springer, 2011, pp. 1–5.

[45] J. Yu, "Intractability of optimal multirobot path planning on planar graphs," *IEEE Robotics and Automation Letters*, vol. 1, no. 1, pp. 33–40, 2015.

[46] J. Banfi, N. Basilico, and F. Amigoni, "Intractability of time-optimal multirobot path planning on 2d grid graphs with holes," *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 1941–1947, 2017.

[47] B. Nebel, "On the computational complexity of multi-agent pathfinding on directed graphs," in *Proceedings of the 30th International Conference on Automated Planning and Scheduling (ICAPS 2020)*, 2020, pp. 212–216.

[48] T. Geft and D. Halperin, "Refined hardness of distance-optimal multi-agent path finding," in *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2022, pp. 481–488.

[49] T. Geft, "Fine-grained complexity analysis of multi-agent path finding on 2d grids," in *Proceedings of the 16th International Symposium on Combinatorial Search (SoCS)*, 2023, pp. 20–28.

[50] H. Ma and S. Koenig, "Optimal target assignment and path finding for teams of agents," *arXiv preprint arXiv:1612.05693*, 2016.

[51] M. Goldenberg *et al.*, "Enhanced partial expansion a," *Journal of Artificial Intelligence Research*, vol. 50, pp. 141–187, 2014.

[52] G. Sharon, R. Stern, M. Goldenberg, and A. Felner, "The increasing cost tree search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 195, pp. 470–495, 2013.

[53] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.

[54] P. Surynek, "Towards optimal cooperative path planning in hard setups through satisfiability solving," in *Pacific Rim International Conference on Artificial Intelligence*, Springer, 2012, pp. 564–576.

[55] E. Erdem, D. G. Kisa, U. Oztok, and P. Schüller, "A general formal framework for pathfinding problems with multiple agents," in *Twenty-Seventh AAAI Conference on Artificial Intelligence*, 2013.

[56] R. J. Luna and K. E. Bekris, "Push and swap: Fast cooperative path-finding with completeness guarantees," in *Twenty-Second International Joint Conference on Artificial Intelligence*, 2011.

[57] B. De Wilde, A. W. Ter Mors, and C. Witteveen, "Push and rotate: A complete multi-agent pathfinding algorithm," *Journal of Artificial Intelligence Research*, vol. 51, pp. 443–492, 2014.

[58] D. Silver, "Cooperative pathfinding.," *Aiide*, vol. 1, pp. 117–122, 2005.

[59] P. Surynek, "A novel approach to path planning for multiple robots in bi-connected graphs," in *2009 IEEE International Conference on Robotics and Automation*, IEEE, 2009, pp. 3613–3619.

[60] M. Barer, G. Sharon, R. Stern, and A. Felner, "Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem," in *Seventh Annual Symposium on Combinatorial Search*, 2014.

[61] J. Li, W. Ruml, and S. Koenig, "Eecbs: A bounded-suboptimal search for multi-agent path finding," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2021.

[62] S. D. Han and J. Yu, "Ddm: Fast near-optimal multi-robot path planning using diversified-path and optimal sub-problem solution database heuristics," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1350–1357, 2020.

[63] K. Okumura, M. Machida, X. Défago, and Y. Tamura, "Priority inheritance with backtracking for iterative multi-agent path finding," in *IJCAI*, 2019.

[64] J. Li, Z. Chen, D. Harabor, P. Stuckey, and S. Koenig, "Anytime multi-agent path finding via large neighborhood search," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2021.

[65] K. Okumura, "Lacam: Search-based algorithm for quick multi-agent pathfinding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 11 655–11 662.

[66] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig, "Searching with consistent prioritization for multi-agent path finding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 7643–7650.

[67] M. Damani, Z. Luo, E. Wenzel, and G. Sartoretti, "Primal _2: Pathfinding via reinforcement and imitation multi-agent learning-lifelong," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2666–2673, 2021.

[68] G. Sartoretti *et al.*, "Primal: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 2378–2385, 2019.

[69] Q. Li, W. Lin, Z. Liu, and A. Prorok, "Message-aware graph attention networks for large-scale multi-robot path planning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5533–5540, 2021.

[70] Z. Ma, Y. Luo, and H. Ma, "Distributed heuristic multi-agent path finding with communication," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 3854–3859.

[71] Z. Ma, Y. Luo, and J. Pan, "Learning selective communication for multi-agent path finding," in *IEEE Robotics and Automation Letters*, vol. 7, 2022, pp. 1455–1462.

[72] S. D. Han, E. J. Rodriguez, and J. Yu, "Sear: A polynomial-time multi-robot path planning algorithm with expected constant-factor optimality guarantee," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2018, pp. 1–9.

[73] T. Guo, S. D. Han, and J. Yu, "Spatial and temporal splitting heuristics for multi-robot motion planning," in *IEEE International Conference on Robotics and Automation*, 2021.

[74] T. Guo and J. Yu, "Targeted parallelization of conflict-based search for multi-robot path planning," *arXiv preprint arXiv:2402.11768*, 2024.

[75] M. Čáp, J. Vokřínek, and A. Kleiner, "Complete decentralized method for on-line multi-robot trajectory planning in well-formed infrastructures," in *Twenty-Fifth International Conference on Automated Planning and Scheduling*, 2015.

[76] M. Ferreira *et al.*, "Self-automated parking lots for autonomous vehicles based on vehicular ad hoc networking," in *2014 IEEE Intelligent Vehicles Symposium Proceedings*, IEEE, 2014, pp. 472–479.

[77] J. Timpner, S. Friedrichs, J. Van Balen, and L. Wolf, "K-stacks: High-density valet parking for automated vehicles," in *2015 IEEE Intelligent Vehicles Symposium (IV)*, IEEE, 2015, pp. 895–900.

[78] M. Nourinejad, S. Bahrami, and M. J. Roorda, "Designing parking facilities for autonomous vehicles," *Transportation Research Part B: Methodological*, vol. 109, pp. 110–127, 2018.

[79] K. R. Gue and B.-S. Kim, "Puzzle-based storage systems," *Naval Research Logistics (NRL)*, vol. 54, 2007.

[80] D. Ratner and M. K. Warmuth, "Finding a shortest solution for the n$\times$ n extension of the 15-puzzle is intractable.," in *AAAI*, 1986, pp. 168–172.

[81] H. Yu, Y. Yu, and M. de Koster, "Optimal algorithms for scheduling multiple simultaneously movable empty cells to retrieve a load in puzzle-based storage systems," *Available at SSRN 3506480*, 2016.

[82] A. Okoso, K. Otaki, S. Koide, and T. Nishi, "High density automated valet parking via multi-agent path finding," in *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, IEEE, 2022, pp. 2146–2153.

[83] T. Guo, S. W. Feng, and J. Yu, "Polynomial time near-time-optimal multi-robot path planning in three dimensions with applications to large-scale uav coordination," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 10 074–10 080.

[84] T. Guo and J. Yu, "Toward efficient physical and algorithmic design of automated garages," *arXiv preprint arXiv:2302.01305*, 2023.

[85] L. Wen, Y. Liu, and H. Li, "Cl-mapf: Multi-agent path finding for car-like robots with kinematic and spatiotemporal constraints," *Robotics and Autonomous Systems*, vol. 150, p. 103 997, 2022.

[86] J. Li, M. Ran, and L. Xie, "Efficient trajectory planning for multiple non-holonomic mobile robots via prioritized trajectory optimization," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 405–412, 2020.

[87] D. Le and E. Plaku, "Multi-robot motion planning with dynamics via coordinated sampling-based expansion guided by multi-agent search," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1868–1875, 2019.

[88] J. Alonso-Mora, A. Breitenmoser, P. Beardsley, and R. Siegwart, "Reciprocal collision avoidance for multiple car-like robots," in *2012 IEEE International Conference on Robotics and Automation*, IEEE, 2012, pp. 360–366.

[89] J. Alonso-Mora, P. Beardsley, and R. Siegwart, "Cooperative collision avoidance for nonholonomic robots," *IEEE Transactions on Robotics*, vol. 34, no. 2, pp. 404–420, 2018.

[90] S. Poduri and G. S. Sukhatme, "Constrained coverage for mobile sensor networks," in *Proceedings IEEE International Conference on Robotics & Automation*, 2004.

[91] D. Rus, B. Donald, and J. Jennings, "Moving furniture with teams of autonomous robots," in *Proceedings IEEE/RSJ International Conference on Intelligent Robots & Systems*, 1995, pp. 235–242.

[92] W. Hönig, J. A. Preiss, T. S. Kumar, G. S. Sukhatme, and N. Ayanian, "Trajectory planning for quadrotor swarms," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 856–869, 2018.

[93] M. Szegedy and J. Yu, "Rubik tables and object rearrangement," *The International Journal of Robotics Research*, vol. 42, no. 6, pp. 459–472, 2023.

[94] E. Lam, P. Le Bodic, D. D. Harabor, and P. J. Stuckey, "Branch-and-cut-and-price for multi-agent pathfinding.," in *IJCAI*, 2019, pp. 1289–1296.

[95] P. Hall, "On representatives of subsets," in *Classic Papers in Combinatorics*, Springer, 2009, pp. 58–62.

[96] D. Bitton, D. J. DeWitt, D. K. Hsaio, and J. Menon, "A taxonomy of parallel sorting," *ACM Computing Surveys (CSUR)*, vol. 16, no. 3, pp. 287–318, 1984.

[97] T. Leighton and P. Shor, "Tight bounds for minimax grid matching with applications to the average case analysis of algorithms," *Combinatorica*, vol. 9, no. 2, pp. 161–187, 1989.

[98] J. Yu and M. LaValle, "Distance optimal formation control on graphs with a tight convergence time guarantee," in *2012 IEEE 51st IEEE Conference on Decision and Control (CDC)*, 2012, pp. 4023–4028.

[99] H. Ma and S. Koenig, "Optimal target assignment and path finding for teams of agents," in *AAMAS*, 2016.

[100] A. Goel, M. Kapralov, and S. Khanna, "Perfect matchings in o(n\logn) time in regular bipartite graphs," *SIAM Journal on Computing*, vol. 42, no. 3, pp. 1392–1404, 2013.

[101] L. R. Ford and D. R. Fulkerson, "Maximal flow through a network," *Canadian journal of Mathematics*, vol. 8, pp. 399–404, 1956.

[102] R. Burkard, M. Dell'Amico, and S. Martello, *Assignment problems: revised reprint*. SIAM, 2012.

[103] H. Ma, T. S. Kumar, and S. Koenig, "Multi-agent path finding with delay probabilities," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, 2017.

[104] K. Okumura, Y. Tamura, and X. Défago, "Iterative refinement for real-time multi-robot path planning," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2021, pp. 9690–9697.

[105] J. Yu and S. M. LaValle, "Multi-agent path planning and network flow," in *Algorithmic foundations of robotics X*, Springer, 2013, pp. 157–173.

[106] J. Yu and M. LaValle, "Distance optimal formation control on graphs with a tight convergence time guarantee," in *2012 IEEE 51st IEEE Conference on Decision and Control (CDC)*, IEEE, 2012, pp. 4023–4028.

[107] P. W. Shor and J. E. Yukich, "Minimax grid matching and empirical measures," *The Annals of Probability*, vol. 19, no. 3, pp. 1338–1348, 1991.

[108] Gurobi Optimization, LLC, *Gurobi Optimizer Reference Manual*, 2021.

[109] L. Perron and V. Furnon, *Or-tools*, version 7.2, Google.

[110] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding in large-scale warehouses.," in *AAMAS*, 2020, pp. 1898–1900.

[111] H. Ma, J. Li, T. K. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding for online pickup and delivery tasks," in *AAMAS*, 2017.

[112] T. Guo and J. Yu, "Toward efficient physical and algorithmic design of automated garages," *arXiv preprint arXiv:2302.01305*, 2023.

[113] K. Azadeh, R. De Koster, and D. Roy, "Robotized and automated warehouse systems: Review and recent developments," *Transportation Science*, vol. 53, no. 4, pp. 917–945, 2019.

[114] F. Caron, G. Marchet, and A. Perego, "Optimal layout in low-level picker-to-part systems," *International Journal of Production Research*, vol. 38, no. 1, pp. 101–117, 2000.

[115] T. Guo and J. Yu, "Sub-1.5 Time-Optimal Multi-Robot Path Planning on Grids in Polynomial Time," in *Proceedings of Robotics: Science and Systems*, New York City, NY, USA, Jun. 2022.

[116] T. Guo, S. W. Feng, M. Szegedy, and J. Yu, "Rubik tables, stack rearrangement, and multi-robot path planning," in *2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, IEEE, 2022, pp. 1–4.

[117] Z. Yan, N. Jouandeau, and A. A. Cherif, "A survey and analysis of multi-robot coordination," *International Journal of Advanced Robotic Systems*, vol. 10, no. 12, p. 399, 2013.

[118] W. Sheng, Q. Yang, J. Tan, and N. Xi, "Distributed multi-robot coordination in area exploration," *Robotics and autonomous systems*, vol. 54, no. 12, pp. 945–955, 2006.

[119] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[120] R. E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM journal on computing*, vol. 1, no. 2, pp. 146–160, 1972.

[121] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

[122] N. Sturtevant, "Benchmarks for grid-based pathfinding," *Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 2, pp. 144–148, 2012.

[123] Yeefung Automation, *First AGV Robotic Parking System in Hong Kong developed by Yeefung*, 2021.

[124] A. K. Nayak, H. Akash, and G. Prakash, "Robotic valet parking system," in *2013 Texas Instruments India Educators' Conference*, IEEE, 2013, pp. 311–315.

[125] T. Guo, S. W. Feng, and J. Yu, "Polynomial Time Near-Time-Optimal Multi-Robot Path Planning in Three Dimensions with Applications to Large-Scale UAV Coordination," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[126] Q. Li, F. Gama, A. Ribeiro, and A. Prorok, "Graph neural networks for decentralized multi-robot path planning," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2020, pp. 11 785–11 792.

[127] Y. Ouyang, B. Li, Y. Zhang, T. Acarman, Y. Guo, and T. Zhang, "Fast and optimal trajectory planning for multiple vehicles in a nonconvex and cluttered environment: Benchmarks, methodology, and experiments," in *2022 International Conference on Robotics and Automation (ICRA)*, IEEE, 2022, pp. 10 746–10 752.

[128] J. A. Reeds and L. A. Shepp, "Optimal paths for a car that goes both forwards and backwards," *Pacific journal of mathematics*, vol. 71, no. 2, pp. 399–409, 1977.

[129] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Practical search techniques in path planning for autonomous driving," *Ann Arbor*, vol. 1001, no. 48105, pp. 18–80, 2008.

[130] J. P. Van Den Berg and M. H. Overmars, "Prioritized motion planning for multiple robots," in *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2005, pp. 430–435.

[131] H. Tang *et al.*, "# Exploration: A study of count-based exploration for deep reinforcement learning," *Advances in neural information processing systems*, vol. 30, 2017.

[132] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding in large-scale warehouses," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 11 272–11 281.

[133] S. D. Han and J. Yu, "Optimizing space utilization for more effective multi-robot path planning," in *2022 International Conference on Robotics and Automation (ICRA)*, IEEE, 2022, pp. 10 709–10 715.

[134] J. Yu, S. D. Han, W. N. Tang, and D. Rus, "A portable, 3d-printing enabled multi-vehicle platform for robotics research and education," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2017, pp. 1475–1480.

[135] T. Guo and J. Yu, "Expected 1.x makespan-optimal multi-agent path finding on grid graphs in low polynomial time," *Journal of Artificial Intelligence Research*, vol. 81, pp. 443–479, 2024.

[136] T. Guo and J. Yu, "Well-connected set and its application to multi-robot path planning," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[137] T. Guo and J. Yu, "Decentralized lifelong path planning for multiple ackerman car-like robots," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.